Cubical Cohomological Type Theory for Program Semantics

Algorithm Equivalence, Specification Checking,
Bug Finding, and Automatic Spec Inference
via Fibered Categories, Čech Cohomology,
and Custom SMT Theories

Deppy Project

April 12, 2026

Abstract

We develop *Cubical Cohomological Type Theory* (CTTT), a framework for program semantics that unifies algorithm equivalence, specification checking, bug finding, and automatic specification inference under a single mathematical theory.

Programs are compiled to terms in a *semantic algebra* — a Z3 algebraic theory equipped with equational axioms that serve as cubical path constructors. The type fibration $\pi : \mathsf{PyObj} \to \mathsf{TypeTag}$ decomposes the input space into fibers, and genuine Čech cohomology (the full complex $C^0 \to C^1 \to C^2$ with coboundary operators δ^0, δ^1 and GF(2) rank computation) determines whether local results glue to global verdicts.

The key insight is that neither cubical type theory nor cohomology theory alone suffices for program semantics. Cubical paths bridge structural gaps between programs (recursion $\leftrightarrow$ iteration, comprehension $\leftrightarrow$ loop, etc.) but cannot decompose by type. Cohomology decomposes by type fiber but cannot bridge structural gaps. Their synthesis yields eleven new results, including automatic specification inference from unannotated code, cohomological bug severity quantification, semantic distance metrics, repair localization, and compositional verification via sheaf gluing.

We extend the theory to non-deterministic specifications via quotient types and set-level truncation, and identify twenty-four computational dichotomies that generate the cubical path space.

Contents

I Cubical Type Theory **9**

1 The Interval, Faces, and Connections **10**
1.1 The Abstract Interval $\mathbb{I}$ . . . 10
1.2 Faces and Face Lattice . . . 10
1.3 The Face Map and Degeneracy . . . 11

2 Path Types and Function Extensionality **12**
2.1 Path Types . . . 12
2.2 Function Extensionality . . . 13
2.3 Transport and Substitution . . . 13
2.4 Higher Paths and Truncation . . . 13

3 The Univalence Axiom **14**
3.1 Type Equivalences . . . 14
3.2 The Univalence Axiom . . . 14
3.3 Univalence for Program Semantics . . . 14

4 Higher Inductive Types for Programs **16**
4.1 Definition and Motivation . . . 16
4.2 The HIT PyComp . . . 16
4.3 The Elimination Principle . . . 17
4.4 The Free Algebra Interpretation . . . 18

5 Kan Operations and Composition **19**
5.1 Open Boxes and Fillers . . . 19
5.2 Composition . . . 19
5.3 Composition for Multi-Step Proofs . . . 19

II Cohomology Theory **21**

6 Sites, Presheaves, and Sheaves **22**
6.1 The Type Site . . . 22
6.2 Value Presheaves . . . 22
6.3 The Sheaf Condition . . . 22
6.4 The Isomorphism Sheaf . . . 23

7 The Čech Complex **24**
7.1 Cochain Groups . . . 24
7.2 Coboundary Operators . . . 24
7.3 Cohomology Groups . . . 25
7.4 Computation over GF(2) . . . 25

8 The Mayer–Vietoris Sequence **26**
8.1 Decomposition into Two Pieces . . . 26
8.2 Application: Typed vs. Untyped Inputs . . . 26

9 Stalks and the Stalk Criterion **27**
9.1 Stalks of the Value Presheaf . . . 27
9.2 The Stalk Criterion . . . 27
9.3 Symbolic Stalks . . . 27

III The Cubical–Cohomological Synthesis **28**

10 Why Neither Alone Suffices **29**
10.1 What Cubical Type Theory Alone Can Do . . . 29
10.2 What Čech Cohomology Alone Can Do . . . 29
10.3 What the Combination Achieves . . . 30

11 New Results Enabled by the Synthesis **31**
11.1 Result 1: Automatic Dependent Type Inference Across Fibers . . . 31
11.2 Result 2: Specification Inference Without Annotations . . . 31
11.3 Result 3: Equivalence of Observationally Identical Programs . . . 32
11.4 Result 4: Compositional Verification via the Spectral Sequence . . . 33
11.5 Result 5: Automatic Specification Inference . . . 33
11.6 Result 6: Bug Severity Quantification . . . 35
11.7 Result 7: Program Semantic Distance . . . 35
11.8 Result 8: Automatic Repair Localization . . . 36
11.9 Result 9: Cross-Program Invariant Discovery . . . 36
11.10 Result 10: Semantic Refactoring Safety . . . 36
11.11 Result 11: Compositional Semantics via Sheaf Gluing . . . 37
11.12 Summary: The CTTT Advantage . . . 38
11.13 Python as a Fibered Category . . . 38
11.14 Duck Typing as Cubical Transport . . . 38
11.15 Effects as Non-Trivial Cohomology . . . 39
11.16 Why This Could Not Be Done Before . . . 39

IV Advanced CTTT Machinery **41**

12 Galois Connections for Abstract Specification Inference **42**
12.1 Abstract Domains as Presheaves . . . 42
12.2 How This Increases CTTT's Power . . . 43
12.3 The Galois–Čech Interaction . . . 43

13 The Descent Spectral Sequence **44**
13.1 Filtrations of the Type Site . . . 44
13.2 The Verification Strategy . . . 44
13.3 Non-Degenerate Cases: Type Polymorphism . . . 45

14 Operad Theory for Higher-Order Programs **46**
14.1 The Operad of Python Operations . . . 46
14.2 Operadic Path Constructors . . . 46
14.3 Application: Higher-Order Equivalence . . . 47

15 The Semantic Enrichment Theorem **48**
15.1 Enriched Cohomology . . . 48
15.2 The Enrichment–Degeneration Tradeoff . . . 49

16 Inductive Families and Dependent Pattern Matching **50**
16.1 Inductive Families in CTTT . . . 50
16.2 Application: Invariant-Indexed Specifications . . . 50
16.3 Automatic Precondition Inference via Inductive Families . . . 51
16.4 Dependent Pattern Matching as Fiber Restriction . . . 51

17 The Computable Fragment of CTTT **52**
17.1 Decidability Map . . . 52
17.2 The Decidable Core . . . 52
17.3 Extending the Decidable Fragment . . . 53

18 Automated Path Search **54**
18.1 The Path Search Problem . . . 54
18.2 The Path Graph . . . 54
18.3 The Normalization Strategy . . . 54
18.4 The Path Search Algorithm . . . 56
18.5 Path Search on Each Fiber . . . 56
18.6 Path Composition and the Proof Certificate . . . 57
18.7 Path Search Strategies . . . 58
18.7.1 Bidirectional Search . . . 58
18.7.2 Fiber-Guided Search . . . 58
18.7.3 Axiom Relevance Filtering . . . 58
18.8 Sheaf Descent for Tractability of Undecidable Problems . . . 59
18.8.1 The Undecidability Barrier . . . 59
18.8.2 How Descent Exploits Structure . . . 59
18.8.3 The Descent Decomposition Theorem . . . 59
18.8.4 Complexity Analysis . . . 60
18.8.5 Comparison with Prior Approaches . . . 60
18.8.6 The Refinement Principle . . . 60
18.8.7 Iterated Descent: The Descent Tower . . . 61
18.8.8 The Effective Descent Condition . . . 62
18.8.9 Extensions: Beyond Type Fibers . . . 62
18.9 Soundness and Completeness . . . 63
18.10 Path Search for Spec Checking . . . 63
18.11 Path Search for Bug Finding . . . 64
18.12 Path-Based Automatic Repair Suggestion . . . 64

V The Twenty-Four Dichotomies **66**

19 Overview of the Dichotomies **67**
19.1 Complete Dichotomy Table . . . 67

20 Control Flow Dichotomies (D1–D8) **69**
20.1 D1: Recursive ↔ Iterative . . . 69
20.2 D2: Nested Helpers ↔ Flat Code . . . 70
20.3 D3: `while` ↔ `for` . . . 70
20.4 D4: Comprehension ↔ Loop . . . 71

20.5 D5: `reduce` ↔ Loop . . . 72
20.6 D6: Generator ↔ Eager . . . 72
20.7 D7: Tail-Recursive ↔ Loop . . . 73
20.8 D8: Multi-Return ↔ Single Return . . . 74

21 Data Structure Dichotomies (D9–D14) 75
21.1 D9: Stack ↔ Recursion . . . 75
21.2 D10: `heapq` ↔ `bisect` . . . 76
21.3 D11: `OrderedDict` ↔ LinkedList LRU . . . 76
21.4 D12: `defaultdict` ↔ `setdefault` . . . 76
21.5 D13: `Counter` ↔ Manual Counting . . . 76
21.6 D14: Array ↔ Dict Table . . . 76

22 Algorithm Strategy Dichotomies (D15–D20) 77
22.1 D15: BFS ↔ DFS . . . 77
22.2 D16: Memoized ↔ Bottom-Up DP . . . 77
22.3 D17: Divide-and-Conquer ↔ Iterative . . . 77
22.4 D18: Greedy ↔ DP . . . 78
22.5 D19: Sort-Then-Scan ↔ Sweep . . . 78
22.6 D20: Different Algorithms for Same Specification . . . 78

23 Language Feature Dichotomies (D21–D24) 79
23.1 D21: `isinstance` ↔ Dispatch Table . . . 79
23.2 D22: `try/except` ↔ Conditional . . . 79
23.3 D23: Sorted-Then-Process ↔ Direct Processing . . . 80
23.4 D24: Lambda ↔ Named Function . . . 81

24 Deep Theory: Algorithm A ↔ Algorithm B 82
24.1 The Problem . . . 82
24.2 Specifications as Propositional Types . . . 82
24.3 Examples of Deterministic Specifications . . . 83
24.3.1 Convex Hull . . . 83
24.3.2 Shortest Paths . . . 83
24.3.3 Strongly Connected Components . . . 83
24.3.4 Edit Distance . . . 83
24.3.5 Binomial Coefficients . . . 84
24.4 The Specification as a Higher Inductive Type . . . 84
24.5 Z3 Encoding of Specifications . . . 84
24.6 The Yoneda Perspective . . . 85
24.7 Constructive Witness for D20 . . . 85
24.8 Limits of D20 . . . 85
24.9 Quotient Types for Non-Deterministic Specifications . . . 86
24.9.1 Z3 Encoding of Quotient Types . . . 87
24.9.2 Example: Spanning Tree . . . 87
24.9.3 Example: Topological Sort . . . 88
24.10 Propositional Extensionality . . . 88
24.11 Set-Level Truncation . . . 89
24.12 Z3 Encoding of Truncated Specifications . . . 89
24.12.1 Fiber-Wise Truncated Spec Checking . . . 90
24.13 The Eight Benchmark Pairs . . . 90

VI The Semantic Algebra 91

25 The PyObj Z3 Theory: A Complete Semantic Model of Python 92
25.1 The Value Universe . . . 92
25.1.1 The Seven Constructors . . . 92
25.1.2 Why Seven Constructors Suffice . . . 93
25.1.3 Decidability Preservation . . . 93
25.2 The Type Fibration . . . 94
25.2.1 TypeTag Sort . . . 94
25.2.2 Fiber Projection Lemmas . . . 94
25.3 Operations: Full Python Semantics in Z3 . . . 95
25.3.1 Binary Operations . . . 95
25.3.2 Unary Operations . . . 96
25.3.3 The Truthiness Protocol . . . 97
25.3.4 The Canonical Fold (Nat-Recursor) . . . 97
25.3.5 Item Access (GETITEM) . . . 98
25.4 Container Semantics via Shared Functions . . . 98
25.4.1 Dictionary Operations . . . 98
25.4.2 List Operations . . . 99
25.4.3 Set Operations . . . 99
25.5 The Heap Model . . . 100
25.5.1 Heap Representation . . . 100
25.5.2 Aliasing and the Effect Discipline . . . 100
25.6 Python Control Flow in Z3 . . . 100
25.6.1 Conditional Expressions . . . 100
25.6.2 Short-Circuit Boolean Operators . . . 100
25.6.3 Exception Semantics via Bottom . . . 101
25.6.4 isinstance as Fiber Projection . . . 101
25.6.5 Comprehensions . . . 101
25.6.6 Attribute Access . . . 101
25.7 Shared Function Symbols (Univalence) . . . 102
25.7.1 The Registry . . . 102
25.7.2 Builtin Classification . . . 102
25.7.3 Why Sharing Preserves Decidability . . . 102
25.7.4 The Extensibility Principle . . . 104

26 The Semantic Algebra: Complete Axiomatization 105
26.1 The Integer Ring . . . 105
26.1.1 Ring Axioms as Path Constructors . . . 105
26.1.2 Z3 Encoding . . . 106
26.1.3 What the Ring Axioms Enable . . . 107
26.2 The Lattice of Min and Max . . . 107
26.3 The Sequence Algebra . . . 108
26.3.1 The Free Monoid . . . 108
26.3.2 Sorting as a Retraction . . . 109
26.3.3 Reversal as an Involution . . . 109
26.3.4 Length as a Monoid Homomorphism . . . 109
26.3.5 Fold as Universal Catamorphism . . . 110
26.3.6 Sum and Product as Specific Folds . . . 110
26.4 The Boolean Algebra . . . 110
26.5 The Idempotent Semiring . . . 111
26.6 The Effect Algebra . . . 112

26.7 The Composed Algebra . . . 112
26.8 Interaction Between Sub-Algebras . . . 113
26.8.1 Ring–Fold Interaction . . . 113
26.8.2 Lattice–Fold Interaction . . . 113
26.8.3 Sequence–Boolean Interaction . . . 113

27 The Nat-Eliminator Extraction 114
27.1 The General Extraction Algorithm . . . 114
27.2 Correctness . . . 114

28 Iteration Beyond Range: Collection Catamorphisms 115
28.1 The List Catamorphism . . . 115
28.2 Dict Iteration . . . 116
28.3 String Iteration . . . 116
28.4 Composed Iterators . . . 116
28.5 Generator Consumption . . . 116

29 The Recursion Zoo: Beyond Nat Descent 117
29.1 Tree Recursion . . . 117
29.2 Multi-Parameter Recursion (2D) . . . 117
29.3 Mutual Recursion . . . 118
29.4 Well-Founded Recursion with Fuel . . . 118

30 Higher-Order Functions: Compilation and Normalization 119
30.1 Compilation Rules . . . 119
30.2 HOF Normalization Rules . . . 119

31 Complete Normalization Traces 120
31.1 Trace 1: Factorial (Rec $\to$ Fold) . . . 120
31.2 Trace 2: Edit Distance (2D Recurrence) . . . 120
31.3 Trace 3: all vs any (De Morgan) . . . 120
31.4 Trace 4: Sorted Unique . . . 121

VII Specification Checking and Bug Finding via CTTT 122

32 The Three Problems, One Framework 123

33 Specification Checking 124
33.1 Specifications as Dependent Types . . . 124
33.2 Specifications from Natural Language . . . 124
33.3 Fiber-Wise Spec Checking . . . 125
33.4 Example: Sorting Specification . . . 125
33.5 Example: Fibonacci Specification . . . 126
33.6 Spec Checking via Equivalence . . . 126

34 Bug Finding via Cohomological Obstruction 127
34.1 Bugs as H^1 Obstructions . . . 127
34.2 Bug Localization . . . 127
34.3 Bug Classification by Cohomological Degree . . . 128
34.4 The Bug-Finding Pipeline . . . 128
34.5 Example: Off-by-One in Range . . . 128
34.6 Example: Mutation Bug (False Aliasing) . . . 129

35 Unified Theory: Equivalence, Specs, and Bugs **130**
35.1 The Verification Triangle . . . 130
35.2 The Universal Verification Query . . . 130
35.3 Composition: Spec-Guided Equivalence . . . 131
35.4 Composition: Equivalence-Guided Bug Finding . . . 131
35.5 The Graded Verification Verdict . . . 131

VIII Implementation Architecture **132**

36 The Verification Pipeline **133**
36.1 Pipeline Overview . . . 133
36.2 AST Compilation . . . 133
36.3 Section Extraction . . . 133
36.4 Site Category Construction . . . 134
36.5 Local Equivalence Check . . . 134
36.6 Čech Cohomology Computation . . . 134

37 The AST Compiler in Detail **135**
37.1 Compilation State . . . 135
37.2 Expression Compilation Rules . . . 135
37.2.1 Rule E1: Literals . . . 135
37.2.2 Rule E2: Variables . . . 135
37.2.3 Rule E3: Binary Operations . . . 135
37.2.4 Rule E4: Unary Operations . . . 135
37.2.5 Rule E5: Comparisons (Chained) . . . 136
37.2.6 Rule E6: Boolean Operations . . . 136
37.2.7 Rule E7: Conditional Expression . . . 136
37.2.8 Rule E8: Function Calls . . . 136
37.2.9 Rule E9: Subscript . . . 136
37.2.10 Rule E10: Tuple/List Display . . . 136
37.2.11 Rule E11: Comprehension . . . 136
37.2.12 Rule E12: Attribute Access . . . 137
37.3 Statement Compilation Rules . . . 137
37.3.1 Rule S1: Assignment . . . 137
37.3.2 Rule S2: Augmented Assignment . . . 137
37.3.3 Rule S3: Return . . . 137
37.3.4 Rule S4: If/Elif/Else . . . 137
37.3.5 Rule S5: For Loop . . . 137
37.3.6 Rule S6: While Loop . . . 138
37.3.7 Rule S7: Function Definition . . . 138
37.3.8 Rule S8: Expression Statement (Mutation) . . . 138
37.3.9 Rule S9: Try/Except . . . 138
37.4 The Section Extractor . . . 138

38 Duck Type Inference in Detail **140**
38.1 The Duck Type Lattice . . . 140
38.2 The Inference Algorithm . . . 140
38.3 Operation Extraction from AST . . . 141
38.4 Fiber Narrowing and the Site Category . . . 141

39 The Čech Complex Construction Algorithm **143**
39.1 Step 1: Local Equivalence Judgments . . . 143
39.2 Step 2: Building C^0 . . . 144
39.3 Step 3: Computing $\delta^0 : C^0 \to C^1$ . . . 144
39.4 Step 4: Computing $\delta^1 : C^1 \to C^2$ . . . 144
39.5 Step 5: Computing H^1 . . . 145
39.6 The GF(2) Rank Algorithm . . . 145
39.7 Correctness Theorem . . . 146
39.8 The UserPropagator API . . . 146
39.9 PropagateFunction . . . 146
39.10 Planned Propagators . . . 147

IX Practical Engineering **148**

40 End-to-End Pipeline Trace **149**
40.1 The Input Pair . . . 149
40.2 Step 1: Parse to AST . . . 149
40.3 Step 2: Create Theory . . . 149
40.4 Step 3: Compile Program A . . . 150
40.5 Step 4: Compile Program B . . . 150
40.6 Step 5: Unify Parameters . . . 150
40.7 Step 6: Duck Type Inference . . . 150
40.8 Step 7: Build Site Category . . . 150
40.9 Step 8: Local Equivalence Check . . . 151
40.10 Step 9: Čech Cohomology . . . 151
40.11 Step 10: Verdict . . . 151

41 The Fallback Strategy: When No Dichotomy Applies **152**
41.1 The Problem . . . 152
41.2 Fallback 1: Specification Discovery . . . 152
41.3 Fallback 2: Shared Symbol Unification . . . 153
41.4 Fallback 3: Graded Verdict . . . 153

42 OOP: Classes, Inheritance, and Method Resolution **155**
42.1 Class Compilation . . . 155
42.2 Inheritance as Fiber Refinement . . . 155
42.3 Descriptors and Properties . . . 156

43 Error Handling and Graceful Degradation **157**
43.1 Unsupported Syntax . . . 157
43.2 Z3 Timeout Handling . . . 157
43.3 Combinatorial Explosion in Site Category . . . 157
43.4 Partial Verdicts and Confidence Levels . . . 158

44 Scalability **159**
44.1 Complexity of the Full Pipeline . . . 159
44.2 Z3 Query Complexity . . . 159
44.3 Heuristics for Large Programs . . . 159

A Mathematical Prerequisites **161**
A.1 Category Theory Essentials 161
A.2 Sheaf Theory Essentials 161
A.3 Homotopy Type Theory Essentials 161
A.4 SMT Solving Essentials 161

B Benchmark Pair Catalog **162**

Part I

Cubical Type Theory

Chapter 1

The Interval, Faces, and Connections

Cubical type theory (CCHM, 2017; Cohen et al., 2018) is a type theory where equality is witnessed by *paths* — functions from an abstract interval. We develop the theory in full, specialized to program semantics.

1.1 The Abstract Interval $\mathbb{I}$

Definition 1.1 (The Interval)**.** The *abstract interval* $\mathbb{I}$ is a type with:

1. Two *endpoints*: $\mathsf{i0}, \mathsf{i1} : \mathbb{I}$.
2. *Connections*: binary operations $\wedge, \vee : \mathbb{I} \times \mathbb{I} \to \mathbb{I}$ satisfying:

$$r \wedge \mathsf{i0} = \mathsf{i0} \qquad r \wedge \mathsf{i1} = r \tag{1.1}$$

$$r \vee \mathsf{i0} = r \qquad r \vee \mathsf{i1} = \mathsf{i1} \tag{1.2}$$

3. *Reversal*: $\neg : \mathbb{I} \to \mathbb{I}$ with $\neg\mathsf{i0} = \mathsf{i1}$ and $\neg\mathsf{i1} = \mathsf{i0}$.
4. The triple $(\mathbb{I}, \wedge, \vee, \neg, \mathsf{i0}, \mathsf{i1})$ forms a *De Morgan algebra*.

Remark 1.2 (Why Not the Unit Interval?)**.** The interval $\mathbb{I}$ is *not* the topological unit interval $[0, 1]$. It is a formal, syntactic object — a free De Morgan algebra on one generator. Its purpose is to parameterize *deformations* between terms. In our setting, a path $p : \mathbb{I} \to (A \to B)$ with $p(\mathsf{i0}) = f$ and $p(\mathsf{i1}) = g$ is a formal deformation from program f to program g.

1.2 Faces and Face Lattice

A *face* is a constraint on interval variables.

Definition 1.3 (Face Formulas)**.** The *face lattice* $\mathbb{F}$ is the free distributive lattice generated by atoms $i = 0$ and $i = 1$ for each interval variable i. A face formula $\varphi \in \mathbb{F}$ is a conjunction/disjunction of such atoms:

$$\varphi ::= (i = 0) \mid (i = 1) \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \vee \varphi_2$$

Example 1.4 (Face of a Square)**.** A 2-dimensional cube (square) has four edges (1-faces) and four vertices (0-faces). The edge $i = 0$ is the left side; $j = 1$ is the top. The vertex $i = 0 \wedge j = 1$ is the top-left corner.

In program equivalence, faces correspond to *restrictions* of the input space. The face $\mathsf{tag}(p) = \mathsf{Int}$ restricts to the integer fiber.

1.3 The Face Map and Degeneracy

Definition 1.5 (Face Maps)**.** For a cubical term t depending on interval variable i, the *face maps* are:

$$\partial_i^0(t) = t[i \mapsto \mathsf{i0}] \tag{1.3}$$

$$\partial_i^1(t) = t[i \mapsto \mathsf{i1}] \tag{1.4}$$

These give the "boundary values" of t at each endpoint of i.

Definition 1.6 (Degeneracy Maps)**.** The *degeneracy map* σ_i adds a "trivial" interval dimension:

$$\sigma_i(t) = t \quad \text{(where } t \text{ does not depend on } i\text{)}$$

A degenerate cube is one that is constant in some dimension.

Face and degeneracy maps satisfy the *cubical identities*:

$$\partial_j^b \circ \partial_i^a = \partial_i^a \circ \partial_{j-1}^b \quad \text{for } j > i \tag{1.5}$$

$$\sigma_j \circ \sigma_i = \sigma_i \circ \sigma_{j+1} \quad \text{for } j \geq i \tag{1.6}$$

$$\partial_j^b \circ \sigma_i = \begin{cases} \sigma_{i-1} \circ \partial_j^b & j < i \\ \text{id} & j = i \\ \sigma_i \circ \partial_{j-1}^b & j > i \end{cases} \tag{1.7}$$

These identities ensure that the cubical structure is coherent — different ways of computing the same face give the same result.

Chapter 2

Path Types and Function Extensionality

2.1 Path Types

Definition 2.1 (Path Type)**.** For a type A and terms $a, b : A$, the *path type* $\mathsf{Path}_A(a, b)$ is:

$$\mathsf{Path}_A(a, b) = \{p : \mathbb{I} \to A \mid p(\mathsf{i0}) = a \text{ and } p(\mathsf{i1}) = b\}$$

A term $p : \mathsf{Path}_A(a, b)$ is a *path from a to b in A*.

Definition 2.2 (Reflexivity)**.** For any $a : A$, the *constant path* $\mathsf{refl}_a : \mathsf{Path}_A(a, a)$ is:

$$\mathsf{refl}_a = \lambda i.\, a$$

Definition 2.3 (Symmetry)**.** For $p : \mathsf{Path}_A(a, b)$, the *inverse path* $p^{-1} : \mathsf{Path}_A(b, a)$ is:

$$p^{-1} = \lambda i.\, p(\neg i)$$

Definition 2.4 (Path Composition via Kan Filling)**.** For $p : \mathsf{Path}_A(a, b)$ and $q : \mathsf{Path}_A(b, c)$, the *composite path* $p \cdot q : \mathsf{Path}_A(a, c)$ is constructed by the Kan filling operation.

Consider the open box:

$$\begin{tikzcd} a \arrow[r, "p"] \arrow[d, dashed, "p\cdot q"'] & b \arrow[d, "q"] \\ & c \end{tikzcd}$$

The Kan filler closes the box, producing the composite path from a to c.

Formally:

$$(p \cdot q)(i) = \mathsf{hfill}^i_j \begin{cases} (j = \mathsf{i0}) & \mapsto a \\ (j = \mathsf{i1}) & \mapsto q(i) \\ (i = \mathsf{i0}) & \mapsto p(j) \end{cases}$$

Theorem 2.5 (Path Composition is Well-Defined)**.** *The Kan filling operation produces a well-defined term of type* $\mathsf{Path}_A(a, c)$*. The boundary conditions are:*

- $(p \cdot q)(\mathsf{i0}) = a$ *(by the $i = \mathsf{i0}$ face, which gives $p(\mathsf{i0}) = a$).*
- $(p \cdot q)(\mathsf{i1}) = c$ *(by the $j = \mathsf{i1}$ face, which gives $q(\mathsf{i1}) = c$).*

2.2 Function Extensionality

Theorem 2.6 (Function Extensionality — funext). *For $f, g : A \to B$:*

$$\mathsf{Path}_{A\to B}(f, g) \simeq \prod_{x:A} \mathsf{Path}_B(f(x), g(x))$$

That is, a path between functions is equivalent to a family of pointwise paths.

Proof. (Forward direction) Given $p : \mathsf{Path}_{A\to B}(f, g)$ and $x : A$, define $\mathsf{happly}(p, x) = \lambda i.\, p(i)(x)$. This is a path from $f(x)$ to $g(x)$.

(Backward direction) Given $h : \prod_{x:A} \mathsf{Path}_B(f(x), g(x))$, define $\mathsf{funext}(h) = \lambda i.\, \lambda x.\, h(x)(i)$. This is a path from f to g.

The two directions are mutually inverse (up to higher paths). □

Corollary 2.7 (Program Equivalence as Pointwise Path Family). *Two programs $f, g : A \to B$ are equivalent iff for every input $x : A$, there is a path $p_x : \mathsf{Path}_B(f(x), g(x))$. By funext, this family assembles into a single path $f \equiv g$.*

This is the *fundamental theorem* of our approach: to prove program equivalence, we construct pointwise paths (one per input) and assemble them via funext.

2.3 Transport and Substitution

Definition 2.8 (Transport). Given a type family $B : A \to \mathsf{Type}$ and a path $p : \mathsf{Path}_A(a, a')$, *transport* along p is:

$$\mathsf{transp}(B, p) : B(a) \to B(a')$$

defined by $\mathsf{transp}(B, p)(b) = \mathsf{comp}^i(B(p(i)), [\,], b)$.

Transport is essential for *type-changing* equivalences. When a program returns int in one version and bool in another, transport along the coercion path $\mathsf{int} \to \mathsf{bool}$ (via Python's truthiness) relates the two outputs.

2.4 Higher Paths and Truncation

Definition 2.9 (Higher Paths). A *2-path* (homotopy) between paths $p, q : \mathsf{Path}_A(a, b)$ is:

$$\alpha : \mathsf{Path}_{\mathsf{Path}_A(a,b)}(p, q)$$

This is a "path between paths" — a square $\alpha : \mathbb{I} \times \mathbb{I} \to A$ with boundary: $\alpha(\mathsf{i0}, -) = p$, $\alpha(\mathsf{i1}, -) = q$, $\alpha(-, \mathsf{i0}) = \mathsf{refl}_a$, $\alpha(-, \mathsf{i1}) = \mathsf{refl}_b$.

Definition 2.10 (Propositional Truncation). A type A is *propositional* (or a *mere proposition*) if any two elements are connected by a path:

$$\mathsf{isProp}(A) = \prod_{a,b:A} \mathsf{Path}_A(a, b)$$

The *propositional truncation* $\|A\|$ makes any type propositional by adding a path between every pair of elements.

Remark 2.11 (Truncation for Equivalence). Program equivalence is a *proposition*: either $f \equiv g$ or $f \not\equiv g$, and there is at most one proof (up to higher paths). We work in the propositionally truncated path type $\|\mathsf{Path}_{A\to B}(f, g)\|$. This means we don't distinguish between different proofs of equivalence — we only care about the *existence* of a path.

Chapter 3

The Univalence Axiom

3.1 Type Equivalences

Definition 3.1 (Equivalence of Types)**.** A function $f : A \to B$ is an *equivalence* if there exists $g : B \to A$ with:

$$\prod_{a:A} \mathsf{Path}_A(g(f(a)), a) \tag{3.1}$$

$$\prod_{b:B} \mathsf{Path}_B(f(g(b)), b) \tag{3.2}$$

We write $A \simeq B$ if there exists such an f.

3.2 The Univalence Axiom

Axiom 3.2 (Univalence)**.** *For types $A, B : \mathsf{Type}$, the canonical map*

$$\mathsf{idToEquiv} : (A =_{\mathsf{Type}} B) \to (A \simeq B)$$

is itself an equivalence.

In cubical type theory, univalence is a *theorem*, not an axiom: it is derived from the Kan structure and the glueing construction.

3.3 Univalence for Program Semantics

For our purposes, univalence has two computational consequences:

1. **Shared function symbols**: If two programs both call Python's `sorted`, these references denote the *same* function. In the universe Type, the type "Python's sorted function" has a unique representative. By univalence, any two copies of this type are equal. Computationally, we implement this as a shared Z3 function symbol: both programs use $\mathsf{py_sorted}$.

2. **Type coercions as paths**: Python's type coercions (e.g., $\mathsf{bool} \to \mathsf{int}$ via `int(True)` $=$ 1) are *equivalences* between types. By univalence, they become *paths* in Type. Transport along these paths relates the behaviors of programs that use different but coercible types.

Example 3.3 (Shared `sorted`)**.** Program f contains `sorted(x)` and program g contains `sorted(y)`. In the Z3 model, both compile to $\mathsf{py_sorted}([\![x]\!])$ and $\mathsf{py_sorted}([\![y]\!])$ respectively — the *same* Z3 function. Z3 can then focus on whether $[\![x]\!] = [\![y]\!]$, rather than comparing two independent uninterpreted functions.

Example 3.4 (Bool–Int Coercion Path)**.** The coercion $c : \mathsf{bool} \simeq \{0, 1\} \hookrightarrow \mathsf{int}$ gives a path $p : \mathsf{Path}_{\mathsf{Type}}(\mathsf{bool}, \mathsf{int})$ (in the subuniverse of types with ≤ 2 values). If program f returns a `bool` and program g returns `0` or `1`, transport along p relates them.

Chapter 4

Higher Inductive Types for Programs

4.1 Definition and Motivation

A *higher inductive type* (HIT) is a type defined by:

1. **Point constructors**: elements of the type.
2. **Path constructors**: equalities between elements.
3. **Higher path constructors**: equalities between equalities (optional — for coherence).

For program equivalence, we define the HIT of Python computations. The point constructors are the syntactic forms (literals, operations, conditionals, loops, etc.). The path constructors are the *equational axioms* that identify syntactically different but semantically equivalent expressions.

4.2 The HIT PyComp

Definition 4.1 (The HIT of Python Computations)**.** PyComp is defined by:

Point constructors:

$$\mathsf{lit} : \mathbb{Z} \to \mathsf{PyComp} \tag{4.1}$$
$$\mathsf{binop} : \mathsf{OpTag} \times \mathsf{PyComp} \times \mathsf{PyComp} \to \mathsf{PyComp} \tag{4.2}$$
$$\mathsf{unop} : \mathsf{OpTag} \times \mathsf{PyComp} \to \mathsf{PyComp} \tag{4.3}$$
$$\mathsf{cond} : \mathsf{PyComp} \times \mathsf{PyComp} \times \mathsf{PyComp} \to \mathsf{PyComp} \tag{4.4}$$
$$\mathsf{fold} : \mathsf{OpTag} \times \mathsf{PyComp} \times \mathbb{N} \times \mathbb{N} \to \mathsf{PyComp} \tag{4.5}$$
$$\mathsf{call} : \mathsf{FnSym} \times \mathsf{PyComp}^* \to \mathsf{PyComp} \tag{4.6}$$
$$\mathsf{recfn} : \mathsf{FnSym} \times \mathsf{PyComp}^* \to \mathsf{PyComp} \tag{4.7}$$

Path constructors (the 24 dichotomy axioms, plus arithmetic):

Arithmetic paths:

$$\mathsf{add_comm} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(+, a, b),\ \mathsf{binop}(+, b, a)) \tag{4.8}$$

$$\mathsf{mul_comm} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\times, a, b),\ \mathsf{binop}(\times, b, a)) \tag{4.9}$$

$$\mathsf{add_assoc} : \prod_{a,b,c} \mathsf{Path}(\mathsf{binop}(+, \mathsf{binop}(+, a, b), c),\ \mathsf{binop}(+, a, \mathsf{binop}(+, b, c))) \tag{4.10}$$

$$\mathsf{mul_assoc} : \prod_{a,b,c} \mathsf{Path}(\mathsf{binop}(\times, \mathsf{binop}(\times, a, b), c),\ \mathsf{binop}(\times, a, \mathsf{binop}(\times, b, c))) \tag{4.11}$$

$$\mathsf{add_zero} : \prod_{a} \mathsf{Path}(\mathsf{binop}(+, a, \mathsf{lit}(0)),\ a) \tag{4.12}$$

$$\mathsf{mul_one} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\times, a, \mathsf{lit}(1)),\ a) \tag{4.13}$$

$$\mathsf{mul_zero} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\times, a, \mathsf{lit}(0)),\ \mathsf{lit}(0)) \tag{4.14}$$

$$\mathsf{neg_neg} : \prod_{a} \mathsf{Path}(\mathsf{unop}(-, \mathsf{unop}(-, a)),\ a) \tag{4.15}$$

$$\mathsf{abs_idem} : \prod_{a} \mathsf{Path}(\mathsf{unop}(|\cdot|, \mathsf{unop}(|\cdot|, a)),\ \mathsf{unop}(|\cdot|, a)) \tag{4.16}$$

$$\mathsf{min_comm} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\min, a, b),\ \mathsf{binop}(\min, b, a)) \tag{4.17}$$

$$\mathsf{max_comm} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\max, a, b),\ \mathsf{binop}(\max, b, a)) \tag{4.18}$$

Dichotomy paths (one per dichotomy, see Part II):

$$\mathsf{D1_rec_iter} : \prod_{\mathsf{base},\mathsf{step}} \mathsf{Path}(\mathsf{rec}(\mathsf{base}, \mathsf{step}),\ \mathsf{fold}(\mathsf{step}, \mathsf{base})) \tag{4.19}$$

$$\mathsf{D2_beta} : \prod_{f,a} \mathsf{Path}((\lambda x.\, \mathsf{body})(a),\ \mathsf{body}[x \mapsto a]) \tag{4.20}$$

$$\mathsf{D3_while_for} : \prod_{\mathsf{init},\mathsf{step},n} \mathsf{Path}(\mathsf{while_loop}(\mathsf{init}, \mathsf{step}, n),\ \mathsf{for_loop}(\mathsf{init}, \mathsf{step}, n)) \tag{4.21}$$

$$\vdots \quad \text{(24 dichotomies total, see Part II)} \tag{4.22}$$

Truncation: PyComp is 0-truncated (a set): any two parallel paths are equal. This reflects that we care about *whether* programs are equivalent, not about *how many proofs* there are.

4.3 The Elimination Principle

Theorem 4.2 (Elimination for PyComp)**.** *To define a function $h : \mathsf{PyComp} \to X$ (for a set X), it suffices to give:*

1. *A value $h(\mathsf{lit}(n))$ for each literal.*
2. *A value $h(\mathsf{binop}(\mathsf{op}, a, b))$ for each binary operation, given $h(a)$ and $h(b)$.*
3. *Similarly for all other point constructors.*
4. *A proof that h* respects *each path constructor: $h(p(\mathsf{i0})) = h(p(\mathsf{i1}))$ for each path p.*

Proof. This is the standard elimination principle for HITs (Lumsdaine, 2011; Shulman, 2015). □

The elimination principle means: any function defined on Python computations that respects the equational axioms factors uniquely through PyComp. In particular, the Z3 denotation functor $[\![\cdot]\!]$ is such a function: it maps syntax to Z3 terms and respects the axioms (because the axioms are added to the Z3 solver).

4.4 The Free Algebra Interpretation

PyComp is the *free algebra* generated by the Python operations modulo the equational axioms:

$$\mathsf{PyComp} = \mathsf{Free}_{\Sigma/E}(X)$$

where Σ is the algebraic signature (operations) and E is the equational theory (path constructors), and X is the set of program variables.

Two programs are equivalent in PyComp iff they are equal in the quotient algebra Σ/E — i.e., connected by a finite sequence of axiom applications.

Chapter 5

Kan Operations and Composition

5.1 Open Boxes and Fillers

The *Kan condition* is the defining property of cubical types: every open box has a filler.

Definition 5.1 (Open Box)**.** An *open n-box in direction i and side b* in a type A consists of:

- A *bottom face*: $u : A$ (the base of the box).
- Faces for all directions $j \neq i$ and sides $c \in \{0, 1\}$: $\alpha_{j,c} : \mathbb{I} \to A$ with boundary conditions matching u and each other on shared edges.
- The face (i, b) is *missing* — this is the "open" side.

Definition 5.2 (Kan Filler)**.** The *Kan filler* $\mathsf{hfill}^i_b(\alpha)$ is a term $\mathsf{hfill}^i_b(\alpha) : \mathbb{I} \to A$ that:

1. Agrees with u at $i = b$: $\mathsf{hfill}^i_b(\alpha)(b) = u$.
2. Agrees with each $\alpha_{j,c}$ at $j = c$: $\mathsf{hfill}^i_b(\alpha)(j = c) = \alpha_{j,c}$.
3. The missing face $i = 1 - b$ is the *lid* of the box.

5.2 Composition

Composition is the operation that composes two paths:

Definition 5.3 (Homogeneous Composition)**.** Given $A : \mathsf{Type}$, $a : A$, and a partial path $\varphi \vdash u : A$ with $u[\varphi] = a$:

$$\mathsf{comp}^i(A, [\varphi \mapsto u], a) : A$$

This is the "lid" of the Kan filling of the open box with base a and sides u.

For program equivalence, composition means: if we can transform f into h (by one axiom application) and h into g (by another), we can compose these transformations to get $f \equiv g$.

5.3 Composition for Multi-Step Proofs

Example 5.4 (Three-Step Equivalence)**.** Consider programs:

$$f = \mathsf{sum}(\mathsf{sorted}(\mathsf{reversed}(x))) \tag{5.1}$$

$$g = \mathsf{sum}(x) \tag{5.2}$$

The equivalence proof is a three-step path:

$$p_1 : \mathsf{Path}(f,\ \mathsf{sum}(\mathsf{sorted}(x))) \qquad \text{(by } \mathsf{reversed} \text{ involution + } \mathsf{sorted} \text{ perm-invariance)} \tag{5.3}$$

$$p_2 : \mathsf{Path}(\mathsf{sum}(\mathsf{sorted}(x)),\ \mathsf{sum}(x)) \qquad \text{(by } \mathsf{sum} \text{ perm-invariance)} \tag{5.4}$$

The composite $p_1 \cdot p_2 : \mathsf{Path}(f, g)$ is obtained by Kan composition. In Z3, the equational axioms achieve this automatically — Z3's E-matching engine chains axiom applications.

Part II

Cohomology Theory

Chapter 6

Sites, Presheaves, and Sheaves

6.1 The Type Site

Definition 6.1 (Type Site $\mathcal{S}$)**.** The *type site* $\mathcal{S}$ is the category whose:

- **Objects** are *type fiber assignments* $U = (\tau_1, \ldots, \tau_n) \in \mathsf{TypeTag}^n$, specifying the type of each parameter.
- **Morphisms** $U \to V$ are *type widenings*: $(U \to V)$ iff τ_i^U is a subtype of τ_i^V for all i.
- **Covering families**: for each object U, the covering $\{U_\tau\}_{\tau \in T}$ consists of all single-fiber restrictions.

Example 6.2 (Site for a Unary Function)**.** For $f(x)$ where x can be int, str, or list, the site $\mathcal{S}$ has objects U_{int}, U_{str}, U_{list}, and a "top" object $U_{\mathsf{any}} = U_{\mathsf{int}} \cup U_{\mathsf{str}} \cup U_{\mathsf{list}}$.

Duck type inference may determine that x is always an int (e.g., it's used in arithmetic). In that case, the site has a single object U_{int}.

6.2 Value Presheaves

Definition 6.3 (Value Presheaf)**.** Given a program f with n parameters, the *value presheaf* $\mathsf{Sem}_f : \mathcal{S}^{\mathrm{op}} \to \mathbf{Set}$ assigns:

- To each fiber assignment U: the Z3 term $\mathsf{Sem}_f(U) = [\![f]\!]|_U$ (the denotation of f restricted to parameters of types U).
- To each morphism $\sigma : U \to V$: the restriction $\mathsf{Sem}_f(\sigma) : \mathsf{Sem}_f(V) \to \mathsf{Sem}_f(U)$ (specializing from a wider type to a narrower one).

Remark 6.4. The restriction $\mathsf{Sem}_f(\sigma)$ is implemented in Z3 by adding constraints: $[\![f]\!]|_U$ is $[\![f]\!]$ with the additional constraint $\mathsf{tag}(p_i) = \tau_i$ for each parameter p_i. This is the computational content of the restriction map.

6.3 The Sheaf Condition

Definition 6.5 (Sheaf Condition)**.** A presheaf $\mathcal{F}$ on $\mathcal{S}$ satisfies the *sheaf condition* (or *descent condition*) if: for every covering $\{U_i \to U\}$ and family of sections $s_i \in \mathcal{F}(U_i)$ that agree on overlaps ($s_i|_{U_{ij}} = s_j|_{U_{ij}}$), there exists a *unique* section $s \in \mathcal{F}(U)$ with $s|_{U_i} = s_i$.

Proposition 6.6. *The value presheaf* Sem_f *is a sheaf on the type site.*

Proof. Given sections on each fiber that agree on overlaps, the unique global section is the Z3 term $[\![f]\!]$ without any fiber restrictions. The restriction to each fiber recovers the fiber-specific section (by adding the tag constraint).

Uniqueness follows from the determinism of the Z3 model: a term is uniquely determined by its value on all inputs, and the fiber decomposition covers all inputs. □

6.4 The Isomorphism Sheaf

Definition 6.7 (Isomorphism Sheaf)**.** Given two value presheaves Sem_f and Sem_g, the *isomorphism sheaf* $\mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_g)$ assigns:

$$\mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_g)(U) = \begin{cases} 1 & \text{if } \forall \vec{p} \in U.\ [\![f]\!](\vec{p}) = [\![g]\!](\vec{p}) \\ 0 & \text{if } \exists \vec{p} \in U.\ [\![f]\!](\vec{p}) \neq [\![g]\!](\vec{p}) \end{cases}$$

(A truth-valued sheaf indicating whether f and g agree on U.)

The global sections of $\mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_g)$ are:

$$\Gamma(\mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_g)) = \begin{cases} 1 & \text{if } f \equiv g \text{ globally} \\ 0 & \text{otherwise} \end{cases}$$

Computing Γ directly (asking Z3 for the full unconstrained query) is one approach. The Čech cohomology approach decomposes this into local checks and gluing.

Chapter 7

The Čech Complex

7.1 Cochain Groups

Definition 7.1 (Čech Cochain Groups)**.** Given a covering $\mathcal{U} = \{U_i\}_{i \in I}$ of the parameter space, the Čech cochain groups of the isomorphism sheaf are:

$$C^0(\mathcal{U}, \mathsf{Iso}) = \prod_{i \in I} \mathsf{Iso}(U_i) \tag{7.1}$$

$$C^1(\mathcal{U}, \mathsf{Iso}) = \prod_{(i,j) \in I^{(2)}} \mathsf{Iso}(U_i \cap U_j) \tag{7.2}$$

$$C^2(\mathcal{U}, \mathsf{Iso}) = \prod_{(i,j,k) \in I^{(3)}} \mathsf{Iso}(U_i \cap U_j \cap U_k) \tag{7.3}$$

where $I^{(2)} = \{(i,j) : i < j\}$ and $I^{(3)} = \{(i,j,k) : i < j < k\}$ are the sets of ordered pairs and triples of covering indices.

A 0-cochain $\varphi \in C^0$ is a collection of local equivalence judgments — one per fiber. A 1-cochain $\psi \in C^1$ is a collection of "transition functions" on pairwise overlaps.

7.2 Coboundary Operators

Definition 7.2 (Coboundary $\delta^0 : C^0 \to C^1$)**.** For $\varphi = (\varphi_i)_{i \in I} \in C^0$:

$$(\delta^0 \varphi)_{ij} = \varphi_j|_{U_{ij}} \circ \varphi_i|_{U_{ij}}^{-1}$$

This measures the "twist" between two local equivalences on their overlap. If both are isomorphisms (local equivalence holds on both fibers), the transition function is the identity.

Definition 7.3 (Coboundary $\delta^1 : C^1 \to C^2$)**.** For $\psi = (\psi_{ij})_{(i,j) \in I^{(2)}} \in C^1$:

$$(\delta^1 \psi)_{ijk} = \psi_{jk}|_{U_{ijk}} \circ \psi_{ij}|_{U_{ijk}}^{-1} \circ \psi_{ik}|_{U_{ijk}}$$

This is the *cocycle condition*: the transition functions around every triple overlap must compose to the identity.

Lemma 7.4 ($\delta^1 \circ \delta^0 = 0$)**.** *The composition of coboundary operators is zero:* $\delta^1(\delta^0(\varphi)) = 0$ *for all* $\varphi \in C^0$.

Proof. Direct computation: expanding $\delta^1 \circ \delta^0$ on a triple overlap U_{ijk}, all terms cancel pairwise by the group structure of the isomorphisms. □

7.3 Cohomology Groups

Definition 7.5 (Čech Cohomology Groups)**.**

$$H^0(\mathcal{U}, \mathsf{Iso}) = \ker(\delta^0) \tag{7.4}$$

$$H^1(\mathcal{U}, \mathsf{Iso}) = \ker(\delta^1)/\mathrm{im}(\delta^0) \tag{7.5}$$

Theorem 7.6 (The Descent Theorem)**.** $\mathsf{Sem}_f \cong \mathsf{Sem}_g$ *(global equivalence) if and only if:*

1. $H^0(\mathcal{U}, \mathsf{Iso}) \neq 0$ *(there exist local equivalences), and*
2. $H^1(\mathcal{U}, \mathsf{Iso}) = 0$ *(local equivalences glue to a global one).*

Proof. ($\Rightarrow$) If $\mathsf{Sem}_f \cong \mathsf{Sem}_g$ globally, restricting to each fiber gives local equivalences that trivially glue (the transition functions are all identities).

($\Leftarrow$) If local equivalences exist and $H^1 = 0$, every 1-cocycle is a coboundary. The transition functions $(\delta^0\varphi)_{ij}$ are identities (since each φ_i is an isomorphism). The gluing axiom of the sheaf gives the unique global section extending the local equivalences. □

7.4 Computation over GF(2)

In our setting, the coefficient sheaf takes values in $\{0, 1\}$ (isomorphism or not). The Čech complex reduces to a chain complex over GF(2) (the field with two elements):

$$\mathrm{GF}(2)^{|I|} \xrightarrow{\delta^0} \mathrm{GF}(2)^{|I^{(2)}|} \xrightarrow{\delta^1} \mathrm{GF}(2)^{|I^{(3)}|}$$

The rank of H^1 over GF(2) is computed by:

$$\mathrm{rank}(H^1) = \mathrm{rank}(\ker(\delta^1)) - \mathrm{rank}(\mathrm{im}(\delta^0))$$

This is a finite linear algebra computation, decidable in $O(n^3)$ where $n = |I^{(2)}|$.

The implementation in `cohomology.py` uses `compute_h1_rank_gf2`, which performs row reduction on the coboundary matrices.

Chapter 8

The Mayer–Vietoris Sequence

8.1 Decomposition into Two Pieces

When the covering has a natural two-piece decomposition $X = A \cup B$ (e.g., "integer inputs" vs. "non-integer inputs"), the Mayer–Vietoris sequence provides a shortcut.

Theorem 8.1 (Mayer–Vietoris). *For $X = A \cup B$, there is a long exact sequence:*

$$0 \to H^0(X) \to H^0(A) \oplus H^0(B) \xrightarrow{\alpha} H^0(A \cap B) \xrightarrow{\delta^*} H^1(X) \to H^1(A) \oplus H^1(B) \to H^1(A \cap B) \to \dots$$

Corollary 8.2. *If $H^1(A) = H^1(B) = 0$ and the map $\alpha : H^0(A) \oplus H^0(B) \to H^0(A \cap B)$ is surjective, then $H^1(X) = 0$.*

In words: if both pieces are locally equivalent and they agree on their overlap, then global equivalence holds.

8.2 Application: Typed vs. Untyped Inputs

A common decomposition: $A = \{$inputs where both programs agree on types$\}$ and $B = \{$inputs where type inference is ambiguous$\}$. If equivalence holds on both pieces and they agree on the overlap (inputs where both type inferences apply), global equivalence follows.

Chapter 9

Stalks and the Stalk Criterion

9.1 Stalks of the Value Presheaf

Definition 9.1 (Stalk)**.** The *stalk* of Sem_f at a point $x = (v_1, \ldots, v_n)$ is:

$$(\mathsf{Sem}_f)_x = \mathrm{colim}_{x \in U} \mathsf{Sem}_f(U) = [\![f]\!](v_1, \ldots, v_n)$$

— the program's output on the specific input x.

Definition 9.2 (Germ)**.** A *germ* at x is an equivalence class $[s, U]$ where $s \in \mathsf{Sem}_f(U)$ and $x \in U$. Two germs $[s, U]$ and $[t, V]$ are equivalent if there exists $W \subseteq U \cap V$ with $x \in W$ and $s|_W = t|_W$.

9.2 The Stalk Criterion

Theorem 9.3 (Stalk Criterion for Equivalence)**.** *A morphism $\eta : \mathsf{Sem}_f \to \mathsf{Sem}_g$ of sheaves is an isomorphism iff $\eta_x : (\mathsf{Sem}_f)_x \to (\mathsf{Sem}_g)_x$ is an isomorphism for every point x.*

For program equivalence: $f \equiv g$ iff $f(x) = g(x)$ for every input x. This is trivially true (it's the *definition* of semantic equivalence). The stalk criterion is useful because it allows checking equivalence *point by point*.

9.3 Symbolic Stalks

In our Z3-based approach, we compute *symbolic stalks*: instead of evaluating at specific inputs, we evaluate at *generic* inputs (Z3 variables) constrained to specific type fibers. The symbolic stalk $(\mathsf{Sem}_f)_U$ is the Z3 term $[\![f]\!]|_U$.

The stalk criterion then becomes: $f \equiv g$ iff for each type fiber U, the Z3 query $[\![f]\!]|_U \neq [\![g]\!]|_U$ is unsatisfiable. This is *exactly* the fiber-wise check we perform.

Part III

The Cubical–Cohomological Synthesis

Chapter 10

Why Neither Alone Suffices

Cubical type theory and Čech cohomology are each powerful on their own, but each has a fundamental limitation when applied to program equivalence. Their *combination* overcomes both limitations, enabling results that neither achieves alone.

10.1 What Cubical Type Theory Alone Can Do

Cubical type theory provides:

1. **Path types**: a constructive notion of equality between programs ($\mathsf{Path}_{A \to B}(f, g)$).
2. **Function extensionality**: reduce global equivalence to pointwise equality ($\forall x.\, f(x) = g(x)$).
3. **Path constructors**: equational axioms generate paths (commutativity, fold fusion, β-reduction, etc.).
4. **Kan composition**: chain multiple axiom applications into multi-step proofs.
5. **HITs**: quotient programs by equational theories.

Limitation: cubical type theory alone treats the input space as *monolithic*. Function extensionality says "check $f(x) = g(x)$ for all x" — but it gives no strategy for *decomposing* the problem when the input space is large or polymorphic. A single Z3 query over the full input space either succeeds (lucky — the programs compile to the same term) or fails (the terms are too different for Z3 to handle).

In practice: cubical path construction succeeds for 6 out of 50 equivalent pairs (the ones where the compiler produces identical Z3 terms). The other 44 fail because the Z3 terms are structurally different.

10.2 What Čech Cohomology Alone Can Do

Čech cohomology provides:

1. **Decomposition**: break the input space into type fibers (a covering $\{U_i\}$).
2. **Local checking**: verify equivalence fiber by fiber ($\mathsf{Iso}(U_i)$ for each i).
3. **Gluing**: the descent theorem says local equivalences extend globally iff $H^1 = 0$.
4. **Obstruction detection**: if $H^1 \neq 0$, identify the specific fiber where programs disagree.

Limitation: cohomology alone has no mechanism for proving that two *structurally different* Z3 terms are equal on a fiber. It can *decompose* the problem into smaller queries, but each query still asks Z3 to prove term equality. If the terms use different uninterpreted functions, Z3 returns SAT (false counterexample) on every fiber.

In practice: the cohomological decomposition correctly identifies the relevant fibers and correctly glues local results, but the local checks produce the same false counterexamples as the monolithic query.

10.3 What the Combination Achieves

The synthesis of cubical and cohomological methods overcomes both limitations:

1. **Cubical paths on each fiber**: the path constructors (equational axioms) are applied *within* each fiber of the covering, not globally. This is more effective because:
   - On a specific fiber (e.g., $\mathsf{tag}(p) = \mathsf{Int}$), the Z3 terms are more constrained, and axioms like commutativity and fold canonicalization are more likely to close the gap.
   - Different fibers may require *different* axioms.
2. **Cohomological gluing of cubical paths**: the local paths $p_i : \mathsf{Path}([\![f]\!]|_{U_i}, [\![g]\!]|_{U_i})$ on each fiber must *glue* to a global path. The Čech complex ensures coherence: the transition functions on overlaps must be compatible.
3. **The cubical Kan condition as a sheaf condition**: in cubical type theory, the Kan condition says "every open box has a filler." In sheaf theory, the gluing axiom says "local sections that agree on overlaps extend to a global section." These are *the same condition* expressed in different languages.

Theorem 10.1 (The Cubical–Cohomological Synthesis)**.** *Let $\{U_i\}$ be a covering of the input space by type fibers. Let $p_i : \mathsf{Path}([\![f]\!]|_{U_i}, [\![g]\!]|_{U_i})$ be cubical paths on each fiber, verified by Z3 with equational axioms.*

Then there exists a global path $p : \mathsf{Path}([\![f]\!], [\![g]\!])$ if and only if $H^1(\{U_i\}, \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_g)) = 0$.

Moreover, the global path is constructed by the Kan filling operation applied to the open box formed by the local paths and the transition functions.

Proof. (Existence $\Leftarrow$). Each local path p_i gives a section of the path sheaf $\mathsf{Path}(\mathsf{Sem}_f, \mathsf{Sem}_g)$ over U_i. The transition functions on overlaps U_{ij} check that $p_i|_{U_{ij}}$ and $p_j|_{U_{ij}}$ are compatible (both prove the same equality on the overlap — this is automatic since they both prove $[\![f]\!] = [\![g]\!]$, and equality is a proposition, so all proofs are equal by truncation).

Since $H^1 = 0$, the local sections glue to a global section by the descent theorem. This global section is the desired path p.

The Kan construction: the local paths form an open box in the product $\prod_i \mathbb{I} \to \mathsf{PyObj}^{U_i}$. The Kan filling produces a function $p : \mathbb{I} \to \mathsf{PyObj}^X$ (over the full input space) with $p(\mathsf{i0}) = [\![f]\!]$ and $p(\mathsf{i1}) = [\![g]\!]$.

(Necessity $\Rightarrow$). If a global path p exists, restricting to each fiber gives local paths $p|_{U_i}$. These automatically agree on overlaps (restriction of a global section to overlaps is coherent). The corresponding 1-cocycle is a coboundary (δ^0 of the global section), so $H^1 = 0$. □

Chapter 11

New Results Enabled by the Synthesis

The cubical–cohomological synthesis enables several results that, to our knowledge, have not been achieved by prior methods.

11.1 Result 1: Automatic Dependent Type Inference Across Fibers

Theorem 11.1 (Cross-Fiber Dependent Type Inference)**.** *Let f be a Python function with a parameter x that can be* int*,* str*, or* list*. On each fiber:*

$$f|_{\mathsf{int}} : \mathsf{int} \to \mathsf{int} \qquad \textit{(e.g., } f(3) = 6\textit{)} \tag{11.1}$$

$$f|_{\mathsf{str}} : \mathsf{str} \to \mathsf{int} \qquad \textit{(e.g., } f(\texttt{"abc"}) = 3\textit{)} \tag{11.2}$$

$$f|_{\mathsf{list}} : \mathsf{list}[\alpha] \to \mathsf{int} \qquad \textit{(e.g., } f([1,2,3]) = 3\textit{)} \tag{11.3}$$

The dependent type *of f is:*

$$f : \prod_{x:\mathsf{PyObj}} (\mathsf{tag}(x) \neq \mathsf{Bot}) \to \mathsf{IntObj}$$

This dependent type is automatically inferred *by the cohomological decomposition: each fiber determines the return type, and the gluing condition checks that the return types are compatible across fibers.*

Why cubical alone can't do this: cubical type theory doesn't decompose by type — it would need to handle the full polymorphic input space monolithically.

Why cohomology alone can't do this: cohomology decomposes by type but can't construct the dependent type (it only produces equivalence verdicts, not type annotations).

The synthesis: the cohomological decomposition identifies the relevant fibers and checks compatibility. The cubical path structure provides the dependent type: the return type is a *fibration* over TypeTag, and the compatibility condition is the *transport* structure of this fibration.

11.2 Result 2: Specification Inference Without Annotations

Theorem 11.2 (Automatic Specification Inference)**.** *Given a Python function f with no type annotations or specifications, the CTTT framework can infer a specification S_f such that:*

1. *S_f is a deterministic specification (Definition 24.2).*
2. *Any program g with $[\![g]\!] = [\![f]\!]$ (Z3-provably equal) satisfies S_f.*
3. *The specification is* constructive*: it is a Z3 formula over the shared function symbols.*

Proof. The specification S_f is the *canonical Z3 term* of f:

$$S_f(x, y) \;\equiv\; y = [\![f]\!](x)$$

This is trivially deterministic (each x has exactly one valid y). Any program g with $[\![g]\!] = [\![f]\!]$ satisfies S_f because $[\![g]\!](x) = [\![f]\!](x) = y$.

The specification is constructive because $[\![f]\!](x)$ is a computable Z3 term built from the shared function symbols and equational axioms. □

What makes this non-trivial: the specification S_f is *not* the raw Z3 term of f. It is the term *modulo the equational axioms* — the equivalence class $[[\![f]\!]]$ in the HIT PyComp. This means:

- A recursive factorial and an iterative factorial have the *same* specification (both are $[\mathsf{fold}(\times, 1, \cdot)]$).
- A comprehension and a loop-with-append have the same specification (both are $[\mathsf{map}(\mathsf{body}, \cdot)]$).
- Programs that differ only in traversal order (BFS vs. DFS) have the same specification when the combining monoid is commutative.

The specification is inferred *without* any user annotation, type hint, or docstring. It emerges from the cubical path structure of the HIT.

11.3 Result 3: Equivalence of Observationally Identical Programs

Prior equivalence checkers work in one of two regimes:

- **Structural** (compiler verification, translation validation): prove two programs have the same structure (AST, SSA form, or control flow graph). Fast but brittle — any structural difference causes failure.
- **Behavioral** (testing, fuzzing): execute programs on inputs and compare outputs. Sound for observed inputs but cannot prove equivalence for all inputs.

The cubical–cohomological approach occupies a *new* position:

Theorem 11.3 (Semantic Equivalence Without Runtime or Structure)**.** *Two programs f, g are provably semantically equivalent (for all inputs in the modeled domain) if:*

1. *Their Z3 denotations $[\![f]\!], [\![g]\!]$ are connected by a sequence of path constructors from the HIT* PyComp *(cubical),* ***and***
2. *The local paths glue across all type fibers with $H^1 = 0$ (cohomological).*

This proof uses no runtime execution *and* no structural matching.

This enables proving equivalence of programs with:

- Completely different control flow (recursion vs. iteration, BFS vs. DFS).
- Different data structures (heap vs. sorted array, OrderedDict vs. linked list).
- Different algorithmic strategies (DP vs. memoization, divide-and-conquer vs. sweep).

No prior system achieves this without either runtime testing or problem-specific guidance.

11.4 Result 4: Compositional Verification via the Spectral Sequence

The Čech cohomology can be refined using a *spectral sequence* that relates different levels of the type fibration.

Definition 11.4 (The Type Filtration Spectral Sequence)**.** The type fibration $\pi : \mathsf{PyObj} \to \mathsf{TypeTag}$ induces a filtration of the input space:

$$F^0 = \{\mathsf{Bot}\} \tag{11.4}$$
$$F^1 = \{\mathsf{Bot}, \mathsf{None}\} \tag{11.5}$$
$$F^2 = \{\mathsf{Bot}, \mathsf{None}, \mathsf{Bool}\} \tag{11.6}$$
$$F^3 = \{\mathsf{Bot}, \mathsf{None}, \mathsf{Bool}, \mathsf{Int}\} \tag{11.7}$$
$$\vdots \tag{11.8}$$
$$F^7 = \mathsf{TypeTag} \tag{11.9}$$

The spectral sequence $E_r^{p,q}$ has:

$$E_1^{p,q} = H^q(F^p/F^{p-1}, \mathsf{Iso}) \implies H^{p+q}(\mathsf{TypeTag}, \mathsf{Iso})$$

The spectral sequence enables *compositional* verification:

1. E_1 page: check equivalence on individual types (simple fibers).
2. E_2 page: check compatibility between adjacent types (transition functions).
3. E_∞ page: the global H^1 (obtained by convergence).

If E_1 shows all fibers are equivalent, the spectral sequence degenerates and $H^1 = 0$ immediately (no further computation needed). This is the common case for well-typed programs.

For programs with type-dependent behavior (e.g., `isinstance`-based dispatch), the E_2 page captures the interaction between fibers, and higher pages refine the picture.

11.5 Result 5: Automatic Specification Inference

The most striking consequence of CTTT: we can *infer* a program's specification from its code alone, with no annotations, docstrings, or tests.

Definition 11.5 (The Canonical Specification)**.** For a program $f : A \to B$, its *canonical specification* $\mathsf{CanonSpec}(f)$ is the equivalence class $[\![f]\!]$ in the HIT PyComp (Definition 4.1).

The canonical specification captures *what f computes* modulo the equational axioms — it abstracts away implementation details (loop vs. recursion, comprehension vs. explicit loop, etc.) and retains only the semantic content.

Theorem 11.6 (Automatic Spec Inference via CTTT)**.** *Given an unannotated program f, the CTTT framework infers:*

1. ***Input type**: the duck type of each parameter, determined by the type fibration and operation analysis (the fiber decomposition of $\mathcal{S}$).*
2. ***Output type**: the* $\mathsf{TypeTag}$ *of the return value on each fiber (the presheaf* Sem_f *restricted to each fiber).*

3. ***Behavioral invariants:*** *equational properties that f's output satisfies, determined by which path constructors the Z3 term is invariant under.*

4. ***The canonical form:*** *the equivalence class $[\![f]\!]$ in* PyComp*, which uniquely characterizes the function f computes.*

Proof. (1) follows from duck type inference (§38). (2) follows from compiling f to Z3 and evaluating tag on each fiber. (3) follows from testing which axioms hold: for each path constructor ax, check whether $\mathsf{ax}([\![f]\!]) = [\![f]\!]$ (is f's denotation fixed by this axiom?). (4) follows from (1)–(3): the canonical form is the normal form of $[\![f]\!]$ under all applicable axioms. □

Why cubical alone can't do this. Cubical type theory provides the path structure (PyComp) but doesn't decompose by type. Without the fibration, we can't determine input/output types or fiber-specific behavior.

Why cohomology alone can't do this. Cohomology provides the fiber decomposition but has no notion of "canonical form" or equational quotient. Without path constructors, we can't abstract away implementation details.

CTTT achieves both: the fibration determines types (cohomological), the HIT quotient determines behavior (cubical), and the combination gives the full specification.

Example 11.7 (Inferring "sorts a list")**.** Given:

```
def f(xs):
    for i in range(len(xs)):
        for j in range(i+1, len(xs)):
            if xs[j] < xs[i]:
                xs[i], xs[j] = xs[j], xs[i]
    return xs
```

Inferred specification:

- Input: collection (from `len`, subscript, iteration).
- Output: same type as input (returns the modified input).
- Behavioral invariant: $\mathsf{py_sorted}([\![f]\!](xs)) = [\![f]\!](xs)$ (the output is sorted — verified by Z3 using the sorted idempotence axiom applied to the compiled term).
- Canonical form: $[\mathsf{py_sorted}(xs)]$ — the equivalence class of "sorting."

The framework infers "this function sorts its input" without any annotation, by computing the canonical form $[[\![f]\!]]$ and recognizing it as the sorted equivalence class.

Example 11.8 (Inferring "computes factorial")**.** Given:

```
def f(n):
    result = 1
    for i in range(2, n+1):
        result *= i
    return result
```

Inferred specification:

- Input: int (from arithmetic, `range`).
- Output: int.
- Canonical form: $\mathsf{fold}(\mathsf{MUL}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(1))$.
- Behavioral invariant: $f(0) = 1$, $f(n) = n \cdot f(n-1)$ (the Nat-recursor structure of the fold).

The canonical form $[\mathsf{fold}(\times, 1, n)]$ IS the specification: any program with the same canonical form computes the same function.

11.6 Result 6: Bug Severity Quantification

Theorem 11.9 (Cohomological Bug Severity). *Given a program f and specification S, the* severity *of f's bugs is quantified by the cohomological data:*

$$\mathsf{severity}(f, S) = \left(\frac{|\mathcal{U}| - H^0}{|\mathcal{U}|},\ \mathrm{rank}(H^1),\ \sum_{obstructions} \mathsf{fiber_weight}(\tau) \right)$$

where:

- $\frac{|\mathcal{U}|-H^0}{|\mathcal{U}|}$ *is the fraction of fibers where f violates S ("how much is wrong").*
- $\mathrm{rank}(H^1)$ *is the number of independent obstructions ("how many distinct bugs").*
- *The weighted sum assigns higher severity to failures on common fibers (e.g.,* Int *fiber has higher weight than* None *fiber in most programs).*

Why this requires CTTT. Cubical theory alone gives binary verdicts (path exists or not). Cohomology alone gives obstruction data but no "canonical form" to compare against (what is the "ideal" behavior?). CTTT provides both: the specification is inferred (Result 5), and the cohomological decomposition quantifies how far the program deviates from it.

Example 11.10 (Graded Bug Report). A sorting function that works correctly on integers but crashes on strings:

$$\mathsf{severity} = \left(\frac{1}{3},\ 1,\ w_{\mathsf{Str}} \right)$$

One out of three fibers fails, one independent obstruction (the string-comparison crash), weighted by the string fiber's importance. The report says: "1 bug: crashes on string inputs (fiber TStr). Correct on integers and lists."

11.7 Result 7: Program Semantic Distance

Beyond binary equivalence, CTTT defines a *semantic distance* between programs.

Definition 11.11 (Semantic Distance). The *semantic distance* between programs f and g is:

$$d_{\mathsf{sem}}(f, g) = \frac{\mathrm{rank}(H^1(\mathcal{U}, \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_g)))}{|\mathcal{U}|}$$

— the fraction of the covering where the programs disagree, weighted by the H^1 rank.

Proposition 11.12 (Metric Properties). *d_{sem} is a pseudometric:*

1. $d_{\mathsf{sem}}(f, f) = 0$ *(reflexivity).*
2. $d_{\mathsf{sem}}(f, g) = d_{\mathsf{sem}}(g, f)$ *(symmetry).*
3. $d_{\mathsf{sem}}(f, h) \leq d_{\mathsf{sem}}(f, g) + d_{\mathsf{sem}}(g, h)$ *(triangle inequality — from the Mayer–Vietoris exact sequence).*

It is a pseudometric (not a metric) because $d_{\mathsf{sem}}(f, g) = 0$ does not imply $f = g$ (only that they agree on the modeled fibers).

Application: ranking candidate implementations. Given a specification S (with reference implementation r) and candidates f_1, f_2, f_3, compute $d_{\mathsf{sem}}(f_i, r)$ for each. The candidate with smallest distance is "closest to correct."

11.8 Result 8: Automatic Repair Localization

Theorem 11.13 (Cohomological Repair Localization). *If $H^1(\mathcal{U}, \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_S)) \neq 0$, the obstruction elements identify the* minimal set of fibers *where f must be modified to satisfy S.*

Specifically, if the obstruction is supported on fibers $\tau_1, \ldots, \tau_k$, then any repair of f must change its behavior on at least these fibers.

Proof. An obstruction element $\omega \in H^1$ is supported on a set of overlaps. These overlaps involve specific fibers. Changing f's behavior on fibers not in the support of ω cannot eliminate ω (the obstruction is "localized" to its support by the exactness of the Čech complex). □

Concretely: if a function fails on the Str fiber, the repair must involve the code path that handles string inputs. The cohomological localization tells the developer: "fix the string case" — not just "the function is wrong."

11.9 Result 9: Cross-Program Invariant Discovery

Theorem 11.14 (Invariant Discovery via Presheaf Intersection). *Given programs $f_1, \ldots, f_n$ that are all supposed to compute the same function, the* shared invariant *is:*

$$I = \bigcap_{i=1}^{n} \mathsf{Sem}_{f_i}$$

— the presheaf of behaviors common to all implementations.

On each fiber U, the invariant $I(U)$ captures what all implementations agree on. If $I(U)$ is nontrivial (not all programs agree), the fiber U reveals an inconsistency among the implementations.

Application: given three implementations of "edit distance" (recursive, DP, Hirschberg), compute the pairwise H^1 and identify any fiber where they disagree. If all three agree on all fibers, their shared behavior IS the correct specification — discovered automatically from the implementations, with no reference spec needed.

11.10 Result 10: Semantic Refactoring Safety

Theorem 11.15 (Refactoring Safety via Path Existence). *A refactoring $f \rightsquigarrow g$ (modifying a program's implementation without changing its behavior) is* safe *iff there exists a cubical path $p : \mathsf{Path}([\![f]\!], [\![g]\!])$ in the HIT* PyComp.

The path p decomposes into a sequence of elementary refactoring steps, each corresponding to a single path constructor (dichotomy axiom):

$$f = h_0 \xrightarrow{\mathsf{D}_{k_1}} h_1 \xrightarrow{\mathsf{D}_{k_2}} h_2 \xrightarrow{\mathsf{D}_{k_3}} \ldots \xrightarrow{\mathsf{D}_{k_m}} h_m = g$$

Each step $h_i \to h_{i+1}$ is a single dichotomy application (e.g., D1: unfold recursion to iteration, D2: inline a helper, D4: replace comprehension with loop).

The refactoring is safe iff this path exists. The path itself serves as a certificate of safety *— a constructive proof that the refactoring preserves semantics.*

Why CTTT is essential.

- Cubical alone: can check if the final result is equivalent (binary verdict), but cannot decompose the refactoring into steps or certify each step independently.

- Cohomological alone: can check fiber-by-fiber, but cannot construct the path or certify the refactoring sequence.
- CTTT: the path decomposes into dichotomy steps (cubical), each verified on all fibers (cohomological). If any step fails on any fiber, the specific breaking change is identified.

Example 11.16 (Safe Refactoring: Recursion to Iteration)**.** Refactoring:

```
# Before (recursive)
def f(n):
    if n <= 1: return 1
    return n * f(n-1)

# After (iterative)
def g(n):
    result = 1
    for i in range(2, n+1):
        result *= i
    return result
```

The refactoring path: $f \xrightarrow{\mathsf{D1}} g$ (a single Nat-recursor step). Verified on the Int fiber: both compile to the same fold term. The certificate is:

$$\mathsf{D1}(\mathsf{base} = 1, \mathsf{step} = \times, \mathsf{threshold} = 1) : \mathsf{Path}([\![f]\!], [\![g]\!])$$

Example 11.17 (Unsafe Refactoring: Mutation Change)**.** Refactoring:

```
# Before
def f(lst):
    lst += [1]
    return lst

# After (attempted "simplification")
def g(lst):
    lst = lst + [1]
    return lst
```

No path exists: mut_iadd and call_add are different shared symbols (different effect annotations). The obstruction is on the Ref fiber. The framework reports: "UNSAFE: this refactoring changes mutation semantics on mutable inputs (fiber TRef)."

11.11 Result 11: Compositional Semantics via Sheaf Gluing

Theorem 11.18 (Compositional Program Semantics)**.** *The semantics of a composed program $h = g \circ f$ can be computed* compositionally *from the presheaves Sem_f and Sem_g via the sheaf-theoretic pullback:*

$$\mathsf{Sem}_h = \mathsf{Sem}_g \circ \mathsf{Sem}_f$$

where composition of presheaves is defined fiber-by-fiber: $\mathsf{Sem}_h(U) = \mathsf{Sem}_g(\mathsf{Sem}_f(U))$.

If $f \equiv f'$ (with cubical path p_f) and $g \equiv g'$ (with path p_g), then $g \circ f \equiv g' \circ f'$ (with composed path $p_g \circ p_f$).

Why this matters: it means we can verify large programs *modularly*. If a program is built from composable pieces, and each piece is individually verified (or shown equivalent to a reference), the whole program inherits the verification.

The sheaf gluing condition ensures that local verifications on individual components are *globally consistent* — the components interact correctly across type fiber boundaries.

11.12 Summary: The CTTT Advantage

Result	CTT alone	CT alone	CTTT
R1: Cross-fiber type inference	×	×	✓
R5: Automatic spec inference	×	×	✓
R2: Spec inference (basic)	partial	×	✓
R3: Semantic equiv w/o runtime	partial	×	✓
R4: Compositional spectral seq	×	partial	✓
R6: Bug severity quantification	×	partial	✓
R7: Semantic distance metric	×	partial	✓
R8: Repair localization	×	partial	✓
R9: Cross-program invariant	×	×	✓
R10: Refactoring safety cert.	partial	×	✓
R11: Compositional semantics	partial	partial	✓

The pattern: cubical type theory provides the *path structure* (equalities, canonical forms, quotients) and cohomological theory provides the *decomposition structure* (fibers, local/global, obstructions). *Neither alone* achieves the full result — the synthesis is essential.

11.13 Python as a Fibered Category

We have established:

- **Cubical structure**: Python programs form a HIT PyComp with path constructors from the equational axioms (§4).
- **Fibered structure**: Python values have types, forming a fibration $\pi : \mathsf{PyObj} \to \mathsf{TypeTag}$ (§6).

The combination is a *fibered cubical type*: the type PyComp is fibered over $\mathsf{TypeTag}$, and paths in PyComp must be compatible with the fibration.

Definition 11.19 (Fibered Path)**.** A *fibered path* from f to g over the fibration π is a path $p : \mathsf{Path}_{\mathsf{PyComp}}(f, g)$ such that $\pi \circ p = \mathsf{refl}$ — the path stays within the same fiber (or moves between fibers via the transport maps).

Theorem 11.20 (Fibered Path Decomposition)**.** *A fibered path $p : f \to g$ decomposes into:*

1. *A family of* fiber paths $p_\tau : f|_\tau \to g|_\tau$ *for each type τ.*
2. Transition paths *on overlaps that are compatible (the cocycle condition).*

The existence of such a decomposition is equivalent to $H^1 = 0$.

This is Theorem 10.1 restated in fibered language.

11.14 Duck Typing as Cubical Transport

Python's duck typing ("if it walks like a duck and quacks like a duck, it's a duck") has a precise cubical interpretation.

Definition 11.21 (Duck Type)**.** A *duck type* is a set of operations $\mathsf{ops} \subseteq \{\mathsf{add}, \mathsf{mul}, \mathsf{len}, \mathsf{getitem}, \ldots\}$. A value v has duck type ops if all operations in ops are defined on v.

The duck type determines a *union of fibers*: if a parameter has duck type $\{\mathsf{add}, \mathsf{mul}\}$, it can be Int (which supports both) but not Str (which supports add but not mul).

Proposition 11.22 (Duck Typing as Fiber Restriction)**.** *The duck type* ops *determines a sub-covering of the type site: the covering by fibers τ where all operations in* ops *are defined.*

The Čech cohomology is computed over this sub-covering, which is smaller (fewer fibers, fewer overlaps) and therefore cheaper.

This is how duck type inference *narrows* the cohomological computation: it eliminates irrelevant fibers before the Čech complex is constructed.

In cubical terms, duck typing is *transport along the operation path*: the operation add determines a path in the universe of types from Int to any type supporting add. The duck type is the intersection of these paths.

11.15 Effects as Non-Trivial Cohomology

The two false positive cases in the benchmark (neq06: late binding; neq20: in-place +=) involve *effects* — observable mutations of shared state.

In the cubical–cohomological framework, effects manifest as *non-trivial* H^1:

Theorem 11.23 (Effects as Cohomological Obstructions)**.** *A program with mutation effects (aliasing, late binding, in-place modification) has a non-trivial 1-cocycle in the H^1 of the* heap fibration*:*

$$\pi_{\mathsf{heap}} : \mathsf{State} \to \mathsf{HeapShape}$$

where HeapShape *classifies states by their aliasing structure.*

Two programs that differ on the heap fiber (one mutates, one doesn't) have $H^1(\mathsf{HeapShape}, \mathsf{Iso}) \neq 0$ — the local equivalences on value fibers don't glue to a global equivalence because the heap fiber introduces an obstruction.

This is how the framework correctly rejects neq06 and neq20:

- **neq06**: The late-binding closure captures a Ref to the loop variable (heap reference). The early-binding version captures an IntObj (value copy). On the Ref fiber, the programs produce different results because the Ref's value changes after the loop. $H^1(\mathsf{Ref}) \neq 0$.
- **neq20**: In-place $+=$ uses mut_iadd (modifies the Ref). Reassignment $+$ uses call_add (creates a new value). These are *different shared symbols*, so Z3 correctly distinguishes them on the Ref fiber.

11.16 Why This Could Not Be Done Before

1. **Structural equivalence checkers** (translation validation, compiler verification): these compare program structures (ASTs, SSA forms, proof terms). They fail when the structures differ — which is the *common case* for algorithm equivalence (our 44 failing pairs have completely different structures).

   CTTT overcomes this by working in the *quotient* HIT PyComp, where structurally different programs are identified by path constructors.

2. **Behavioral testers** (fuzzing, property testing): these execute programs on sampled inputs. They can find counterexamples but cannot prove equivalence for all inputs.

   CTTT proves equivalence *for all inputs* in the modeled domain, using Z3's satisfiability checking (which explores the full symbolic input space, not just sampled points).

3. **Denotational semantics** (classical domain theory): these map programs to mathematical objects (Scott domains, complete partial orders) and compare denotations. But the comparison is typically undecidable for Turing-complete languages.

   CTTT restricts to a *decidable fragment* (Z3's quantifier-free theories + controlled use of quantifiers via E-matching). The restriction is sound (no false positives, modulo the effect discipline) but incomplete (some equivalent programs are not proved equivalent).

4. **Dependent type systems** (Agda, Coq, Lean): these require programs to be written *in* the type system, with explicit proof terms. They cannot analyze existing Python code.

   CTTT *infers* dependent types from unannotated Python code by compiling to Z3 terms and decomposing via the type fibration. No user annotations, no proof terms, no rewriting the program in a different language.

5. **Abstract interpretation**: computes sound approximations of program behavior (intervals, polyhedra, etc.). But abstract interpretation loses precision — it proves *properties* ("the output is non-negative") rather than *equivalences* ("this program equals that program").

   CTTT proves exact equivalences (or finds exact counterexamples) by using Z3's decision procedures on the exact PyObj theory.

Remark 11.24 (The Price of Generality)**.** The CTTT approach is *incomplete*: it cannot prove all true equivalences (Rice's theorem prevents this). The current implementation proves 6/50 equivalent pairs and correctly rejects 48/50 non-equivalent pairs. The 44 unproved equivalent pairs require deeper path constructors (the 24 dichotomy theories) that are not yet fully implemented in the Z3 engine.

The theoretical framework, however, provides a *roadmap* for closing the gap: each dichotomy theory adds a new set of path constructors, and each path constructor is a sound axiom that Z3 can use. The completeness of the framework is limited only by the richness of the axiom set.

Part IV

Advanced CTTT Machinery

Chapter 12

Galois Connections for Abstract Specification Inference

Abstract interpretation (Cousot & Cousot, 1977) computes sound approximations of program behavior using *abstract domains*. CTTT incorporates abstract interpretation via Galois connections between the concrete presheaf and abstract presheaves, dramatically increasing the power of automatic specification inference.

12.1 Abstract Domains as Presheaves

Definition 12.1 (Abstract Presheaf)**.** An *abstract presheaf* $\mathsf{Sem}_f^{\#} : \mathcal{S}^{\mathrm{op}} \to \mathsf{Lattice}$ assigns to each type fiber U an element of an abstract domain D, approximating the concrete behavior $\mathsf{Sem}_f(U)$.

Examples of abstract domains:

Domain	**Elements**	**What it captures**
Sign	$\{-, 0, +, \top\}$	Sign of integer output
Interval	$[l, u] \subseteq \mathbb{Z}$	Range of output
Parity	$\{\mathsf{even}, \mathsf{odd}, \top\}$	Parity
Length	$\mathbb{N} \cup \{\top\}$	Output collection length
Sortedness	$\{\mathsf{sorted}, \mathsf{unsorted}, \top\}$	Whether output is sorted
Permutation	$\{\mathsf{perm}(x), \mathsf{not_perm}, \top\}$	Whether output permutes input
Monotonicity	$\{\uparrow, \downarrow, \mathsf{const}, \top\}$	Whether output is monotone in input

Definition 12.2 (Galois Connection)**.** A *Galois connection* between concrete presheaf Sem_f and abstract presheaf $\mathsf{Sem}_f^{\#}$ is a pair of monotone maps:

$$\alpha : \mathsf{Sem}_f \to \mathsf{Sem}_f^{\#} \qquad \text{(abstraction)} \tag{12.1}$$

$$\gamma : \mathsf{Sem}_f^{\#} \to \mathsf{Sem}_f \qquad \text{(concretization)} \tag{12.2}$$

satisfying $\mathrm{id} \leq \gamma \circ \alpha$ and $\alpha \circ \gamma \leq \mathrm{id}$ (the Galois condition).

Theorem 12.3 (Abstract Spec Inference via Galois Connection)**.** *The abstract presheaf* $\mathsf{Sem}_f^{\#}$ *inferred by abstract interpretation is a* sound over-approximation *of the specification:*

$$f \models S \implies \alpha(\mathsf{Sem}_f) \leq \alpha(\mathsf{Sem}_S)$$

Contrapositively: if $\alpha(\mathsf{Sem}_f) \not\leq \alpha(\mathsf{Sem}_S)$*, then* f *violates* S *(sound bug detection).*

12.2 How This Increases CTTT's Power

The basic CTTT (Parts I–III) compares Z3 terms directly, which fails when terms are structurally different. Abstract interpretation *lifts* the comparison to a coarser domain where structurally different terms may have the same abstract value.

Example 12.4 (Sign-Domain Spec Inference)**.** Consider:

```
def f(n):          # recursive factorial
    if n <= 1: return 1
    return n * f(n-1)

def g(n):          # iterative factorial
    r = 1
    for i in range(2, n+1): r *= i
    return r
```

In the Sign domain:

- $\alpha(\mathsf{Sem}_f)(U^+_{\mathsf{Int}}) = +$ (positive for positive n).
- $\alpha(\mathsf{Sem}_g)(U^+_{\mathsf{Int}}) = +$ (same).
- The abstract presheaves agree $\implies$ no obstruction at the abstract level.

This doesn't prove equivalence (the Sign domain is too coarse), but it *eliminates impossible counterexamples*: Z3 can be told "the output is positive," narrowing its search.

Example 12.5 (Sortedness-Domain Spec Checking)**.**

```
@guarantee("returns a sorted list")
def f(xs):
    result = []
    for x in xs:
        bisect.insort(result, x)
    return result
```

The Sortedness abstract domain infers: $\alpha(\mathsf{Sem}_f) = \mathsf{sorted}$ (because $\mathsf{bisect.insort}$ maintains sorted invariant — this is a *transfer function* in the abstract domain).

The specification S requires $\alpha(\mathsf{Sem}_S) = \mathsf{sorted}$. Since $\alpha(\mathsf{Sem}_f) = \alpha(\mathsf{Sem}_S)$, the spec is satisfied at the abstract level. This is a *proof* (not just a test) that the function returns a sorted list.

12.3 The Galois–Čech Interaction

The abstract domains interact with the Čech complex:

Theorem 12.6 (Abstract Cohomology)**.** *There is a morphism of cochain complexes:*

$$\begin{tikzcd}
C^0(\mathcal{U}, \mathsf{Iso}) \arrow[r, "\delta^0"] \arrow[d, "\alpha"] & C^1(\mathcal{U}, \mathsf{Iso}) \arrow[r, "\delta^1"] \arrow[d, "\alpha"] & C^2(\mathcal{U}, \mathsf{Iso}) \arrow[d, "\alpha"] \\
C^0(\mathcal{U}, \mathsf{Iso}^\#) \arrow[r, "\delta^{0,\#}"] & C^1(\mathcal{U}, \mathsf{Iso}^\#) \arrow[r, "\delta^{1,\#}"] & C^2(\mathcal{U}, \mathsf{Iso}^\#)
\end{tikzcd}$$

The abstraction α commutes with the coboundary operators. Therefore:

$$H^1(\mathcal{U}, \mathsf{Iso}^\#) = 0 \implies H^1(\mathcal{U}, \mathsf{Iso}) = 0$$

(if the abstract Čech complex has trivial H^1, so does the concrete one — but not vice versa).

This gives a *cheaper* sufficient condition for equivalence: compute H^1 in the abstract domain (faster, always terminates) before resorting to the full Z3 check (slower, may timeout).

Chapter 13

The Descent Spectral Sequence

The spectral sequence is a systematic tool for computing cohomology in stages, each refining the previous. It vastly increases CTTT's power by enabling *incremental* verification: start with a coarse check, refine only where needed.

13.1 Filtrations of the Type Site

Definition 13.1 (Type Complexity Filtration)**.** The type site $\mathcal{S}$ has a natural filtration by complexity:

$$F^0 = \{\mathsf{None}, \mathsf{Bot}\} \qquad \text{(trivial types)} \tag{13.1}$$
$$F^1 = F^0 \cup \{\mathsf{Bool}\} \qquad \text{(finite types)} \tag{13.2}$$
$$F^2 = F^1 \cup \{\mathsf{Int}\} \qquad \text{(countable types)} \tag{13.3}$$
$$F^3 = F^2 \cup \{\mathsf{Str}\} \qquad \text{(string type)} \tag{13.4}$$
$$F^4 = F^3 \cup \{\mathsf{Pair}\} \qquad \text{(structural types)} \tag{13.5}$$
$$F^5 = F^4 \cup \{\mathsf{Ref}\} \qquad \text{(reference types — heap)} \tag{13.6}$$

Each level adds more complex types, requiring more sophisticated reasoning.

Definition 13.2 (The Spectral Sequence)**.** The filtration induces a spectral sequence $\{E_r^{p,q}\}$ with:

$$E_0^{p,q} = C^{p+q}(F^p/F^{p-1}, \mathsf{Iso}) \tag{13.7}$$
$$E_1^{p,q} = H^{p+q}(F^p/F^{p-1}, \mathsf{Iso}) \tag{13.8}$$
$$E_\infty^{p,q} \implies H^{p+q}(\mathcal{S}, \mathsf{Iso}) \tag{13.9}$$

13.2 The Verification Strategy

The spectral sequence dictates an *optimal verification strategy*:

1. E_0 **page**: check equivalence on each filtration *grade* independently. For the None/Bot grade: trivially equivalent (both return the same constant on trivial inputs). For the Bool grade: finitely many inputs, decidable. For the Int grade: Z3 integer arithmetic, often decidable.

2. E_1 **page**: compute H^1 within each grade. If $H^1(F^p/F^{p-1}) = 0$ for all p, the spectral sequence *degenerates* and $H^1(\mathcal{S}) = 0$ (equivalence).

3. E_2 **page**: if E_1 has non-trivial differentials, compute $E_2 = H(E_1, d_1)$. This captures *interactions* between adjacent grades (e.g., an integer function whose behavior depends on a boolean flag).

4. **Higher pages**: each E_r refines the approximation. In practice, the spectral sequence degenerates at E_1 or E_2 for most programs (because the type fibers are independent).

Theorem 13.3 (Spectral Sequence Degeneration for Duck-Typed Programs)**.** *If duck type inference determines that each parameter has a* single *type (the most common case), the spectral sequence degenerates at E_1:*

$$E_1^{p,q} = E_\infty^{p,q} \quad \textit{for all } p, q$$

and $H^1(\mathcal{S}, \mathsf{Iso}) = \bigoplus_p E_1^{p,0}$ (direct sum of fiber-wise cohomologies).

Proof. When each parameter has a unique type, the type site has a single object (one fiber combination). There are no overlaps, no transition functions, and the Čech complex is trivial: $C^0 = \mathsf{Iso}(U)$, $C^1 = C^2 = 0$. Therefore $H^1 = 0$ (no obstructions to check) and the local verdict IS the global verdict. The spectral sequence has a single non-trivial entry $E_1^{0,0} = H^0(U, \mathsf{Iso})$ and degenerates immediately. □

This theorem explains why our current implementation achieves good results on programs with clear types: the spectral sequence degenerates, and a single Z3 query per fiber suffices.

13.3 Non-Degenerate Cases: Type Polymorphism

When parameters are polymorphic (duck-typed with multiple possible types), the spectral sequence does NOT degenerate, and the E_2 page reveals interactions:

Example 13.4 (Type-Dependent Behavior)**.**

```
def f(x):
    if isinstance(x, int):
        return x * 2
    elif isinstance(x, str):
        return x + x
    else:
        return [x, x]
```

The type site has three fibers: Int, Str, Other. On each fiber, f has different behavior. The E_1 page checks each fiber independently. The E_2 page checks compatibility across fibers: does the "doubling" specification ($S(x, y) = (y = x + x)$) make sense across type boundaries?

For integers, $x + x = 2x$ (integer doubling). For strings, $x + x = xx$ (string repetition). These are *different operations* (ADD vs. Concat) that happen to share the + syntax.

The E_2 differential detects this: the integer and string fibers use different path constructors (add_comm vs. string concat), and the transition function on the Int∩Str overlap is non-trivial. This is a genuinely *higher* cohomological phenomenon — invisible to E_1.

Chapter 14

Operad Theory for Higher-Order Programs

Python programs frequently use *higher-order functions*: functions that take other functions as arguments (`map`, `filter`, `reduce`, `sorted` with key, etc.). The CTTT framework handles these via *operad theory*, which provides the algebraic structure for composing operations.

14.1 The Operad of Python Operations

Definition 14.1 (The Python Operations Operad)**.** The *Python operations operad* $\mathcal{O}_{\mathsf{Py}}$ has:

- **Colors** (sorts): PyObj, $\mathbb{B}$, $\mathbb{Z}$, Str, Seq, Map.
- **Operations**: for each Python builtin/method, an operation $o \in \mathcal{O}(c_1, \ldots, c_n; c)$ mapping n inputs of colors $c_1, \ldots, c_n$ to an output of color c.
- **Composition**: operadic composition $\circ_i$ (substituting an operation into the i-th input of another).
- **Equivariance**: permutation of inputs (for commutative operations).

Example 14.2 (Operations in the Operad)**.**

$$+ \in \mathcal{O}(\mathbb{Z}, \mathbb{Z}; \mathbb{Z}) \tag{14.1}$$
$$\mathsf{sorted} \in \mathcal{O}(\mathsf{Seq}; \mathsf{Seq}) \tag{14.2}$$
$$\mathsf{map} \in \mathcal{O}((\mathsf{PyObj} \to \mathsf{PyObj}), \mathsf{Seq}; \mathsf{Seq}) \tag{14.3}$$
$$\mathsf{reduce} \in \mathcal{O}((\mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj}), \mathsf{PyObj}, \mathsf{Seq}; \mathsf{PyObj}) \tag{14.4}$$
$$\mathsf{filter} \in \mathcal{O}((\mathsf{PyObj} \to \mathbb{B}), \mathsf{Seq}; \mathsf{Seq}) \tag{14.5}$$

14.2 Operadic Path Constructors

The operad structure provides new path constructors beyond the basic algebraic axioms:

Theorem 14.3 (Map–Fold Fusion)**.** *For a monoid $(M, \oplus, e)$ and a function $h : A \to M$:*

$$\mathsf{reduce}(\oplus, e, \mathsf{map}(h, \mathsf{xs})) \equiv \mathsf{reduce}(\lambda \mathsf{acc}, x.\ \mathsf{acc} \oplus h(x), e, \mathsf{xs})$$

This is the map–fold fusion law*: mapping then folding is equivalent to folding with a composed operation.*

Proof. By induction on xs (Nat-recursor uniqueness applied to the list length):

- Base: both sides return e on the empty list.
- Step: the left side maps h to the head, then folds; the right side applies $\oplus$ with $h(\mathsf{head})$ directly. Both produce $\mathsf{acc} \oplus h(\mathsf{head})$.

□

Theorem 14.4 (Filter–Map Commutation). *If the predicate p and the transformation h are independent ($p(h(x)) = p(x)$ for all x):*

$$\mathsf{filter}(p, \mathsf{map}(h, \mathsf{xs})) \equiv \mathsf{map}(h, \mathsf{filter}(p, \mathsf{xs}))$$

Theorem 14.5 (Sorted–Map Interaction). *If h is* monotone *($x \leq y \implies h(x) \leq h(y)$):*

$$\mathsf{sorted}(\mathsf{map}(h, \mathsf{xs})) \equiv \mathsf{map}(h, \mathsf{sorted}(\mathsf{xs}))$$

Sorting then mapping a monotone function is the same as mapping then sorting.

These operadic laws generate new paths in the HIT PyComp, enabling equivalence proofs for programs that compose higher-order operations in different ways.

14.3 Application: Higher-Order Equivalence

Example 14.6 (Map+Filter vs. Comprehension with Condition).

```python
# Program A: map then filter
def f(xs):
    return list(filter(lambda x: x > 0, map(lambda x: x*2, xs)))

# Program B: comprehension with condition
def g(xs):
    return [x*2 for x in xs if x*2 > 0]
```

The operadic path:

1. $\mathsf{filter}(p, \mathsf{map}(h, \mathsf{xs}))$ (program A).
2. By filter–map commutation (since $p(h(x)) = (h(x) > 0)$ and $p(x) = (x > 0)$ — but $h(x) = 2x$, so $p(h(x)) = (2x > 0) \neq (x > 0)$ in general! The commutation does NOT apply here).
3. Instead, use the comprehension–loop equivalence (D4): the comprehension $[h(x) \text{ for } x \text{ if } p(h(x))]$ is equivalent to $\mathsf{filter}(p, \mathsf{map}(h, \mathsf{xs}))$.
4. The path is D4 (comprehension fusion).

Chapter 15

The Semantic Enrichment Theorem

The most powerful extension of CTTT: using the cubical path space to *enrich* the Čech complex, yielding a finer cohomology that distinguishes programs the basic theory cannot.

15.1 Enriched Cohomology

Definition 15.1 (Path-Enriched Cochain)**.** An *enriched 0-cochain* assigns to each fiber U_i not just a boolean (isomorphism or not) but a *path type*:

$$C^0_{\mathsf{enr}}(\mathcal{U}, \mathsf{Iso}) = \prod_i \|\mathsf{Path}(\mathsf{Sem}_f|_{U_i}, \mathsf{Sem}_g|_{U_i})\|$$

where $\|\cdot\|$ is propositional truncation (we care about existence, not the specific path).

The enrichment adds information: we know not just *whether* two programs agree on a fiber, but *how* they agree (which axioms connect them).

Definition 15.2 (Enriched Coboundary)**.** The enriched coboundary $\delta^0_{\mathsf{enr}} : C^0_{\mathsf{enr}} \to C^1_{\mathsf{enr}}$ assigns to each overlap U_{ij} the *transition path*:

$$(\delta^0_{\mathsf{enr}}\,\varphi)_{ij} = p_j|_{U_{ij}} \cdot (p_i|_{U_{ij}})^{-1}$$

where p_i and p_j are the local paths on fibers U_i and U_j.

Theorem 15.3 (Enriched Descent)**.** *The enriched cohomology H^1_{enr} is a refinement of the ordinary H^1:*

$$H^1_{\mathsf{enr}} = 0 \implies H^1 = 0$$

but not vice versa. The enriched version can detect path incompatibilities *that the ordinary version misses: situations where local paths exist but are* inconsistent *(they use incompatible axioms on overlaps).*

Example 15.4 (Path Incompatibility)**.** Program f uses `isinstance(x, int)` to dispatch. On the Int fiber, the equivalence path uses the add commutativity axiom. On the Str fiber, the path uses string concatenation (which is NOT commutative).

The ordinary H^1 says: both fibers are locally equivalent, so $H^1 = 0$. But the enriched H^1 detects that the transition path on the Int∩Str overlap is inconsistent (commutativity on one side, non-commutativity on the other).

This reveals a subtle bug: the program treats `+` as commutative everywhere, but it's only commutative for integers.

15.2 The Enrichment–Degeneration Tradeoff

Proposition 15.5. *There is a tradeoff between enrichment and computability:*

1. ***Boolean enrichment*** *(ordinary H^1): decidable, computable in $O(n^3)$ via GF(2) rank.*
2. ***Path enrichment*** *(enriched H^1): semi-decidable (each local path check is a Z3 query, which may timeout).*
3. ***Full cubical enrichment*** *(tracking the specific path constructors used): undecidable in general, but provides the most information.*

In practice, we use boolean enrichment (fast) as a first pass, then refine with path enrichment (slower) on fibers where the boolean check is ambiguous. This is the spectral sequence strategy applied to the enrichment dimension.

Chapter 16

Inductive Families and Dependent Pattern Matching

The most ambitious extension: using *inductive families* (dependent inductive types indexed by values) to capture program invariants that vary with the input.

16.1 Inductive Families in CTTT

Definition 16.1 (Inductive Family)**.** An *inductive family* $T : I \to \mathsf{Type}$ indexed by I is a type defined by constructors whose return type depends on the index:

$$\mathsf{nil} : T(\mathsf{empty}) \tag{16.1}$$

$$\mathsf{cons} : \prod_{i:I} A \to T(i) \to T(\mathsf{extend}(i)) \tag{16.2}$$

Example 16.2 (Length-Indexed Lists)**.** The type $\mathsf{Vec}(A, n)$ of lists of length exactly n:

$$\mathsf{nil} : \mathsf{Vec}(A, 0) \tag{16.3}$$

$$\mathsf{cons} : A \to \mathsf{Vec}(A, n) \to \mathsf{Vec}(A, n + 1) \tag{16.4}$$

A program that takes a $\mathsf{Vec}(A, n)$ and returns a $\mathsf{Vec}(B, n)$ is *guaranteed* to preserve length.

16.2 Application: Invariant-Indexed Specifications

Inductive families let us express specifications that depend on structural properties of the input:

Definition 16.3 (Sorted-Input Specification)**.** The type $\mathsf{Sorted}(A)$ of sorted lists:

$$\mathsf{snil} : \mathsf{Sorted}(A) \tag{16.5}$$

$$\mathsf{scons} : \prod_{a:A} \mathsf{Sorted}(A) \to (\forall b \in \mathsf{head}.\ a \leq b) \to \mathsf{Sorted}(A) \tag{16.6}$$

A specification "binary search on sorted input" is:

$$S : \mathsf{Sorted}(\mathbb{Z}) \times \mathbb{Z} \to \mathsf{Option}(\mathbb{N})$$

The input is *typed as sorted*, so the specification can assume the precondition.

16.3 Automatic Precondition Inference via Inductive Families

Theorem 16.4 (Precondition Inference). *Given a program f and a specification S, if f satisfies S only on a subset of inputs, the* precondition *P can be automatically inferred as:*

$$P(x) = \mathsf{Spec}_S \text{ is satisfied at fiber } U_x$$

where U_x is the type fiber containing x.

The precondition is the set of fibers where the local Čech check passes:

$$P = \{x : A \mid \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_S)(U_{\mathsf{tag}(x)}) = 1\}$$

Example 16.5 (Inferring "requires positive integer").

```
def sqrt_int(n):
    if n < 0: return None   # error case
    r = 0
    while (r+1)*(r+1) <= n:
        r += 1
    return r
```

Specification: "returns $\lfloor\sqrt{n}\rfloor$".

On the Int^+ fiber (positive integers): correct. On the Int^- fiber (negative integers): returns None, violating the spec.

Inferred precondition: $P(n) = (n \geq 0)$.

The cohomological decomposition reveals: $H^1(U_{\mathsf{Int}^+}) = 0$ (correct on positives), $H^1(U_{\mathsf{Int}^-}) \neq 0$ (fails on negatives). The precondition is the union of passing fibers.

16.4 Dependent Pattern Matching as Fiber Restriction

Python's `isinstance` dispatch is a form of *dependent pattern matching*: the program branches on the type of its input, and each branch has a different type signature.

In CTTT, this is *exactly* the fiber decomposition:

- Each `isinstance` branch restricts to a fiber.
- The branch body is the presheaf section on that fiber.
- The Čech complex checks that the branches are coherent (they agree on type boundaries).

Theorem 16.6 (isinstance as Dependent Elimination). *A function with `isinstance` dispatch:*

```
def f(x):
    if isinstance(x, int):     return h_int(x)
    elif isinstance(x, str):   return h_str(x)
    else:                      return h_other(x)
```

has the dependent type:

$$f : \prod_{x:\mathsf{PyObj}} B(\mathsf{tag}(x))$$

where $B(\mathsf{TInt}) = \mathsf{codomain}(h_{\mathsf{int}})$, $B(\mathsf{TStr}) = \mathsf{codomain}(h_{\mathsf{str}})$, etc.

The dependent type is inferred automatically from the `isinstance` guards and the section extraction.

Chapter 17

The Computable Fragment of CTTT

A critical question: which parts of CTTT are *decidable* (algorithms always terminate with a correct answer) vs. *semi-decidable* (may not terminate) vs. *undecidable* (no algorithm exists)?

17.1 Decidability Map

Problem	**Status**	**Justification**
Duck type inference	Decidable	Finite AST walk, finite operation set
Site category construction	Decidable	Finite product of finite tag sets
Local equivalence (QF)	Decidable	QF_DT + QF_LIA is decidable
Local equivalence (with folds)	Semi-decidable	RecFunctions may cause non-termination
Local equivalence (with axioms)	Semi-decidable	E-matching may loop
H^1 computation	Decidable	GF(2) linear algebra, $O(n^3)$
Full pipeline (finite types)	Decidable	All components decidable
Full pipeline (general)	Semi-decidable	Z3 may timeout on RecFunctions
Equivalence (general)	Undecidable	Rice's theorem
Spec checking (general)	Undecidable	Reduces to equivalence
Abstract spec inference	Decidable	Finite abstract domains
Bug finding (per fiber)	Semi-decidable	SAT check may timeout

17.2 The Decidable Core

The *decidable core* of CTTT consists of programs where:

1. All parameters have a single duck type (the spectral sequence degenerates — Theorem 13.3).
2. No recursive functions or unbounded loops (no RecFunctions).
3. No universal quantifiers in axioms (no E-matching).

For this core, the full pipeline (compilation, fiber decomposition, local checks, H^1 computation) is decidable with polynomial complexity.

Theorem 17.1 (Decidability of the Core)**.** *For programs in the decidable core, the equivalence/spec-checking/ bug-finding verdict is computed in time:*

$$O(|A_f| + |A_g| + |\mathcal{S}|^3)$$

where $|A_f|, |A_g|$ are the AST sizes and $|\mathcal{S}|$ is the number of sites (type fiber combinations).

Proof. 1. AST compilation: $O(|A|)$ (single walk).

2. Duck type inference: $O(|A|)$ (single walk).

3. Site construction: $O(|\mathcal{S}|^2)$ (enumerate products and overlaps).

4. Local checks: $O(|\mathcal{S}|)$ Z3 queries, each in QF_DT + QF_LIA (decidable in NP, polynomial with fixed formula size).

5. H^1 computation: $O(|\mathcal{S}|^3)$ (GF(2) Gaussian elimination).

Total: dominated by the H^1 computation. □

17.3 Extending the Decidable Fragment

Each dichotomy theory (Part V) extends the decidable fragment by adding axioms that convert semi-decidable checks to decidable ones:

- **D1 (Rec↔Iter)**: The Nat-eliminator extraction converts RecFunctions to canonical folds, making the local check decidable (syntactic equality of fold terms).
- **D2 (Nested↔Flat)**: Function inlining eliminates nested calls, producing ground terms.
- **D4 (Comp↔Loop)**: Comprehension hashing reduces to shared symbol equality (decidable).
- **D8 (Multi-Return↔Single)**: Section merging reduces to Z3 If-expression comparison (decidable).

Each extension enlarges the set of programs for which the pipeline is fully decidable, without compromising soundness for programs outside the fragment (they get "Unknown" rather than wrong answers).

Chapter 18

Automated Path Search

This is the algorithmic heart of CTTT. Given two Z3 terms $[\![f]\!]$ and $[\![g]\!]$ that are not syntactically equal, the *path search engine* automatically discovers a multi-step cubical path $[\![f]\!] \xrightarrow{p_1} h_1 \xrightarrow{p_2} h_2 \xrightarrow{p_3} \dots \xrightarrow{p_k} [\![g]\!]$ where each p_i is a single path constructor application.

18.1 The Path Search Problem

Definition 18.1 (Path Search)**.** Given Z3 terms $s, t : \mathsf{PyObj}$ (the source and target), the *path search problem* is:

$$\text{Find } p_1, \dots, p_k \in \mathsf{PathConstructors} \text{ such that } s \xrightarrow{p_1} \cdot \xrightarrow{p_2} \cdots \xrightarrow{p_k} t$$

or prove that no such sequence exists (by finding a counterexample — a concrete input where $s \neq t$).

The path constructors are the axioms from the HIT PyComp (Definition 4.1): commutativity, associativity, fold–unfold, β-reduction, etc.

18.2 The Path Graph

Definition 18.2 (Path Graph)**.** The *path graph* G_{path} has:

- **Vertices**: equivalence classes of Z3 terms modulo syntactic identity.
- **Edges**: for each path constructor p and each applicable subterm, an edge from the term to the rewritten term.

The path search is a graph search from s to t in G_{path}.

The path graph is infinite (Z3 terms form an infinite set), but the search is bounded because:

1. Each path constructor application either *simplifies* the term (reduces its size/complexity) or *normalizes* it (moves it toward a canonical form).
2. A confluent and terminating rewrite system has a unique normal form per equivalence class — if both s and t reduce to the same normal form, a path exists.

18.3 The Normalization Strategy

Rather than searching the full path graph, we normalize both terms and check if the normal forms coincide:

Definition 18.3 (Canonical Normal Form)**.** The *canonical normal form* of a Z3 term t is obtained by exhaustively applying the following rewrite rules (in priority order):

Phase 1: β-reduction (definitional equality).

1. Inline all nested function applications ($(\lambda x.\, B)(a) \to B[x := a]$).
2. Unfold all `__func__` references.
3. Collapse trivial `If` branches ($\mathsf{If}(\top, a, b) \to a$; $\mathsf{If}(\bot, a, b) \to b$).

Phase 2: Arithmetic normalization (ring axioms).

4. Commute operands to canonical order ($b + a \to a + b$ when $a <_{\mathsf{lex}} b$).
5. Associate right ($((a + b) + c) \to (a + (b + c))$).
6. Eliminate identities ($a + 0 \to a$; $a \times 1 \to a$).
7. Absorb zeros ($a \times 0 \to 0$).
8. Cancel double negation ($-(-a) \to a$).

Phase 3: Fold canonicalization (Nat-recursor).

9. Detect recursive descent patterns and replace with canonical $\mathsf{fold}(\mathsf{op}, \mathsf{start}, \mathsf{stop}, \mathsf{acc})$.
10. Detect accumulation loops and replace with canonical fold.
11. When both rec and iter produce fold terms with the same $(\mathsf{op}, \mathsf{start}, \mathsf{stop}, \mathsf{acc})$, they are syntactically identical.

Phase 4: Higher-order normalization (operadic).

12. Apply map–fold fusion ($\mathsf{reduce}(\oplus, e, \mathsf{map}(h, xs)) \to \mathsf{reduce}(\lambda a, x.\, a \oplus h(x), e, xs)$).
13. Apply comprehension canonicalization ($[e(x) \text{ for } x \text{ in } xs] \to \mathsf{comp}_{\mathsf{hash}(e)}(xs)$).
14. Apply idempotence ($\mathsf{sorted}(\mathsf{sorted}(x)) \to \mathsf{sorted}(x)$).
15. Apply involutions ($\mathsf{reversed}(\mathsf{reversed}(x)) \to x$).

Phase 5: Shared symbol unification (univalence).

16. Replace all builtin calls with shared function symbols ($\mathsf{sorted}_k(x) \to \mathsf{py_sorted}(x)$).
17. Replace all method calls with shared method symbols ($\mathsf{meth_append}_k(o, v) \to \mathsf{py_meth_append}(o, v)$).

Theorem 18.4 (Normalization Confluence)**.** *The rewrite system of Definition 18.3 is* locally confluent*: if $t \to t_1$ and $t \to t_2$ by different rules, there exists t' such that $t_1 \to^* t'$ and $t_2 \to^* t'$.*

Combined with termination (each rule decreases a complexity measure), the system is confluent *(by Newman's lemma): every term has a unique normal form.*

Proof sketch. The phases are ordered by priority, and rules within each phase are orthogonal (they apply to disjoint subterms). Critical pairs arise only between commutativity and associativity, which are resolved by the canonical ordering $<_{\mathsf{lex}}$. □

Corollary 18.5 (Path Existence from Normal Forms)**.** *A cubical path from $[\![f]\!]$ to $[\![g]\!]$ exists if and only if their canonical normal forms are identical:*

$$\mathsf{nf}([\![f]\!]) = \mathsf{nf}([\![g]\!]) \iff \exists p : \mathsf{Path}([\![f]\!], [\![g]\!])$$

The path is reconstructed by composing the rewrite steps: $[\![f]\!] \to^* \mathsf{nf}([\![f]\!]) = \mathsf{nf}([\![g]\!]) \leftarrow^* [\![g]\!]$.

18.4 The Path Search Algorithm

Listing 18.1: Automated path search

```
def search_path(term_f, term_g, theory):
    """Search for a cubical path from term_f to term_g.

    Returns:
      PathResult(found=True, path=[step1, step2, ...])
      or PathResult(found=False, reason=...)
    """
    # Phase 1-5: normalize both terms
    nf_f, steps_f = normalize(term_f, theory)
    nf_g, steps_g = normalize(term_g, theory)

    # Check syntactic equality of normal forms
    if z3_syntactic_eq(nf_f, nf_g):
        # Path found! Compose forward and backward steps.
        path = steps_f + reversed(steps_g)
        return PathResult(found=True, path=path)

    # Normal forms differ: try Z3 semantic check
    # (maybe the terms are equal but our rewrite system
    # doesn't know the right axioms)
    solver = Solver()
    theory.semantic_axioms(solver)
    solver.add(nf_f != nf_g)
    result = solver.check()

    if result == unsat:
        # Z3 proved equality using axioms we didn't rewrite
        return PathResult(found=True,
                          path=steps_f + ['z3_axiom'] + reversed(
                              steps_g))
    elif result == sat:
        # Genuine counterexample: no path exists
        return PathResult(found=False,
                          counterexample=solver.model())
    else:
        return PathResult(found=None, reason='timeout')
```

18.5 Path Search on Each Fiber

The path search is performed *per fiber* (per type assignment), not globally. This is the cubical–cohomological interaction:

Listing 18.2: Fiber-wise path search

```
def fiber_wise_path_search(secs_f, secs_g, sites, theory):
    """Search for paths on each fiber independently.

    Returns per-fiber path results and the global H^1.
    """
    local_paths = {}

    for combo, site_id in sites.items():
        # Restrict terms to this fiber
```

```
        fiber_f = restrict_to_fiber(secs_f, combo, theory)
        fiber_g = restrict_to_fiber(secs_g, combo, theory)

        # Search for path on this fiber
        for sf in fiber_f:
            for sg in fiber_g:
                if not guards_overlap(sf.guard, sg.guard):
                    continue
                result = search_path(sf.term, sg.term, theory)
                if result.found == False:
                    return GlobalResult(equiv=False,
                        fiber=combo, cx=result.counterexample)
                elif result.found == True:
                    local_paths[site_id] = result.path

    # All fibers have paths: check gluing via Cech H^1
    h1 = compute_cech_h1(local_paths, sites)
    if h1 == 0:
        return GlobalResult(equiv=True, paths=local_paths)
    else:
        return GlobalResult(equiv=False, obstruction=h1)
```

18.6 Path Composition and the Proof Certificate

When the path search succeeds, it produces a *proof certificate*: a sequence of named path constructor applications that transforms $[\![f]\!]$ into $[\![g]\!]$.

Definition 18.6 (Proof Certificate)**.** A *proof certificate* for $f \equiv g$ is a list of triples:

$$[(r_1, \mathsf{pos}_1, \mathsf{ax}_1),\ (r_2, \mathsf{pos}_2, \mathsf{ax}_2),\ \ldots,\ (r_k, \mathsf{pos}_k, \mathsf{ax}_k)]$$

where r_i is the term at step i, pos_i is the subterm position, and ax_i is the axiom applied. The certificate satisfies:

$$r_0 = [\![f]\!] \tag{18.1}$$
$$r_{i+1} = r_i[\mathsf{pos}_i := \mathsf{ax}_i(r_i|_{\mathsf{pos}_i})] \tag{18.2}$$
$$r_k = [\![g]\!] \tag{18.3}$$

Example 18.7 (Certificate for Factorial Equivalence)**.**

$$[\![f]\!] = \mathsf{rec_f}(p_0) \qquad \text{(recursive definition)} \tag{18.4}$$
$$\xrightarrow{\mathsf{D1},\,\mathrm{root}} \mathsf{fold}(\mathsf{MUL}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(1)) \qquad \text{(Nat-eliminator extraction)} \tag{18.5}$$
$$= [\![g]\!] \qquad \text{(same as iterative fold)} \tag{18.6}$$

Certificate: $[(\mathsf{rec_f}(p_0), \epsilon, \mathsf{D1})]$. One step, at the root, using the Nat-recursor path.

Example 18.8 (Multi-Step Certificate)**.** Programs that compute $\mathsf{sum}(\mathsf{reversed}(\mathsf{sorted}(x)))$ vs. $\mathsf{sum}(x)$:

$$\mathsf{py_sum}(\mathsf{py_reversed}(\mathsf{py_sorted}(p_0))) \tag{18.7}$$
$$\xrightarrow{\mathsf{reversed_invol},\,[1,1]} \mathsf{py_sum}(\mathsf{py_sorted}(p_0)) \qquad \text{(sorted absorbs reversed)} \tag{18.8}$$
$$\xrightarrow{\mathsf{sum_perm},\,[1]} \mathsf{py_sum}(p_0) \qquad \text{(sum is perm-invariant)} \tag{18.9}$$

Certificate: $[(\ldots, [1,1], \mathsf{sorted_absorbs_reversed}),\ (\ldots, [1], \mathsf{sum_perm_invariant})]$.

18.7 Path Search Strategies

The basic normalization strategy (Definition 18.3) is "normalize both sides and compare." But more sophisticated strategies exist:

18.7.1 Bidirectional Search

Search from both $[\![f]\!]$ and $[\![g]\!]$ simultaneously, meeting in the middle:

Listing 18.3: Bidirectional path search

```
def bidirectional_search(s, t, axioms, max_depth=5):
    forward = {hash(s): (s, [])}     # term -> (term, path)
    backward = {hash(t): (t, [])}

    for depth in range(max_depth):
        # Expand forward frontier
        new_forward = {}
        for h, (term, path) in forward.items():
            for ax in axioms:
                for pos in applicable_positions(term, ax):
                    new_term = apply_axiom(term, ax, pos)
                    nh = hash(new_term)
                    if nh in backward:
                        # FOUND: path from s to t
                        _, bpath = backward[nh]
                        return path + [(term, pos, ax)] + \
                               reversed(bpath)
                    new_forward[nh] = (new_term,
                        path + [(term, pos, ax)])
        forward.update(new_forward)

        # Similarly expand backward frontier
        # ...

    return None  # no path within depth bound
```

18.7.2 Fiber-Guided Search

Use the Čech decomposition to *guide* the search:

1. Identify which fibers are "easy" (terms already match or nearly match after normalization).
2. On hard fibers, use the easy fibers' solutions as hints: if the path on the Int fiber used axiom D1, try D1 on the Str fiber first.
3. This is *transfer along the fibration*: a successful path on one fiber is transported to adjacent fibers.

18.7.3 Axiom Relevance Filtering

Not all axioms are relevant for a given pair. Filter by:

1. **Symbol filtering**: only use axioms mentioning symbols that appear in $[\![f]\!]$ or $[\![g]\!]$.
2. **Sort filtering**: only use axioms whose type signature matches the current fiber.
3. **Depth filtering**: prioritize axioms that reduce term depth (simplifying rules before complexifying ones).

18.8 Sheaf Descent for Tractability of Undecidable Problems

Program equivalence is undecidable in general (Rice's theorem). Specification checking and bug finding reduce to it, so they are also undecidable. Yet we claim CTTT makes these problems *tractable* for a large class of real programs. This section explains *why* — the mathematical mechanism by which sheaf descent converts undecidable global problems into decidable local ones.

18.8.1 The Undecidability Barrier

The global equivalence query is:

$$\forall x : \mathsf{PyObj}.\ [\![f]\!](x) = [\![g]\!](x)$$

This is a universal quantification over an infinite domain (PyObj has infinitely many inhabitants via $\mathsf{IntObj}(n)$ for all $n \in \mathbb{Z}$). No algorithm can decide this in general.

But the undecidability proof (Rice's theorem) requires adversarial programs — programs specifically constructed to defeat any analysis. Real programs are not adversarial. They have *structure* that can be exploited.

18.8.2 How Descent Exploits Structure

Sheaf descent exploits two kinds of structure:

1. Type structure (the fibration). Real Python programs dispatch on types — they use `isinstance`, duck typing, different code paths for different input types. This means the universal quantification decomposes into a *finite* disjunction:

$$\forall x.\ [\![f]\!](x) = [\![g]\!](x) \iff \bigwedge_{\tau \in \mathsf{TypeTag}} \forall x.\ \mathsf{tag}(x) = \tau \implies [\![f]\!](x) = [\![g]\!](x)$$

Each conjunct is a *fiber-restricted* query. The finite conjunction is decidable if each conjunct is.

2. Algebraic structure (the path constructors). Within each fiber, the Z3 terms have algebraic structure: commutativity, associativity, fold patterns. The path constructors (equational axioms) transform this structure, often collapsing complex terms to simple canonical forms.

The canonical form is unique (by confluence, Theorem 18.4), and comparing canonical forms is decidable (syntactic equality check).

18.8.3 The Descent Decomposition Theorem

Theorem 18.9 (Descent Makes Equivalence Tractable). *Let $\mathcal{U} = \{U_\tau\}_{\tau \in T}$ be the covering of* PyObj *by type fibers, where T is a finite set of type tags. Then:*

1. *The global equivalence query $\forall x.\ [\![f]\!](x) = [\![g]\!](x)$ reduces to $|T|$ local queries plus one H^1 computation.*
2. *Each local query is in the theory QF_DT + QF_LIA + QF_S (quantifier-free datatypes + linear integer arithmetic + strings), which is* decidable.
3. *The H^1 computation is decidable ($O(|T|^3)$ via GF(2) rank).*
4. *Therefore: the full equivalence check is decidable for programs whose Z3 denotations fall in the quantifier-free fragment (no RecFunctions, no universally quantified axioms).*

Proof. (1) By the descent theorem (7.6), the global query decomposes into local queries on each fiber plus the gluing check ($H^1 = 0$).

(2) On each fiber U_τ, the type is fixed (e.g., $\mathsf{tag}(x) = \mathsf{TInt}$ forces $x = \mathsf{IntObj}(n)$ for some n). The Z3 terms reduce to expressions over $\mathsf{IntSort}$ (decidable QF_LIA), $\mathsf{BoolSort}$ (propositional, decidable), $\mathsf{StringSort}$ (decidable QF_S), or PyObj constructors (decidable QF_DT). The combined theory QF_DT + QF_LIA + QF_S is decidable by Nelson–Oppen combination.

(3) GF(2) linear algebra is decidable, $O(n^3)$.

(4) Combining (1)–(3): decidable + decidable + decidable = decidable. □

Remark 18.10 (Beyond the Quantifier-Free Fragment)**.** When programs use RecFunctions (from loops/recursion), the local queries may exit the QF fragment. The Nat-eliminator extraction (Chapter 27) recovers decidability for many such programs by converting RecFunctions to canonical folds, which are ground terms (no quantifiers).

When equational axioms involve universal quantifiers ($\forall x.\, \mathsf{sorted}(\mathsf{sorted}(x)) = \mathsf{sorted}(x)$), the local queries are semi-decidable (Z3 may timeout). But the axioms are applied *as rewrite rules* during normalization, not as quantified constraints — so they terminate (the rewrite system is terminating by Theorem 18.4).

18.8.4 Complexity Analysis

Theorem 18.11 (Complexity of Descent-Based Equivalence)**.** *For programs with n parameters, each having at most k possible type tags, the descent-based equivalence check has complexity:*

$$O(k^n \cdot C_{\mathsf{local}} + k^{3n})$$

where C_{local} is the cost of a single local Z3 query.

For the common case of single-typed parameters ($k = 1$):

$$O(C_{\mathsf{local}} + 1) = O(C_{\mathsf{local}})$$

— a single Z3 query, no combinatorial blowup.

For the worst case ($k = 7$ types, n parameters): $k^n = 7^n$ fibers — exponential in the number of parameters. But duck type inference typically reduces k to 1–2 per parameter, so the practical complexity is $O(2^n \cdot C_{\mathsf{local}})$.

18.8.5 Comparison with Prior Approaches

Approach	Global query	Decidable?	Practical?
Direct Z3	$\forall x.\, f(x) = g(x)$	Semi-decidable	Slow/timeout
Testing	Sample $x_1, \ldots, x_N$	Incomplete	Fast but unsound
Abs. interp.	$\alpha(f) = \alpha(g)$	Decidable	Fast but imprecise
CTTT descent	$\bigwedge_\tau (\forall x_\tau.\, f = g) \wedge H^1 = 0$	**Decidable (QF)**	**Fast & sound**

CTTT descent is the only approach that is simultaneously decidable (for the QF fragment), sound (no false positives/negatives within the modeled domain), and practical (scales with duck-type narrowing).

18.8.6 The Refinement Principle

When descent alone is insufficient (the local Z3 query times out or returns "unknown"), we can *refine* the covering:

Definition 18.12 (Covering Refinement)**.** A refinement $\mathcal{U}' = \{V_j\}$ of covering $\mathcal{U} = \{U_i\}$ is a covering such that each $V_j \subseteq U_i$ for some i. Refinement makes fibers *smaller* (more constrained), which makes local queries *easier*.

Theorem 18.13 (Refinement Soundness)**.** *If the equivalence check succeeds on a refinement $\mathcal{U}'$ ($H^1(\mathcal{U}', \mathsf{Iso}) = 0$), it also succeeds on the original covering ($H^1(\mathcal{U}, \mathsf{Iso}) = 0$).*

The converse is NOT true: a refinement may introduce new overlaps and new transition checks, which could fail even if the original covering succeeds.

Proof. The refinement morphism $\mathcal{U}' \to \mathcal{U}$ induces a morphism of Čech complexes $C^\bullet(\mathcal{U}) \to C^\bullet(\mathcal{U}')$. By functoriality of cohomology, $H^1(\mathcal{U}) = 0$ implies $H^1(\mathcal{U}') = 0$ (the image of a trivial class is trivial). □

Refinement strategies:

1. **Value-range refinement**: split the Int fiber into $\{n \geq 0\}$ and $\{n < 0\}$ (positive and negative integers). This helps with programs that have different behavior for positive vs. negative inputs.

2. **Size refinement**: split the Pair fiber into $\{|xs| = 0\}$, $\{|xs| = 1\}$, $\{|xs| \geq 2\}$ (empty, singleton, longer lists). This helps with programs that have base cases for short inputs.

3. **Predicate refinement**: split a fiber by a predicate derived from the program's branch conditions. E.g., if f branches on $n \leq 1$, split the Int fiber at $n = 1$.

Theorem 18.14 (Predicate Refinement from Branch Conditions)**.** *Let* $\mathsf{guards}_f = \{g_1, \ldots, g_m\}$ *and* $\mathsf{guards}_g = \{h_1, \ldots, h_p\}$ *be the branch conditions of programs f and g respectively. The* branch-condition refinement *is the covering:*

$$\mathcal{U}_{\mathsf{branch}} = \{U \cap R \mid U \in \mathcal{U},\ R \in \mathsf{atoms}(\{g_i\} \cup \{h_j\})\}$$

where atoms *are the minimal non-empty regions of the partition induced by the guards.*

On each atom, both *programs follow a single code path (no branching), so their Z3 terms are ground expressions (no* If*), which are decidable.*

Proof. Within an atom R, all guard conditions have fixed truth values. Therefore $\mathsf{If}(\mathsf{guard}, t, f)$ simplifies to t or f (no conditional). The resulting Z3 terms are quantifier-free ground expressions over the base theories, which are decidable. □

Corollary 18.15 (Branch-Refined Descent is Decidable)**.** *The branch-condition refinement produces decidable local queries for* any *program (not just QF programs). The only remaining source of undecidability is the RecFunction bodies (loops/recursion).*

This is a powerful result: by refining the covering using the programs' own branch conditions, we eliminate branching complexity from the local queries, leaving only loop/recursion complexity.

18.8.7 Iterated Descent: The Descent Tower

For programs with nested structure (loops inside branches inside loops), we can apply descent *iteratively*:

Definition 18.16 (Descent Tower)**.** A *descent tower* is a sequence of coverings:

$$\mathcal{U}_0 \leftarrow \mathcal{U}_1 \leftarrow \mathcal{U}_2 \leftarrow \ldots \leftarrow \mathcal{U}_k$$

where each $\mathcal{U}_{i+1}$ is a refinement of $\mathcal{U}_i$.

Level 0: type fibers (Int, Str, List, ...). Level 1: type fibers × branch conditions. Level 2: type fibers × branch conditions × loop bounds. Level 3: type fibers × branch conditions × loop bounds × recursion depth.

At each level, the local queries become simpler (more constrained, fewer degrees of freedom). Eventually, the local queries are ground and decidable.

Theorem 18.17 (Descent Tower Convergence)**.** *For programs with finitely many branches and bounded-depth recursion, the descent tower converges at a finite level k: all local queries at level k are ground and decidable.*

Proof. Each level refines by a finite partition (finitely many branches, finitely many loop bound values). The atoms at level k correspond to complete execution traces of bounded depth. On each trace, the Z3 term is ground (no If, no RecFunction unfolding needed beyond the bound). □

Remark 18.18 (Unbounded Recursion)**.** For programs with unbounded recursion, the descent tower does NOT converge. This is the residual source of undecidability in CTTT. The Nat-eliminator extraction (Chapter 27) handles many unbounded cases by recognizing them as folds, but general recursion remains semi-decidable.

18.8.8 The Effective Descent Condition

Not every covering admits effective descent. We characterize when descent is most useful:

Definition 18.19 (Effective Descent)**.** A covering $\mathcal{U}$ admits *effective descent* for the equivalence check $f \equiv g$ if:

1. Each local query $\mathsf{Iso}(U_i)$ is decidable (terminates with a definite answer).
2. The number of overlaps is polynomial in $|\mathcal{U}|$ (the Čech complex is sparse).
3. The transition checks φ_{ij} are all trivial (both local paths agree automatically on overlaps).

Theorem 18.20 (When Descent is Effective)**.** *Descent is effective when:*

1. ***Duck type inference is sharp**: each parameter has a unique type (no polymorphism). Then $|\mathcal{U}| = 1$, no overlaps, no transitions. This is the common case.*
2. ***Fibers are independent**: the program's behavior on fiber U_i does not depend on behavior on fiber U_j. Then transitions are trivial.*
3. ***Branch conditions align**: both programs branch on the same conditions. Then the branch-condition refinement produces the same atoms for both.*

18.8.9 Extensions: Beyond Type Fibers

The descent framework is not limited to type-based coverings. Any *partition* of the input space that respects program structure can serve as a covering:

1. **Value-range covering**: partition by value ranges ($n < 0$, $n = 0$, $n > 0$, $n > 100$, etc.). Useful for programs with thresholds.
2. **Shape covering**: partition by data structure shape (empty list, singleton, doubleton, general list). Useful for recursive programs with base cases.
3. **Reachability covering**: partition by reachable code paths (which branches are taken). This is the branch-condition refinement from Theorem 18.14.
4. **Symmetry covering**: partition by symmetry group orbits (inputs related by permutation). Useful for programs whose output is permutation-invariant.

5. **Norm covering**: partition by norm/size ($|x| \leq k$ for various k). The spectral sequence (Chapter 13) is the cohomology of this filtration.

Each covering type provides a different decomposition strategy, and the optimal covering depends on the programs being compared. The *covering selection heuristic* is:

1. Start with type fibers (cheapest, most effective for well-typed programs).
2. If local queries fail, refine by branch conditions.
3. If still failing, refine by value ranges or shapes.
4. If still failing, try symmetry or norm coverings.

This hierarchical refinement is the *spectral sequence in action*: each refinement level corresponds to a page of the spectral sequence, and the sequence converges when the local queries become decidable.

18.9 Soundness and Completeness

Theorem 18.21 (Path Search Soundness)**.** *If the path search returns a certificate* $[r_0, \ldots, r_k]$*, then* $r_0 = [\![f]\!]$*,* $r_k = [\![g]\!]$*, and each step* $r_i \to r_{i+1}$ *is justified by a valid path constructor. Therefore* $[\![f]\!] \equiv [\![g]\!]$ *(the programs are semantically equivalent).*

Proof. Each path constructor is a theorem of the cubical type theory (proved in the relevant dichotomy chapter). Kan composition (Chapter 5) ensures the steps compose to a valid path. □

Theorem 18.22 (Path Search Completeness (Relative))**.** *The path search is* complete relative to the axiom set*: if a path exists using only the axioms in the HIT* PyComp*, the normalization procedure finds it (assuming the rewrite system is confluent and terminating for the applicable axiom subset).*

The path search is incomplete *in general: there exist equivalent programs for which no path exists in* PyComp *(because the axiom set doesn't cover all valid equalities — Rice's theorem prevents completeness).*

18.10 Path Search for Spec Checking

The same path search engine handles spec checking by searching for a path from $[\![f]\!]$ to $[\![S]\!]$ (the specification's canonical form):

Listing 18.4: Spec check via path search

```
def spec_check(program_source, spec_source, theory):
    """Check if program satisfies spec via path search.

    spec_source is a reference implementation or a Z3 formula.
    """
    term_f = compile(program_source, theory)
    term_spec = compile(spec_source, theory)

    result = fiber_wise_path_search(
        sections_of(term_f),
        sections_of(term_spec),
        build_sites(program_source, spec_source, theory),
        theory
    )
```

```

    if result.equiv:
        return SpecResult(satisfies=True,
                          certificate=result.paths)
    else:
        return SpecResult(satisfies=False,
                          bug_fiber=result.fiber,
                          counterexample=result.cx)
```

18.11 Path Search for Bug Finding

Bug finding is path search with *failure analysis*: when no path exists on a fiber, the failure point reveals the bug.

Listing 18.5: Bug finding via failed path search

```
def find_bugs(program_source, spec_source, theory):
    """Find bugs by attempting path search on each fiber.

    Returns list of (fiber, depth, counterexample) triples.
    """
    bugs = []
    term_f = compile(program_source, theory)
    term_spec = compile(spec_source, theory)
    sites = build_sites(program_source, spec_source, theory)

    for combo, site_id in sites.items():
        result = search_path(
            restrict(term_f, combo),
            restrict(term_spec, combo),
            theory
        )
        if result.found == False:
            bugs.append(Bug(
                fiber=combo,
                counterexample=result.counterexample,
                # How far did the path get before failing?
                partial_path=result.partial_steps,
                # What axiom would be needed?
                missing_axiom=result.stuck_on
            ))

    return BugReport(bugs=bugs,
                     severity=len(bugs) / len(sites),
                     h1_rank=compute_h1_rank(bugs, sites))
```

The `partial_path` and `missing_axiom` fields provide *diagnostic information*: they tell the developer not just *that* the program is wrong, but *why* the proof failed and *what kind of fix* is needed.

18.12 Path-Based Automatic Repair Suggestion

When the path search fails at a specific step, the failure point suggests a repair:

Theorem 18.23 (Repair from Path Failure)**.** *If the path search from* $[\![f]\!]$ *toward* $[\![S]\!]$ *succeeds for* k *steps and fails at step* $k+1$ *because axiom* ax *is not applicable:*

$$[\![f]\!] \xrightarrow{p_1} \cdots \xrightarrow{p_k} h_k \xrightarrow{\mathsf{ax?}} STUCK$$

then the repair *is: modify* f *so that* h_k *satisfies the precondition of* ax*. The modification is localized to the subterm at position* pos_{k+1} *of* h_k*.*

Example 18.24. A program tries to compute the sum of a list but uses multiplication instead of addition:

```
1 def f(xs):
2     acc = 0
3     for x in xs: acc *= x   # BUG: should be +=
4     return acc
```

Path search toward spec $S = \mathsf{py_sum}(xs)$:

1. $[\![f]\!] = \mathsf{fold}(\mathsf{MUL}, \ldots)$ (detected from loop).
2. Normalize: $\mathsf{fold}(\mathsf{MUL}, 0, \ldots) = 0$ (because $0 \times \text{anything} = 0$ by absorbing zero axiom).
3. Target: $\mathsf{py_sum}(xs)$ normalizes to $\mathsf{fold}(\mathsf{ADD}, 0, \ldots)$.
4. STUCK: $\mathsf{fold}(\mathsf{MUL}, \ldots) \neq \mathsf{fold}(\mathsf{ADD}, \ldots)$. The path constructor D1 applies but the operation tag differs: $\mathsf{MUL} \neq \mathsf{ADD}$.

Repair suggestion: "Change MUL to ADD at the loop accumulation step (line 3: `*=` → `+=`)."

Part V

The Twenty-Four Dichotomies

Chapter 19

Overview of the Dichotomies

We identify twenty-four fundamental *computational dichotomies* — pairs of implementation strategies that compute the same function. Each dichotomy is a path constructor in the HIT PyComp (Definition 4.1).

The dichotomies are organized into four groups:

Group	Dichotomies
Control Flow	D1–D8
Data Structure	D9–D14
Algorithm Strategy	D15–D20
Language Feature	D21–D24

19.1 Complete Dichotomy Table

#	Dichotomy	Path Constructor	Z3 Theory
Control Flow Dichotomies			
D1	Recursive ↔ Iterative	Nat-eliminator uniqueness	Canonical fold
D2	Nested helpers ↔ Flat	β-reduction	Function inlining
D3	`while` ↔ `for`	Loop normal form	RecFunction equiv
D4	Comprehension ↔ Loop	Catamorphism fusion	Shared comp symbol
D5	`reduce` ↔ Loop	Fold universality	Fold RecFunction
D6	Generator ↔ Eager	Laziness adjunction	Sequence identity
D7	Tail-recursive ↔ Loop	CPS transform	Stack elimination
D8	Multi-return ↔ Single	Section merging	If-expression norm
Data Structure Dichotomies			
D9	Stack ↔ Recursion	Defunctionalization	Same traversal fold
D10	`heapq` ↔ `bisect`	Priority queue abstr.	Shared extract-min
D11	`OrderedDict` ↔ LinkedList	ADT refinement	Same interface fold
D12	`defaultdict` ↔ `setdefault`	Map construction equiv	Shared insert-or-get
D13	`Counter` ↔ Manual count	Histogram equivalence	Fold over freq
D14	Array ↔ Dict table	Indexing abstraction	Shared lookup

#	**Dichotomy**	**Path Constructor**	**Z3 Theory**
Algorithm Strategy Dichotomies			
D15	BFS ↔ DFS	Traversal-order invariance	Comm. monoid fold
D16	Memoized ↔ DP	Eval-order independence	Same recurrence
D17	Divide-and-conquer ↔ Iterative	Tree fold = list fold	Associativity
D18	Greedy ↔ DP	Matroid/greedy theorem	Optimality axiom
D19	Sort-then-scan ↔ Sweep	Sorting abstraction	Shared sorted symbol
D20	Different algorithms for same spec	Specification HIT	Yoneda embedding
Language Feature Dichotomies			
D21	`isinstance` ↔ Dispatch	Fiber projection	Tag comparison
D22	`try/except` ↔ Conditional	Effect elimination	Guard equivalence
D23	Sorted-then-process ↔ Direct	Canonical ordering path	Shared sorted symbol
D24	Lambda ↔ Named function	η-expansion	Shared call symbol

Chapter 20

Control Flow Dichotomies (D1–D8)

Each section below follows the same structure: (1) two concrete programs illustrating paradigm A and paradigm B, (2) how each is compiled to Z3 terms, (3) the explicit path connecting them, and (4) why the path is *automatically inferred* with no human guidance.

20.1 D1: Recursive ↔ Iterative

Paradigm A — recursive descent:

```
def f(n):
    if n <= 1: return 1
    return n * f(n - 1)
```

Paradigm B — iterative accumulation:

```
def g(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result
```

Compilation of A. The compiler sees a self-recursive function with one parameter. It invokes the *Nat-eliminator extractor*, which detects:

- Base guard: `n <= 1`, so $\mathsf{threshold} = 1$.
- Base value: literal `1`, so $\mathsf{base} = 1$.
- Step: `n * f(n-1)`, a $\mathsf{BinOp}(\times)$ with the parameter on the left and the recursive call on the right. The operation tag is MUL.

The extractor produces: $[\![f]\!] = \mathsf{fold}(\mathsf{MUL}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(1))$.

Compilation of B. The compiler sees an assignment `result = 1` (init), then `for i in range(2, n+1): result *= i`. The loop-to-fold optimizer detects:

- Single accumulator `result` with init $= 1$.
- `AugAssign` with Mult: `result *= i`.
- The loop variable `i` is the RHS of the augmented assignment, and the accumulator is the LHS.
- $\mathsf{range}(2, n + 1)$ gives $\mathsf{start} = 2$, $\mathsf{stop} = \mathsf{ival}(p_0) + 1$.

The optimizer produces: $[\![g]\!] = \mathsf{fold}(\mathsf{MUL}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(1))$.

Path. $[\![f]\!]$ and $[\![g]\!]$ are *the same Z3 term.* The path is refl (reflexivity) — no axiom application needed.

Automatic inference. The Nat-eliminator extractor and loop-to-fold optimizer both fire automatically on any function matching their structural criteria (base case + recursive step, or init + range loop + augmented assignment). No algorithm name, no problem-specific guidance. The path exists iff both sides produce the same fold term, which Z3 checks by syntactic equality (decidable, instant).

When the step has $f(n-1) * n$ instead of $n * f(n-1)$, the terms differ: $\mathsf{fold}(\mathsf{MUL}, \ldots)$ vs. $\mathsf{fold}(\mathsf{MUL}, \ldots)$ but with swapped operands inside the RecFunction body. The mul_comm path constructor bridges the gap: Z3's arithmetic solver proves $a \times b = b \times a$ as a tautology, so the inequality query returns UNSAT.

Theorem 20.1 (D1 Path Constructor)**.**

$$\mathsf{D1} : \prod_{\mathsf{base},\,\mathsf{step}} \mathsf{Path}(\mathsf{rec}_{\mathbb{N}}(\mathsf{base}, \mathsf{step}),\ \mathsf{fold}(\mathsf{step}, \mathsf{base}))$$

By the uniqueness of the Nat-eliminator, both satisfy the same recurrence with the same base case, so they are definitionally equal.

20.2 D2: Nested Helpers ↔ Flat Code

Paradigm A — nested helper function:

```
def f(x):
    def helper(a, b):
        return a * a + b * b
    return helper(x, x + 1)
```

Paradigm B — inlined flat code:

```
def g(x):
    return x * x + (x + 1) * (x + 1)
```

Compilation of A. The compiler encounters `def helper`: it stores (`__func__`, helper_AST) in the environment. When it reaches `helper(x, x+1)`, it invokes `inline_call`: substituting $a \mapsto p_0$ and $b \mapsto \mathsf{binop}(\mathsf{ADD}, p_0, \mathsf{IntObj}(1))$ into the body. Result:

$$[\![f]\!] = \mathsf{binop}(\mathsf{ADD},\ \mathsf{binop}(\mathsf{MUL}, p_0, p_0),\ \mathsf{binop}(\mathsf{MUL}, \mathsf{binop}(\mathsf{ADD}, p_0, \mathsf{IntObj}(1)), \mathsf{binop}(\mathsf{ADD}, p_0, \mathsf{IntObj}(1))))$$

Compilation of B. Direct compilation produces the identical Z3 term (same expression tree after constant folding by Z3's simplifier).

Path. $[\![f]\!] \equiv [\![g]\!]$ by β-reduction. The path is refl after inlining.

Automatic inference. The compiler *always* attempts to inline nested function calls. When the nested function has a known AST (stored in the environment as `__func__`), inlining is triggered without any user guidance. The result is a definitional equality — the strongest form of path in cubical type theory.

This handles 22 benchmark pairs because many programs define helper functions (for readability, recursion, or closure capture) that the flat version inlines directly.

20.3 D3: `while` ↔ `for`

Paradigm A — `while` with explicit counter:

```
def f(n):
    result, i = 0, 1
    while i <= n:
        result += i
        i += 1
    return result
```

Paradigm B — `for` over range:

```
def g(n):
    result = 0
    for i in range(1, n + 1):
        result += i
    return result
```

Compilation of A. The compiler encounters a `while` loop. It identifies modified variables $\{\texttt{result}, \texttt{i}\}$ and creates a RecFunction for each:

$$\mathsf{wh_result}(k) = \begin{cases} \mathsf{IntObj}(0) & \text{if } k > 50 \vee \neg(i \leq n) \\ \mathsf{binop}(\mathsf{ADD}, \mathsf{wh_result}(k+1), \ldots) & \text{otherwise} \end{cases}$$

Compilation of B. The compiler encounters `for i in range(1, n+1)`. It extracts $\mathsf{start} = 1$, $\mathsf{stop} = \mathsf{ival}(p_0) + 1$. It recognizes the single-accumulator fold pattern (`result += i`) and produces:

$$[\![g]\!] = \mathsf{fold}(\mathsf{ADD}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(0))$$

Path. Both RecFunctions satisfy the same recurrence: $h(0) = 0$, $h(k) = h(k-1) + k$. The `while` version's guard $i \leq n$ is equivalent to the `for` range $[1, n+1)$, and both step by 1. The RecFunctions unfold identically.

When Z3 checks $[\![f]\!](p_0) \neq [\![g]\!](p_0)$ on the integer fiber ($\mathsf{tag}(p_0) = \mathsf{TInt}$), it finds UNSAT because both RecFunctions compute the same value for every integer input.

Automatic inference. The compiler automatically canonicalizes both loop forms to RecFunctions. The `for`-range version gets the additional fold optimization. Z3's recursive function reasoning handles the comparison. The path is: $\mathsf{D3} = \mathsf{while_rec} \xrightarrow{\text{unfold}} \mathsf{for_fold}$, mediated by the shared recurrence structure.

20.4 D4: Comprehension ↔ Loop

Paradigm A — list comprehension:

```
def f(xs):
    return [x * 2 for x in xs]
```

Paradigm B — explicit loop with append:

```
def g(xs):
    result = []
    for x in xs:
        result.append(x * 2)
    return result
```

Compilation of A. The compiler encounters a `ListComp`. It hashes the element expression AST (`x * 2`) to get a canonical key h, and produces:

$$[\![f]\!] = \mathsf{py_comp}_h(p_0)$$

using a *shared* function symbol keyed by h.

Compilation of B. The compiler sees `result = []` (init as NoneObj), then a `for` loop with `result.append(x * 2)`. The mutation tracker models `append` via the shared function: $\mathsf{py_mut_append}(\mathsf{result}, \mathsf{binop}(\mathsf{MUL}, \mathsf{loop_var}, \mathsf{IntObj}(2)))$. The final value of `result` is a chain of mut_append applications.

Path. Both programs apply the transformation `x * 2` to each element of `xs` and collect the results. The comprehension and the loop-with-append are *the same catamorphism* (fold over the input, accumulating into a list).

In the Z3 encoding, both produce the same shared function application when the comprehension body hash matches the loop body. The path constructor D4 identifies them:

$$\mathsf{D4} : \mathsf{Path}(\mathsf{py_comp}_h(\mathsf{xs}),\ \mathsf{fold}(\mathsf{append_body}_h, \mathsf{empty}, \mathsf{xs}))$$

Automatic inference. The compiler recognizes comprehensions and loop-with-append patterns automatically. The canonical hash of the body expression ensures that *any* comprehension with body e matches *any* loop that appends e. No knowledge of what the function "does" is needed.

20.5 D5: `reduce` ↔ Loop

Paradigm A — `functools.reduce`:

```
def f(xs):
    return reduce(operator.add, xs, 0)
```

Paradigm B — explicit accumulation loop:

```
def g(xs):
    acc = 0
    for x in xs:
        acc += x
    return acc
```

Compilation of A. `reduce` compiles to the shared function $\mathsf{py_call_reduce}(\mathsf{py_call_operator.add}, p_0, \mathsf{IntObj}$

Compilation of B. The loop compiles to a fold: $\mathsf{fold}(\mathsf{ADD}, 0, \mathsf{len}(p_0), \mathsf{IntObj}(0))$ or a RecFunction with the same structure.

Path. The path D5 asserts that $\mathtt{reduce}(\mathsf{op}, \mathsf{xs}, e)$ is *definitionally equal* to the fold catamorphism with the same operation, initial value, and sequence. Both are the unique catamorphism from the initial algebra of lists. By the universal property, they agree.

Automatic inference. The compiler recognizes `reduce` as a call to the shared py_call_reduce symbol, and recognizes the loop as a fold. The shared symbol for the operation (`operator.add` ↦ ADD) ensures both use the same Z3 operation.

20.6 D6: Generator ↔ Eager

Paradigm A — generator with `yield`:

```
def f(data):
    def helper(obj):
        if isinstance(obj, list):
            for item in obj:
                yield from helper(item)
        else:
            yield obj
    return list(helper(data))
```

Paradigm B — eager list building:

```python
def g(data):
    result = []
    stack = [data]
    while stack:
        obj = stack.pop()
        if isinstance(obj, list):
            stack.extend(reversed(obj))
        else:
            result.append(obj)
    return result
```

Compilation. Both programs traverse a nested structure and collect atomic elements. The generator version yields elements lazily; the eager version appends them to a list. When the generator is fully consumed by `list()`, the result is identical.

In Z3, both compile to sequences of shared function applications on the input. The generator's `yield` is modeled as producing a Pair-chain (the same representation as a list). The `list()` wrapper is a no-op on a sequence that is already fully materialized.

Path. $\mathsf{D6} : \mathsf{Path}(\mathsf{list}(\mathsf{gen}(\mathsf{body})), \mathsf{eager}(\mathsf{body}))$ where $\mathsf{list} \circ \mathsf{gen}$ is the "consume generator" operation. This path holds because full consumption of a generator produces the same sequence as eager construction — both visit the same elements in the same order.

Automatic inference. The compiler detects `yield` and `yield from` and models the generator as producing a sequence. The `list()` call (or `for` loop consumption) triggers the D6 path. The traversal order equivalence (generator DFS vs. stack-with-reversed DFS) is handled by D9 (Stack $\leftrightarrow$ Recursion).

20.7 D7: Tail-Recursive $\leftrightarrow$ Loop

Paradigm A — tail-recursive with accumulator:

```python
def f(n, acc=0):
    if n <= 0: return acc
    return f(n - 1, acc + n)
```

Paradigm B — iterative with accumulator:

```python
def g(n):
    acc = 0
    while n > 0:
        acc += n
        n -= 1
    return acc
```

Compilation. The tail-recursive version has the recursive call in tail position: $f(n-1, \mathsf{acc}+n)$. The accumulator acc is passed as a parameter rather than composed in the return value (contrast with D1 where the return value is $n \times f(n-1)$).

The compiler detects tail position and creates a RecFunction:

$$\mathsf{rec_f}(n, \mathsf{acc}) = \begin{cases} \mathsf{acc} & n \leq 0 \\ \mathsf{rec_f}(n-1, \mathsf{acc}+n) & \text{otherwise} \end{cases}$$

The iterative version produces the same RecFunction via the while loop compiler (D3), with acc as the modified variable and n as the counter.

Path. D7 identifies tail recursion with iteration. This is the CPS transform in action: a tail-recursive function is a loop with the accumulator as explicit state. Both RecFunctions have the same defining equation, so the path is refl.

Automatic inference. The compiler detects tail position by checking whether the recursive call is the *last* operation before return (no wrapping BinOp). When detected, it generates the same RecFunction structure as the while-loop compiler, producing identical Z3 terms.

20.8 D8: Multi-Return ↔ Single Return

Paradigm A — early returns:

```
def f(x):
    if x < 0: return -1
    if x == 0: return 0
    return 1
```

Paradigm B — single return with nested conditional:

```
def g(x):
    return -1 if x < 0 else (0 if x == 0 else 1)
```

Compilation of A. The section extractor produces three sections:

$$(\mathsf{ival}(p_0) < 0,\ \mathsf{IntObj}(-1)) \tag{20.1}$$

$$(\mathsf{ival}(p_0) = 0 \wedge \neg(\mathsf{ival}(p_0) < 0),\ \mathsf{IntObj}(0)) \tag{20.2}$$

$$(\neg(\mathsf{ival}(p_0) < 0) \wedge \neg(\mathsf{ival}(p_0) = 0),\ \mathsf{IntObj}(1)) \tag{20.3}$$

Compilation of B. The `IfExp` chain compiles to a single section:

$$(\top,\ \mathsf{If}(\mathsf{ival}(p_0) < 0,\ \mathsf{IntObj}(-1),\ \mathsf{If}(\mathsf{ival}(p_0) = 0,\ \mathsf{IntObj}(0),\ \mathsf{IntObj}(1))))$$

Path. The Čech check compares each of A's sections against B's single section. For each overlap (where A's guard is satisfiable within B's guard $\top$), Z3 checks:

$$\mathsf{guard}_i \wedge \mathsf{term}_i \neq \mathsf{If}(\ldots)$$

For the first section ($\mathsf{ival}(p_0) < 0$): the If-expression evaluates to $\mathsf{IntObj}(-1)$ on that region, matching A's term. Z3 proves UNSAT. Similarly for sections 2 and 3.

The path D8 is the *section merging* operation: reconstructing a single conditional from disjoint sections. In cubical type theory, this is the gluing of sections on a cover — exactly the sheaf condition.

Automatic inference. The section extractor and the Čech checker handle this automatically. The extractor produces sections regardless of whether the function uses early returns or a single expression. The checker compares each pair of overlapping sections by Z3 SAT query. If all overlaps agree, the sections glue ($H^1 = 0$) and the programs are equivalent.

Chapter 21

Data Structure Dichotomies (D9–D14)

21.1 D9: Stack ↔ Recursion

Paradigm A — recursive tree traversal:

```
def f(tree):
    if tree is None: return 0
    return 1 + f(tree.left) + f(tree.right)
```

Paradigm B — explicit stack:

```
def g(tree):
    count = 0
    stack = [tree]
    while stack:
        node = stack.pop()
        if node is None: continue
        count += 1
        stack.append(node.left)
        stack.append(node.right)
    return count
```

Path construction. Both compute the tree catamorphism with $\mathsf{base} = 0$ and $\mathsf{combine}(l, r, _) = 1+l+r$. The recursive version uses the call stack; the iterative version uses an explicit Pair-chain as stack.

The compiler produces:

- A: $\mathsf{tree_rec}(\mathsf{tree_rec_f}, p_0)$ — a tree RecFunction.
- B: $\mathsf{wh_count}(0, [p_0])$ — a while-loop RecFunction over stack state.

Automatic inference. The path is inferred when:

1. The recursive version's RecFunction matches the tree catamorphism pattern (base case on None/leaf, recursive calls on children).
2. The iterative version's while-loop processes a stack by popping and pushing children — the compiler detects the append/pop pattern on the stack variable.
3. The combine function is commutative (+ here), so traversal order doesn't matter (BFS/DFS/any order).

Certificate: $\mathsf{D9}(\mathsf{tree_cata}, +, 0) \cdot \mathsf{add_comm}$.

21.2 D10: `heapq` ↔ `bisect`

Paradigm A: Kahn's toposort using `heapq.heappush`/`heappop`.

Paradigm B: Kahn's toposort using `bisect.insort`/`pop(0)`.

Path construction. Both maintain a sorted frontier of zero-indegree nodes. Both extract the minimum element at each step. The path uses the *ADT abstraction*: any implementation of the priority queue interface (insert, extract_min) that maintains the sorted invariant produces the same sequence of extracted elements.

The compiler maps both to the same shared function symbols: py_meth_heappush and py_call_bisect_insort both map to py_pq_insert; py_meth_heappop and `.pop(0)` both map to py_pq_extract_min.

Automatic inference. The compiler maintains a table of *ADT equivalence classes*: operations that implement the same abstract interface. When both programs use operations from the same equivalence class on the same data, they get the same shared symbols.

21.3 D11: `OrderedDict` ↔ LinkedList LRU

Paradigm A: LRU cache using `OrderedDict.move_to_end`.

Paradigm B: LRU cache using a doubly-linked list + dict.

Path. Both implement the LRU interface: get(k) (returns value, moves to front), put(k, v) (inserts or updates, evicts LRU if full). Both shared-symbol to py_lru_get, py_lru_put. Same interface ⇒ same behavior on the same sequence of operations (by ADT abstraction).

21.4 D12: `defaultdict` ↔ `setdefault`

Both implement "get existing value or create default":

```
# A: d = defaultdict(list); d[k].append(v)
# B: d.setdefault(k, []).append(v)
```

Both compile to: py_mut_append(py_meth_setdefault(d, k, NoneObj), v) (same shared symbols). Path: refl.

21.5 D13: `Counter` ↔ Manual Counting

`Counter(xs)` is syntactic sugar for fold($\lambda d, x.$ py_meth___setitem__(d, x, binop(ADD, py_meth_get(d, x, IntO

A manual counting loop compiles to the same fold. Path: D5 (reduce↔loop).

21.6 D14: Array ↔ Dict Table

A list used as `dp[i]` and a dict used as `dp[i]` both compile to py_meth___getitem__(dp, i) (same shared symbol). Path: refl (same interface).

Chapter 22

Algorithm Strategy Dichotomies (D15–D20)

22.1 D15: BFS $\leftrightarrow$ DFS

When the output is *traversal-order independent* (e.g., a count, sum, set, or sorted result), BFS and DFS produce the same answer.

Theorem 22.1 (D15: Traversal-Order Invariance)**.** *If the combining operation is a commutative monoid:*

$$\mathsf{D15} : \mathsf{Path}(\mathsf{fold}(\mathsf{op}, e, \mathsf{bfs_order}(G)),\ \mathsf{fold}(\mathsf{op}, e, \mathsf{dfs_order}(G)))$$

when (op, e) *is a commutative monoid.*

Z3 encoding: The commutative monoid axioms (add_comm, add_assoc, etc.) enable Z3 to prove that folds over different permutations of the same multiset produce the same result.

Benchmark coverage: 5 pairs.

22.2 D16: Memoized $\leftrightarrow$ Bottom-Up DP

Both compute the least fixpoint of the same recurrence.

Theorem 22.2 (D16: Evaluation-Order Independence)**.** *For a monotone operator F on a complete lattice:*

$$\mathrm{lfp}(F) = \bigsqcup_{n \geq 0} F^n(\bot)$$

This fixpoint is independent of the order in which $F^n(\bot)$ is computed (top-down with memoization vs. bottom-up filling).

Benchmark coverage: 1 pair directly (eq19), but implicitly present in many DP pairs.

22.3 D17: Divide-and-Conquer $\leftrightarrow$ Iterative

A divide-and-conquer algorithm splits the input, recurses, and combines: $f(\mathsf{xs}) = \mathsf{combine}(f(\mathsf{left}), f(\mathsf{right}))$. An iterative algorithm processes elements one by one: $g(\mathsf{xs}) = \mathsf{fold}(\mathsf{step}, e, \mathsf{xs})$.

When combine is associative:

$$\mathsf{combine}(f(\mathsf{left}), f(\mathsf{right})) = \mathsf{fold}(\mathsf{step}, e, \mathsf{left} \mathbin{+\!\!+} \mathsf{right})$$

by associativity of the fold.

22.4 D18: Greedy ↔ DP

When the problem has the *matroid property* (greedy choice property + optimal substructure), greedy and DP produce the same optimal solution.

This is the hardest dichotomy to verify automatically because it requires proving the matroid property, which is problem-specific. However, for the benchmark pairs that exhibit it (eq47: stock trading), the shared specification axiom suffices.

22.5 D19: Sort-Then-Scan ↔ Sweep

Sorting the input then scanning linearly is equivalent to processing events in a sweep line. Both produce the same result because:

1. Sorting produces a canonical order.
2. Both algorithms process elements in this order.
3. The processing logic is the same (just expressed differently).

Z3 encoding: The py_sorted shared function symbol with the idempotence axiom ensures that both approaches produce the same sorted order.

22.6 D20: Different Algorithms for Same Specification

The catch-all dichotomy: two genuinely different algorithms that satisfy the same specification. Examples:

- Graham scan vs. monotone chain (convex hull).
- Tarjan vs. Kosaraju (SCC).
- Matrix exponentiation vs. fast doubling (Fibonacci).
- Multiplicative vs. Pascal (binomial coefficients).

Theorem 22.3 (D20: Specification Uniqueness (HIT Truncation))**.** *If specification S is propositional (at most one valid output per input) and both programs satisfy S, they are equivalent.*

This requires encoding S as a Z3 constraint and proving uniqueness.

Chapter 23

Language Feature Dichotomies (D21–D24)

23.1 D21: `isinstance` ↔ Dispatch Table

Paradigm A — isinstance chain:

```
def f(x):
    if isinstance(x, int):    return x * 2
    elif isinstance(x, str):  return x + x
    elif isinstance(x, list): return x + x
    else:                     return None
```

Paradigm B — dispatch dict:

```
def g(x):
    dispatch = {int: lambda v: v*2, str: lambda v: v+v,
                list: lambda v: v+v}
    handler = dispatch.get(type(x))
    return handler(x) if handler else None
```

Compilation of A. Each `isinstance(x, T)` compiles to $\mathsf{BoolObj}(\mathsf{tag}(p_0) = \tau_T)$ (fiber projection). The section extractor produces one section per branch:

$$(\mathsf{tag}(p_0) = \mathsf{TInt},\ \mathsf{binop}(\mathsf{MUL}, p_0, \mathsf{IntObj}(2))) \tag{23.1}$$

$$(\mathsf{tag}(p_0) = \mathsf{TStr},\ \mathsf{binop}(\mathsf{ADD}, p_0, p_0)) \tag{23.2}$$

$$(\mathsf{tag}(p_0) = \mathsf{TPair},\ \mathsf{binop}(\mathsf{ADD}, p_0, p_0)) \tag{23.3}$$

$$(\text{else},\ \mathsf{NoneObj}) \tag{23.4}$$

Compilation of B. The dispatch dict is compiled by inlining each lambda (D2). The `type(x)` call maps to $\mathsf{tag}(p_0)$. The `dict.get` + conditional produces the same If-chain. Result: identical sections.

Path: refl after D2 inlining + fiber projection.

Automatic inference. The compiler recognizes `isinstance` as fiber projection (built into compilation rule E8). Dict-based dispatch compiles via D2 (lambda inlining) + shared symbol for `type`. Both produce the same tag-based If-chain.

23.2 D22: `try/except` ↔ Conditional

Paradigm A — exception-based:

```python
def f(x, y):
    try:
        return x // y
    except ZeroDivisionError:
        return 0
```

Paradigm B — guard-based:

```python
def g(x, y):
    if y == 0:
        return 0
    return x // y
```

Compilation of A. The try/except wraps the body in a handler context. The $\mathsf{binop}(\mathsf{FLOORDIV}, p_0, p_1)$ returns Bottom when $\mathsf{ival}(p_1) = 0$. The except handler catches Bottom and returns $\mathsf{IntObj}(0)$.

The section extractor produces: $(\top,\ \mathsf{If}(\mathsf{is_Bottom}(\mathsf{binop}(\mathsf{FLOORDIV}, p_0, p_1)),\ \mathsf{IntObj}(0),\ \mathsf{binop}(\mathsf{FLOORDIV},$

Compilation of B. The guard $y == 0$ compiles to $\mathsf{ival}(p_1) = 0$. Sections: $(\mathsf{ival}(p_1) = 0,\ \mathsf{IntObj}(0))$ and $(\mathsf{ival}(p_1) \neq 0,\ \mathsf{binop}(\mathsf{FLOORDIV}, p_0, p_1))$.

Path. On the Int×Int fiber: when $\mathsf{ival}(p_1) = 0$, FLOORDIV returns Bottom, so the If in A returns $\mathsf{IntObj}(0)$ — matching B's first section. When $\mathsf{ival}(p_1) \neq 0$, FLOORDIV succeeds, the If returns the result — matching B's second section. Z3 proves UNSAT on the disagreement query.

Automatic inference. The compiler models try/except as Bottom-catching. The $\mathsf{is_Bottom}$ check on the try body is equivalent to the guard condition that would cause the exception. Z3's built-in reasoning about the FLOORDIV RecFunction (which returns Bottom on zero divisor) closes the gap automatically.

23.3 D23: Sorted-Then-Process ↔ Direct Processing

Paradigm A — sort then scan:

```python
def f(intervals):
    sorted_iv = sorted(intervals, key=lambda iv: iv[0])
    merged = [sorted_iv[0]]
    for s, e in sorted_iv[1:]:
        if s <= merged[-1][1]:
            merged[-1] = (merged[-1][0], max(merged[-1][1], e))
        else:
            merged.append((s, e))
    return merged
```

Paradigm B — event sweep:

```python
def g(intervals):
    events = []
    for s, e in intervals:
        events.append((s, 0))    # open
        events.append((e, 1))    # close
    events.sort()
    # ... sweep logic producing same merged intervals ...
```

Path. Both programs produce the same merged intervals. The key path constructor: $\mathsf{py_sorted}$ is shared, and the sorted axiom S5 (permutation invariance) ensures that sorting the input in different ways produces compatible orderings. Both programs' outputs depend only on the *multiset* of interval endpoints, not their original order.

Automatic inference. The compiler detects py_sorted (shared symbol) in both programs. The sorted permutation-invariance axiom (S5) allows Z3 to unify the two sorted sequences. The remaining logic (scan vs. sweep) produces the same list fold over the sorted data.

23.4 D24: Lambda ↔ Named Function

Paradigm A: `f = lambda x:  x * 2; return f(n)`

Paradigm B: `def f(x):  return x * 2; return f(n)`

Compilation. Both compile identically: the lambda stores (__func__, AST) in the environment, and the call site invokes D2 (inline_call), producing binop(MUL, p_0, IntObj(2)).

Path: refl (definitional equality via η-expansion: $\lambda x.\, f(x) = f$).

Automatic inference. Both forms are stored as `__func__` AST nodes. The inlining mechanism (D2) treats them identically. No special logic needed.

Chapter 24

Deep Theory: Algorithm A $\leftrightarrow$ Algorithm B

Dichotomy D20 is the deepest and most challenging: two *genuinely different algorithms* that compute the same mathematical function. Unlike D1–D19 and D21–D24, which can be resolved by syntactic transformation or algebraic axioms, D20 requires reasoning about the *mathematical specification* that both algorithms satisfy.

This chapter develops the full CTTT theory of specification-level equivalence.

24.1 The Problem

Consider two programs:

- f: Graham scan for convex hull.
- g: Andrew's monotone chain for convex hull.

These share no structural similarity: f sorts by polar angle and uses a stack with cross-product tests; g sorts lexicographically and builds upper/lower hulls. No syntactic transformation, fold fusion, or commutativity axiom connects them.

Yet both compute the *same* mathematical function: the convex hull of a point set. The equivalence is a *mathematical theorem* ("the convex hull of a finite point set is unique"), not a syntactic identity.

24.2 Specifications as Propositional Types

In CTTT, a *specification* is a dependent type:

Definition 24.1 (Specification)**.** A specification for a function $h : A \to B$ is a type family $S : A \times B \to \mathsf{Type}$ such that:

1. **Satisfiability**: For every $x : A$, there exists $y : B$ with $S(x, y)$ inhabited.
2. **Functionality**: For every $x : A$, the type $\Sigma_{y:B} S(x, y)$ is *propositional* (at most one y satisfies $S(x, -)$).

A specification satisfying both conditions is called *deterministic*.

Theorem 24.2 (Deterministic Specification Uniqueness)**.** *If S is a deterministic specification and both f and g satisfy S (i.e., $S(x, f(x))$ and $S(x, g(x))$ are inhabited for all x), then $f \equiv g$.*

Proof. By functionality, $\Sigma_{y:B} S(x, y)$ is propositional. Both $(f(x), \mathsf{pf}_f)$ and $(g(x), \mathsf{pf}_g)$ inhabit this type. By propositionality, they are connected by a path. The first projection of this path gives $\mathsf{Path}_B(f(x), g(x))$. By funext (Theorem 2.6), $f \equiv g$. □

This is the *fundamental theorem* for D20: if we can show both algorithms satisfy the same deterministic specification, they must compute the same function.

24.3 Examples of Deterministic Specifications

24.3.1 Convex Hull

Definition 24.3 (Convex Hull Specification)**.** $S_{\mathrm{CH}}(P, H)$ holds iff:

1. $H \subseteq P$ (the hull vertices are input points).
2. H is in convex position (no point is inside the convex hull of the others).
3. Every $p \in P$ is inside or on the boundary of the convex hull of H.
4. H is listed in counterclockwise order starting from the leftmost-lowest point.

Proposition 24.4. *S_{CH} is deterministic.*

Proof. Uniqueness: the set of extreme points of a finite point set is unique. The counterclockwise ordering from the leftmost-lowest point is unique. □

Both Graham scan and monotone chain satisfy S_{CH}. By Theorem 24.2, they are equivalent.

24.3.2 Shortest Paths

Definition 24.5 (Shortest Path Specification)**.** $S_{\mathrm{SP}}(G, s, D)$ holds iff for each vertex v:

$$D[v] = \min\{w(\pi) : \pi \text{ is a path from } s \text{ to } v \text{ in } G\}$$

where $w(\pi) = \sum_{e \in \pi} w(e)$ is the path weight.

Proposition 24.6. *S_{SP} is deterministic (assuming non-negative weights and well-defined minimum).*

Both Dijkstra's algorithm and Bellman–Ford satisfy S_{SP} (for non-negative weights). By Theorem 24.2, they produce the same distance array.

24.3.3 Strongly Connected Components

Definition 24.7 (SCC Specification)**.** $S_{\mathrm{SCC}}(G, C)$ holds iff C is the unique partition of $V(G)$ into maximal strongly connected subsets, listed in reverse topological order of the DAG of SCCs.

Tarjan's algorithm and Kosaraju's algorithm both satisfy S_{SCC}, with the SCCs in the same canonical order.

24.3.4 Edit Distance

Definition 24.8 (Edit Distance Specification)**.** $S_{\mathrm{ED}}(s_1, s_2, d)$ holds iff d equals the minimum number of single-character insertions, deletions, and substitutions to transform s_1 into s_2.

This is deterministic (the minimum exists and is unique). Wagner–Fischer and Hirschberg both satisfy it.

24.3.5 Binomial Coefficients

Definition 24.9 (Binomial Specification)**.** $S_{\binom{n}{k}}(n, k, v)$ holds iff $v = \frac{n!}{k!(n-k)!}$.

The multiplicative formula and Pascal's triangle both compute this. Uniqueness follows from the uniqueness of $n!$.

24.4 The Specification as a Higher Inductive Type

Each specification S defines a HIT T_S:

Definition 24.10 (Specification HIT)**.** Given a deterministic specification $S : A \times B \to \mathsf{Type}$, the *specification HIT* T_S has:

- **Point constructor**: for each program f satisfying S, a point $\iota(f) : T_S$.
- **Path constructor**: for any two programs f, g satisfying S:

$$\mathsf{det}_S : \mathsf{Path}_{T_S}(\iota(f), \iota(g))$$

- **Truncation**: T_S is 0-truncated (propositional).

The path constructor det_S is the computational content of Theorem 24.2: any two implementations of a deterministic specification are connected.

24.5 Z3 Encoding of Specifications

A specification $S(x, y)$ is encoded as a Z3 formula over the shared PyObj theory:

Listing 24.1: Specification encoding in Z3

```
# Specification: "y is the sorted permutation of x"
def sorted_spec(x, y, solver, T):
    S = T.S
    sf = T.shared_fn('sorted', 1)
    # y == sorted(x)
    solver.add(y == sf(x))
    # sorted(sorted(x)) == sorted(x) (idempotence)
    solver.add(ForAll([x], sf(sf(x)) == sf(x)))
```

Listing 24.2: Specification uniqueness check

```
# If both f and g satisfy sorted_spec, they are equal
x = Const('x', S)
solver.add(T.shared_fn('sorted', 1)(x) !=
           T.shared_fn('sorted', 1)(x))
# Trivially UNSAT: same shared function symbol
```

For more complex specifications (shortest paths, edit distance), the specification is encoded as a Z3 RecFunction satisfying the recurrence:

Listing 24.3: Edit distance specification

```
# d(i,j) = min(d(i-1,j)+1, d(i,j-1)+1,
#              d(i-1,j-1) + cost(i,j))
# with d(0,j)=j and d(i,0)=i
# Both DP and memoized versions satisfy this.
```

24.6 The Yoneda Perspective

The Yoneda lemma provides a deep perspective on specification-level equivalence.

Theorem 24.11 (Yoneda Lemma)**.** *For a presheaf $F : \mathcal{C}^{\mathrm{op}} \to \mathbf{Set}$ and an object $A \in \mathcal{C}$:*

$$\mathrm{Nat}(\mathrm{Hom}(A, -), F) \cong F(A)$$

Applied to programs: a program $f : A \to B$ is uniquely determined by its behavior under all "observations" $t : B \to C$:

$$f \equiv g \iff \forall t : B \to C.\, t \circ f \equiv t \circ g$$

This is the *observational equivalence* formulation of program equivalence. Two programs are equivalent iff no observation can distinguish them.

In Z3, this is implemented by checking that for each shared function symbol t (an "observation"), $t([\![f]\!](p)) = t([\![g]\!](p))$. The shared symbols serve as a finite approximation of "all observations."

24.7 Constructive Witness for D20

For each D20 pair, the equivalence proof has the structure:

1. **Identify the specification** S that both programs satisfy.
2. **Prove S is deterministic**: show that $\Sigma_y S(x, y)$ is propositional for all x.
3. **Construct the path**: apply Theorem 24.2 to obtain $f \equiv g$.

Step 1 is done by the Z3 semantic algebra: the shared function symbols and axioms encode the specification implicitly. When both programs use the same shared functions with the same axioms, they inhabit the same specification.

Step 2 is done by the Z3 axioms: the idempotence, commutativity, and associativity axioms ensure that specifications involving sorting, folding, etc., are deterministic.

Step 3 is done by Theorem 24.2 (which is a metatheorem of CTTT, not something Z3 needs to prove).

24.8 Limits of D20

D20 cannot resolve all algorithm-level equivalences. It fails when:

- The specification is not deterministic (e.g., "find any spanning tree" — there may be multiple valid outputs).
- The specification cannot be expressed in the Z3 theory (e.g., requires reasoning about infinite structures).
- The programs satisfy different specifications that happen to agree on all finite inputs but differ on edge cases.

We now develop the full theory for non-deterministic specifications.

24.9 Quotient Types for Non-Deterministic Specifications

A *non-deterministic* specification allows multiple valid outputs for a single input. Examples:

- "Find *any* spanning tree of the graph."
- "Return *a* topological ordering" (there may be many).
- "Find *a* shortest path" (there may be ties).
- "Return *any* element satisfying the predicate."

For such specifications, Theorem 24.2 does not apply: two programs can both satisfy the specification yet produce different (equally valid) outputs. We need a richer type-theoretic structure.

Definition 24.12 (Quotient Type)**.** Given a type B and an equivalence relation $\sim$ on B, the *quotient type* $B/\!\sim$ is the HIT with:

- Point constructor: $q : B \to B/\!\sim$ (the quotient map).
- Path constructor: for all $b_1, b_2 : B$ with $b_1 \sim b_2$:

$$\mathsf{quot} : \mathsf{Path}_{B/\sim}(q(b_1), q(b_2))$$

- Truncation: $B/\!\sim$ is a set (0-truncated).

Definition 24.13 (Non-Deterministic Specification via Quotient)**.** A non-deterministic specification $S : A \to \mathcal{P}(B)$ (mapping each input to a *set* of valid outputs) defines an equivalence relation on outputs:

$$b_1 \sim_{S,x} b_2 \iff b_1 \in S(x) \wedge b_2 \in S(x)$$

(two outputs are equivalent if they are both valid for input x).

Two programs f, g are S-equivalent iff for all x:

$$q(f(x)) = q(g(x)) \quad \text{in } B/\!\sim_{S,x}$$

That is, their outputs are in the same equivalence class — both are valid responses to the specification.

Theorem 24.14 (Non-Deterministic Specification Equivalence)**.** *Two programs $f, g : A \to B$ are S-equivalent iff for all $x : A$:*

$$f(x) \in S(x) \ \wedge \ g(x) \in S(x)$$

The cubical path witnessing this equivalence is:

$$p_x = \mathsf{quot}(f(x), g(x)) : \mathsf{Path}_{B/\sim_{S,x}}(q(f(x)), q(g(x)))$$

which exists because both $f(x)$ and $g(x)$ are in $S(x)$, so $f(x) \sim_{S,x} g(x)$.

Proof. By Definition 24.12, if $f(x) \sim_{S,x} g(x)$ (both are valid), then quot provides a path in $B/\!\sim_{S,x}$. By function extensionality, the family $\lambda x.\, p_x$ gives:

$$p : \mathsf{Path}_{A \to B/\sim_S}(q \circ f,\ q \circ g)$$

□

24.9.1 Z3 Encoding of Quotient Types

The quotient is encoded in Z3 by replacing the inequality query. Instead of asking "$\exists x.\, f(x) \neq g(x)$", we ask:

$$\exists x.\ f(x) \notin S(x)\ \vee\ g(x) \notin S(x)$$

If UNSAT: both programs satisfy S on all inputs, so they are S-equivalent (even if $f(x) \neq g(x)$ for some x).

If SAT: at least one program violates S — the counterexample identifies which program fails and on what input.

Listing 24.4: Non-deterministic spec check in Z3

```
def check_nondet_equiv(f_term, g_term, spec_pred, params, solver):
    """Check S-equivalence: both f and g satisfy spec S.

    spec_pred(x, y) is a Z3 BoolRef that is True iff y in S(x).
    """
    x = params[0]  # symbolic input

    # Check: does f violate S?
    solver.push()
    solver.add(Not(spec_pred(x, f_term)))
    f_violates = solver.check()
    solver.pop()

    # Check: does g violate S?
    solver.push()
    solver.add(Not(spec_pred(x, g_term)))
    g_violates = solver.check()
    solver.pop()

    if f_violates == unsat and g_violates == unsat:
        return EQUIVALENT  # both satisfy S everywhere
    elif f_violates == sat:
        return BUG_IN_F, solver.model()
    elif g_violates == sat:
        return BUG_IN_G, solver.model()
    else:
        return UNKNOWN
```

24.9.2 Example: Spanning Tree

Listing 24.5: Two spanning tree algorithms

```
def kruskal(graph):
    """Kruskal's MST (sorted by weight, union-find)."""
    edges = sorted(graph.edges, key=lambda e: e.weight)
    # ... union-find MST construction ...
    return tree

def prim(graph):
    """Prim's MST (priority queue from start vertex)."""
    # ... heap-based MST construction ...
    return tree
```

The specification $S_{\text{MST}}(G, T)$ is:

1. T is a spanning tree of G (connected, acyclic, spans all vertices).
2. T has minimum total weight among all spanning trees.

When edge weights are distinct, the MST is unique and S is deterministic. When weights have ties, multiple valid MSTs exist.

In the non-deterministic case:

- $\mathsf{kruskal}(G)$ and $\mathsf{prim}(G)$ may produce *different* MSTs (due to tie-breaking).
- But both satisfy S_{MST}.
- In the quotient $B/{\sim_{S_{\text{MST}}}}$, both outputs are in the same equivalence class.
- The Z3 check verifies that both satisfy S (UNSAT on both negations), concluding S-equivalence.

24.9.3 Example: Topological Sort

The specification "return a topological ordering of the DAG" is non-deterministic: a DAG with n vertices may have exponentially many valid orderings.

Two topological sort algorithms (Kahn's BFS vs. DFS-based) both satisfy the specification:

$$S_{\text{topo}}(G, \sigma) \iff \forall (u, v) \in E(G).\ \sigma^{-1}(u) < \sigma^{-1}(v)$$

(every edge goes from earlier to later in the ordering).

They may produce different orderings, but both are valid. In the quotient, they are equal.

If the specification is strengthened to "return the *lexicographically smallest* topological ordering", it becomes deterministic, and Theorem 24.2 applies directly.

24.10 Propositional Extensionality

An alternative to quotient types is *propositional extensionality*:

Axiom 24.15 (Propositional Extensionality)**.** *For propositions* $P, Q : \mathsf{Prop}$*:*

$$(P \leftrightarrow Q) \to (P = Q)$$

Two propositions with the same truth value are equal.

In cubical type theory, propositional extensionality is a consequence of univalence (Axiom 3.2).

Application to specifications: if the specification $S(x, y)$ is a proposition (either inhabited or empty, with no computational content), then propositional extensionality says:

$$S(x, f(x)) \leftrightarrow S(x, g(x)) \implies S(x, f(x)) = S(x, g(x))$$

So if both $f(x)$ and $g(x)$ satisfy S (both sides are $\top$), they are *equal as propositions*. This doesn't make $f(x)$ and $g(x)$ equal as *values*, but it makes their *validity status* equal.

Combined with the quotient construction: if validity is all we care about (not the specific output), propositional extensionality collapses the quotient to a single point.

24.11 Set-Level Truncation

The *set-level truncation* $\|B\|_0$ of a type B is the quotient that identifies all elements:

Definition 24.16 (Set Truncation)**.** $\|B\|_0$ is the HIT with:

- $|b| : \|B\|_0$ for each $b : B$.
- $\mathsf{trunc} : \prod_{x,y:\|B\|_0} \mathsf{Path}(x, y)$ (all elements are equal).

Applied to specifications: $\|S(x)\|_0$ is either empty (no valid output) or a single point (at least one valid output exists, and all valid outputs are identified).

Two programs f, g satisfying S produce elements $|f(x)|, |g(x)| \in \|S(x)\|_0$. By truncation, $|f(x)| = |g(x)|$.

Theorem 24.17 (Equivalence Modulo Non-Deterministic Spec)**.** *If S is a (possibly non-deterministic) specification and both f and g satisfy S for all inputs, then:*

$$q \circ f \equiv q \circ g : A \to \|B\|_0/{\sim_S}$$

where q is the quotient-then-truncate map.

Proof. For each x, both $f(x)$ and $g(x)$ are in $S(x)$. The quotient $B/{\sim_{S,x}}$ identifies them (they are in the same equivalence class). The truncation $\|B/{\sim_{S,x}}\|_0$ ensures this is a mere proposition. By propositional extensionality, $|q(f(x))| = |q(g(x))|$. By funext, $q \circ f = q \circ g$. □

24.12 Z3 Encoding of Truncated Specifications

In practice, the Z3 encoding of truncated specifications is simpler than the type theory suggests:

Listing 24.6: Truncated spec check

```
def check_truncated_equiv(f_term, g_term, spec_pred,
                          params, solver):
    """Check: both satisfy spec S.

    We do NOT check f(x) == g(x).
    We check f(x) in S(x) AND g(x) in S(x).
    If both hold, they are equivalent modulo S.
    """
    x = params[0]

    # Combined query: exists x where f or g violates S
    solver.push()
    solver.add(Or(
        Not(spec_pred(x, f_term)),
        Not(spec_pred(x, g_term))
    ))
    result = solver.check()
    solver.pop()

    if result == unsat:
        return S_EQUIVALENT  # both satisfy S everywhere
    else:
        m = solver.model()
        # Which one fails?
        # ... extract counterexample ...
        return S_INEQUIVALENT, m
```

24.12.1 Fiber-Wise Truncated Spec Checking

The cohomological decomposition applies to truncated specs:

1. For each type fiber U_τ: check that both f and g satisfy S when restricted to fiber τ.
2. Build the C^0 cochain: entry is 1 iff both satisfy S on that fiber.
3. Compute H^1: measures whether the local S-satisfactions are globally coherent.
4. $H^1 = 0 \implies$ globally S-equivalent.

The only change from deterministic spec checking is the local query: instead of "$f(x) = g(x)$" we check "$f(x) \in S(x) \wedge g(x) \in S(x)$". The cohomological gluing machinery is unchanged.

24.13 The Eight Benchmark Pairs

The eight pairs classified as Algo↔Algo are:

ID	Alg A	Alg B	Specification
07	Graham scan	Monotone chain	Convex hull (unique ordered extreme points)
29	deepcopy + sort	BFS copy + sort	Deep copy with sorted canonical form
33	Factoradic	itertools.permutations	n-th permutation in lexicographic order
35	all-based	any-based (negated)	Universal quantification (De Morgan)
40	Multiplicative	Pascal's triangle	Binomial coefficient $\binom{n}{k}$
47	DP stock profit	Greedy + DP	Maximum profit with k transactions
neq06	Late-binding	Early-binding	NOT equivalent (mutation effect)
neq20	In-place +=	Reassignment +	NOT equivalent (aliasing effect)

Note: eq35 (all vs. any) is resolved by the De Morgan axioms (boolean paths), not by specification reasoning. The remaining five EQ pairs require genuine specification-level arguments:

- **eq07**: uniqueness of convex hull vertices and canonical ordering.
- **eq29**: uniqueness of deep copy (object identity doesn't affect sorted canonical representation).
- **eq33**: uniqueness of the n-th lexicographic permutation.
- **eq40**: uniqueness of $\binom{n}{k}$ (integer arithmetic identity).
- **eq47**: uniqueness of optimal profit (by the optimality of the DP recurrence).

The two NEQ pairs are correctly distinguished by the *effect discipline* (mutation vs. pure operations use different Z3 symbols).

Part VI

The Semantic Algebra

Chapter 25

The PyObj Z3 Theory: A Complete Semantic Model of Python

This chapter defines the Z3 theory that models Python's value space, operations, and execution semantics. The theory is designed to be *maximally expressive* (capturing as much of Python's semantics as possible) while *preserving decidability* (every previously decidable query remains decidable after adding new features).

The key design principle: **new expressiveness is added via new constructors and new shared functions, never via quantifiers**. This keeps all fiber-restricted queries in the quantifier-free fragment QF_DT + QF_LIA + QF_S, which is decidable.

25.1 The Value Universe

25.1.1 The Seven Constructors

The universe of Python values is modeled as a Z3 algebraic data type with seven constructors:

Definition 25.1 (PyObj ADT).

$$\mathsf{PyObj} ::= \mathsf{IntObj}(i : \mathbb{Z}) \mid \mathsf{BoolObj}(b : \mathbb{B}) \mid \mathsf{StrObj}(s : \mathsf{String}) \mid \mathsf{NoneObj} \mid \mathsf{Pair}(a : \mathsf{PyObj}, b : \mathsf{PyObj}) \mid \mathsf{Ref}(\mathsf{addr} : \mathbb{Z}) \mid$$

Listing 25.1: PyObj ADT: complete Z3 implementation

```python
from z3 import *

D = Datatype('PyObj')
D.declare('IntObj',  ('ival', IntSort()))
D.declare('BoolObj', ('bval', BoolSort()))
D.declare('StrObj',  ('sval', StringSort()))
D.declare('NoneObj')
D.declare('Pair', ('fst', D), ('snd', D))
D.declare('Ref',  ('addr', IntSort()))
D.declare('Bottom')
S = D.create()

# Constructor shortcuts
def mkint(n):  return S.IntObj(IntVal(n))
def mkbool(b): return S.BoolObj(BoolVal(b))
def mkstr(s):  return S.StrObj(StringVal(s))
def mklist(*elts):
    r = S.NoneObj
    for e in reversed(elts): r = S.Pair(e, r)
    return r
```

Constructor	Python type(s)	Z3 field	Exactness
IntObj	`int`	Z3 IntSort (unbounded)	Exact
BoolObj	`bool`	Z3 BoolSort	Exact
StrObj	`str`	Z3 StringSort	Exact
NoneObj	`None`	(no field)	Exact
Pair	`list`, `tuple`	cons cell	Structural
Ref	mutable objects	heap address	Abstract
Bottom	exceptions, $\perp$	(no field)	Abstract

25.1.2 Why Seven Constructors Suffice

Python has hundreds of types (`int`, `float`, `complex`, `str`, `bytes`, `list`, `tuple`, `set`, `frozenset`, `dict`, `deque`, `defaultdict`, `Counter`, `OrderedDict`, and every user-defined class). Our seven constructors cover them via *abstraction*:

Python type	PyObj constructor	Justification
`int`	IntObj	Exact: Z3 IntSort is unbounded, matching Python's arbitrary-precision i
`float`	IntObj	Approximate: we model float as int (sound for integer-valued floats)
`complex`	Pair(IntObj, IntObj)	Real and imaginary parts
`bool`	BoolObj	Exact
`str`	StrObj	Exact: Z3 StringSort models Unicode strings with concat, length, etc.
`bytes`	StrObj	Approximate
`None`	NoneObj	Exact (singleton)
`list`	Pair-chain	Structural: $[1, 2, 3]$ = Pair(1, Pair(2, Pair(3, NoneObj)))
`tuple`	Pair-chain	Same encoding as list (Python tuples and lists have same structure)
`set`	Ref	Abstract: modeled by identity (heap address), operations via shared func
`frozenset`	Ref	Abstract
`dict`	Ref	Abstract
`deque`	Ref	Abstract
`defaultdict`	Ref	Abstract
`Counter`	Ref	Abstract
`OrderedDict`	Ref	Abstract
user class instance	Ref	Abstract
function/lambda	Ref	Abstract (callable ref)
generator	Ref	Abstract (stateful iterator)
exception	Bottom	Abstract (control flow)
`NotImplemented`	Bottom	Abstract

The abstraction is *sound*: if the Z3 theory proves two programs equivalent, they are genuinely equivalent for all concrete Python values that the constructors faithfully represent. The abstraction may be *incomplete*: programs that differ only on types mapped to Ref (e.g., two different dict implementations) may not be distinguished.

25.1.3 Decidability Preservation

Proposition 25.2 (Adding Constructors Preserves Decidability)**.** *The theory QF_DT (quantifier-free datatypes) is decidable for any finite set of constructors. Adding a new constructor to the* PyObj *ADT does not break decidability — it adds new cases to pattern matching but keeps the theory in QF_DT.*

This means we can extend PyObj with new constructors (e.g., FloatObj(real : RealSort), SetObj(elems : ArraySort)) without breaking decidability of existing queries. We choose seven constructors for practical efficiency (fewer cases in RecFunctions), not for theoretical necessity.

25.2 The Type Fibration

25.2.1 TypeTag Sort

The type tag function is the fibration structure map:

Definition 25.3 (Type Fibration)**.** $\mathsf{tag} : \mathsf{PyObj} \to \mathsf{TypeTag}$ where $\mathsf{TypeTag} = \{\mathsf{TInt}, \mathsf{TBool}, \mathsf{TStr}, \mathsf{TNone}, \mathsf{TPair},$ is defined by:

$$\mathsf{tag}(v) = \begin{cases} \mathsf{TInt} & \text{if } v = \mathsf{IntObj}(\ldots) \\ \mathsf{TBool} & \text{if } v = \mathsf{BoolObj}(\ldots) \\ \mathsf{TStr} & \text{if } v = \mathsf{StrObj}(\ldots) \\ \mathsf{TNone} & \text{if } v = \mathsf{NoneObj} \\ \mathsf{TPair} & \text{if } v = \mathsf{Pair}(\ldots) \\ \mathsf{TRef} & \text{if } v = \mathsf{Ref}(\ldots) \\ \mathsf{TBot} & \text{if } v = \mathsf{Bottom} \end{cases}$$

This is implemented as a Z3 RecFunction and a Z3 EnumSort. The RecFunction dispatches on the constructor, returning the corresponding enum constant.

Listing 25.2: TypeTag: complete Z3 implementation

```
TTag, (TInt, TBool, TStr, TNone, TPair, TRef, TBot) = \
    EnumSort('TTag', ['TInt','TBool','TStr',
                      'TNone','TPair','TRef','TBot'])

tag = RecFunction('tag', S, TTag)
x = Const('_tag_x', S)
RecAddDefinition(tag, [x],
    If(S.is_IntObj(x),  TInt,
    If(S.is_BoolObj(x), TBool,
    If(S.is_StrObj(x),  TStr,
    If(S.is_NoneObj(x), TNone,
    If(S.is_Pair(x),    TPair,
    If(S.is_Ref(x),     TRef,
                        TBot)))))))
```

25.2.2 Fiber Projection Lemmas

The following lemmas are used repeatedly in the Čech complex construction. They are all decidable consequences of the ADT theory.

Lemma 25.4 (Fiber Determines Constructor)**.** $\mathsf{tag}(v) = \mathsf{TInt} \iff \exists n : \mathbb{Z}.\ v = \mathsf{IntObj}(n)$.

More generally, knowing the tag determines the constructor shape, which Z3 exploits via the $\mathsf{is_IntObj}$, $\mathsf{is_BoolObj}$, *... tester functions.*

Lemma 25.5 (Fiber Disjointness)**.** *The fibers are pairwise disjoint:* $\mathsf{tag}(v) = \tau_1 \wedge \mathsf{tag}(v) = \tau_2 \implies \tau_1 = \tau_2$. *This is automatic from the ADT theory (each value has exactly one constructor).*

Lemma 25.6 (Fiber Exhaustion)**.** *The fibers cover all values:* $\bigvee_{\tau \in \mathsf{TypeTag}} \mathsf{tag}(v) = \tau$. *This is the "no junk" property of inductive types.*

25.3 Operations: Full Python Semantics in Z3

25.3.1 Binary Operations

The binop RecFunction encodes Python's binary operations with full type dispatch:

Definition 25.7 (Binary Operation Semantics)**.** $\mathsf{binop} : \mathbb{Z} \times \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj}$ where the first argument is an operation tag.

Integer $\times$ **Integer** ($\mathsf{tag}(a) = \mathsf{tag}(b) = \mathsf{TInt}$):

$$\mathsf{binop}(\mathsf{ADD}, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) = \mathsf{IntObj}(m+n) \tag{25.1}$$
$$\mathsf{binop}(\mathsf{SUB}, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) = \mathsf{IntObj}(m-n) \tag{25.2}$$
$$\mathsf{binop}(\mathsf{MUL}, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) = \mathsf{IntObj}(m \times n) \tag{25.3}$$
$$\mathsf{binop}(\mathsf{FLOORDIV}, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) = \begin{cases} \mathsf{IntObj}(m \div n) & n \neq 0 \\ \mathsf{Bottom} & n = 0 \end{cases} \tag{25.4}$$
$$\mathsf{binop}(\mathsf{MOD}, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) = \begin{cases} \mathsf{IntObj}(m \bmod n) & n \neq 0 \\ \mathsf{Bottom} & n = 0 \end{cases} \tag{25.5}$$
$$\mathsf{binop}(\mathsf{LT}, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) = \mathsf{BoolObj}(m < n) \tag{25.6}$$
$$\mathsf{binop}(\mathsf{LE}, \ldots) = \mathsf{BoolObj}(m \leq n) \tag{25.7}$$
$$\mathsf{binop}(\mathsf{GT}, \ldots) = \mathsf{BoolObj}(m > n) \tag{25.8}$$
$$\mathsf{binop}(\mathsf{GE}, \ldots) = \mathsf{BoolObj}(m \geq n) \tag{25.9}$$
$$\mathsf{binop}(\mathsf{EQ}, \ldots) = \mathsf{BoolObj}(m = n) \tag{25.10}$$
$$\mathsf{binop}(\mathsf{NE}, \ldots) = \mathsf{BoolObj}(m \neq n) \tag{25.11}$$
$$\mathsf{binop}(\mathsf{MAX}, \ldots) = \mathsf{If}(m \geq n, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) \tag{25.12}$$
$$\mathsf{binop}(\mathsf{MIN}, \ldots) = \mathsf{If}(m \leq n, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) \tag{25.13}$$

String $\times$ **String** ($\mathsf{tag}(a) = \mathsf{tag}(b) = \mathsf{TStr}$):

$$\mathsf{binop}(\mathsf{ADD}, \mathsf{StrObj}(s), \mathsf{StrObj}(t)) = \mathsf{StrObj}(\mathsf{Concat}(s,t)) \tag{25.14}$$
$$\mathsf{binop}(\mathsf{EQ}, \mathsf{StrObj}(s), \mathsf{StrObj}(t)) = \mathsf{BoolObj}(s = t) \tag{25.15}$$
$$\mathsf{binop}(\mathsf{NE}, \ldots) = \mathsf{BoolObj}(s \neq t) \tag{25.16}$$

Cross-type equality: For any a, b with $\mathsf{tag}(a) \neq \mathsf{tag}(b)$:

$$\mathsf{binop}(\mathsf{EQ}, a, b) = \mathsf{BoolObj}(a = b) \qquad \text{(structural equality)} \tag{25.17}$$
$$\mathsf{binop}(\mathsf{NE}, a, b) = \mathsf{BoolObj}(a \neq b) \tag{25.18}$$

All other cross-type combinations: Bottom (TypeError in Python).

Proposition 25.8 (binop Decidability)**.** *On the* TInt *fiber,* binop *reduces to Z3's native* IntSort *operations (decidable QF_LIA for linear operations, semi-decidable QF_NIA for multiplication). On the* TStr *fiber, it reduces to Z3's string theory (decidable QF_S for concat and equality). On other fibers, it returns* Bottom *(decidable constant).*

Therefore: binop *does not break decidability on any fiber.*

Listing 25.3: binop: complete Z3 implementation

```
ADD=1; SUB=2; MUL=3; FLOORDIV=5; MOD=6
LT=20; LE=21; GT=22; GE=23; EQ=24; NE=25
MAX=30; MIN=31

```

```
binop = RecFunction('binop', IntSort(), S, S, S)
op = Int('_bo')
a, b = Const('_ba', S), Const('_bb', S)
ai, bi = S.ival(a), S.ival(b)
ii = And(S.is_IntObj(a), S.is_IntObj(b))
ss = And(S.is_StrObj(a), S.is_StrObj(b))

int_result = \
  If(op==ADD, S.IntObj(ai+bi),
  If(op==SUB, S.IntObj(ai-bi),
  If(op==MUL, S.IntObj(ai*bi),
  If(op==FLOORDIV, If(bi!=0, S.IntObj(ai/bi), S.Bottom),
  If(op==MOD, If(bi!=0, S.IntObj(ai%bi), S.Bottom),
  If(op==LT, S.BoolObj(ai<bi),
  If(op==LE, S.BoolObj(ai<=bi),
  If(op==GT, S.BoolObj(ai>bi),
  If(op==GE, S.BoolObj(ai>=bi),
  If(op==EQ, S.BoolObj(ai==bi),
  If(op==NE, S.BoolObj(ai!=bi),
  If(op==MAX, If(ai>=bi, a, b),
  If(op==MIN, If(ai<=bi, a, b),
  S.Bottom))))))))))))))

str_result = \
  If(op==ADD, S.StrObj(Concat(S.sval(a), S.sval(b))),
  If(op==EQ, S.BoolObj(S.sval(a)==S.sval(b)),
  If(op==NE, S.BoolObj(S.sval(a)!=S.sval(b)),
  S.Bottom)))

RecAddDefinition(binop, [op, a, b],
    If(ii, int_result,
    If(ss, str_result,
    If(op==EQ, S.BoolObj(a==b),
    If(op==NE, S.BoolObj(a!=b),
    S.Bottom)))))
```

25.3.2 Unary Operations

Definition 25.9 (Unary Operation Semantics). $\mathsf{unop} : \mathbb{Z} \times \mathsf{PyObj} \to \mathsf{PyObj}$

On IntObj:

$$\mathsf{unop}(\mathsf{NEG}, \mathsf{IntObj}(n)) = \mathsf{IntObj}(-n) \tag{25.19}$$

$$\mathsf{unop}(\mathsf{ABS}, \mathsf{IntObj}(n)) = \mathsf{IntObj}(\mathsf{If}(n \geq 0, n, -n)) \tag{25.20}$$

$$\mathsf{unop}(\mathsf{NOT}, \mathsf{IntObj}(n)) = \mathsf{BoolObj}(n = 0) \tag{25.21}$$

$$\mathsf{unop}(\mathsf{BOOL}, \mathsf{IntObj}(n)) = \mathsf{BoolObj}(n \neq 0) \tag{25.22}$$

$$\mathsf{unop}(\mathsf{INT}, \mathsf{IntObj}(n)) = \mathsf{IntObj}(n) \tag{25.23}$$

On BoolObj:

$$\mathsf{unop}(\mathsf{NOT}, \mathsf{BoolObj}(b)) = \mathsf{BoolObj}(\neg b) \tag{25.24}$$

$$\mathsf{unop}(\mathsf{BOOL}, \mathsf{BoolObj}(b)) = \mathsf{BoolObj}(b) \tag{25.25}$$

$$\mathsf{unop}(\mathsf{INT}, \mathsf{BoolObj}(b)) = \mathsf{IntObj}(\mathsf{If}(b, 1, 0)) \tag{25.26}$$

On StrObj:

$$\mathsf{unop}(\mathsf{LEN}, \mathsf{StrObj}(s)) = \mathsf{IntObj}(\mathsf{Length}(s)) \tag{25.27}$$
$$\mathsf{unop}(\mathsf{BOOL}, \mathsf{StrObj}(s)) = \mathsf{BoolObj}(\mathsf{Length}(s) > 0) \tag{25.28}$$

On NoneObj: $\mathsf{unop}(\mathsf{BOOL}, \mathsf{NoneObj}) = \mathsf{BoolObj}(\bot)$; $\mathsf{unop}(\mathsf{NOT}, \mathsf{NoneObj}) = \mathsf{BoolObj}(\top)$.
On Pair: $\mathsf{unop}(\mathsf{BOOL}, \mathsf{Pair}(a, b)) = \mathsf{BoolObj}(\top)$ (non-empty list is truthy).

25.3.3 The Truthiness Protocol

Python's $__\mathsf{bool}__$ protocol is central to control flow. Every value has a truth value:

Definition 25.10 (Truthiness). $\mathsf{truthy} : \mathsf{PyObj} \to \mathbb{B}$

$$\mathsf{truthy}(\mathsf{IntObj}(n)) = (n \neq 0) \tag{25.29}$$
$$\mathsf{truthy}(\mathsf{BoolObj}(b)) = b \tag{25.30}$$
$$\mathsf{truthy}(\mathsf{StrObj}(s)) = (\mathsf{Length}(s) > 0) \tag{25.31}$$
$$\mathsf{truthy}(\mathsf{NoneObj}) = \bot \tag{25.32}$$
$$\mathsf{truthy}(\mathsf{Pair}(a, b)) = \top \tag{25.33}$$
$$\mathsf{truthy}(\mathsf{Bottom}) = \bot \tag{25.34}$$
$$\mathsf{truthy}(\mathsf{Ref}(_)) = \top \tag{25.35}$$

Remark 25.11 (Faithfulness of Truthiness). This exactly matches CPython's `__bool__` for all modeled types. The only approximation is Ref: we model all reference objects as truthy, which is correct for non-empty containers but wrong for empty containers stored as Refs. This is sound for the Čech analysis because the Ref fiber is handled abstractly (via shared function symbols, not by evaluating truthiness).

Listing 25.4: truthy: complete Z3 implementation

```
truthy = RecFunction('truthy', S, BoolSort())
x = Const('_tr', S)
RecAddDefinition(truthy, [x],
    If(S.is_IntObj(x),  S.ival(x) != 0,
    If(S.is_BoolObj(x), S.bval(x),
    If(S.is_StrObj(x),  Length(S.sval(x)) > 0,
    If(S.is_NoneObj(x), False,
    If(S.is_Bottom(x),  False,
    True))))))  # Pair, Ref are truthy
```

25.3.4 The Canonical Fold (Nat-Recursor)

Definition 25.12 (Canonical Fold). $\mathsf{fold} : \mathbb{Z} \times \mathbb{Z} \times \mathbb{Z} \times \mathsf{PyObj} \to \mathsf{PyObj}$

$$\mathsf{fold}(\mathsf{op}, i, \mathsf{stop}, \mathsf{acc}) = \begin{cases} \mathsf{acc} & \text{if } i \geq \mathsf{stop} \\ \mathsf{binop}(\mathsf{op}, \mathsf{IntObj}(i), \mathsf{fold}(\mathsf{op}, i+1, \mathsf{stop}, \mathsf{acc})) & \text{otherwise} \end{cases} \tag{25.36}$$

Listing 25.5: fold: complete Z3 implementation

```
fold = RecFunction('fold',
    IntSort(), IntSort(), IntSort(), S, S)
op, i, stop = Ints('_fo _fi _fs')
acc = Const('_fa', S)
RecAddDefinition(fold, [op, i, stop, acc],
```

```
    If(i >= stop, acc,
       binop(op, S.IntObj(i),
             fold(op, i+1, stop, acc))))
```

The fold is the *universal* iteration primitive. Both recursive and iterative programs that operate by applying a binary operation across a range compile to this single RecFunction, enabling equivalence by syntactic identity (D1 path).

Proposition 25.13 (Fold Decidability)**.** *The fold does not break decidability of previously decidable queries. When both programs compile to the* same *fold term, the comparison is syntactic equality (decidable, $O(1)$). When they compile to* different *fold terms, Z3 handles the comparison via its RecFunction reasoning (semi-decidable in general, but decidable for ground arguments).*

25.3.5 Item Access (GETITEM)

Definition 25.14 (Subscript Semantics)**.** $\mathsf{binop}(\mathsf{GETITEM}, \mathsf{collection}, \mathsf{index})$ implements Python's $\mathsf{__getitem__}$:

On Pair-chains (list indexing):

$$\mathsf{binop}(\mathsf{GETITEM}, \mathsf{Pair}(a, d), \mathsf{IntObj}(0)) = a \tag{25.37}$$

$$\mathsf{binop}(\mathsf{GETITEM}, \mathsf{Pair}(a, d), \mathsf{IntObj}(n)) = \mathsf{binop}(\mathsf{GETITEM}, d, \mathsf{IntObj}(n-1)) \quad n > 0 \tag{25.38}$$

On StrObj (character indexing):

$$\mathsf{binop}(\mathsf{GETITEM}, \mathsf{StrObj}(s), \mathsf{IntObj}(n)) = \mathsf{StrObj}(\mathsf{SubString}(s, n, 1))$$

On Ref (dict/set access): uses shared function symbol $\mathsf{py_meth___getitem__}$.

25.4 Container Semantics via Shared Functions

For types modeled by Ref (dicts, sets, deques, etc.), operations are modeled via shared function symbols with *algebraic axioms*. This is more expressive than the ADT constructors (we capture behavioral properties without modeling internal structure) and preserves decidability (shared functions are uninterpreted; axioms are optional path constructors).

25.4.1 Dictionary Operations

Definition 25.15 (Dict Semantic Functions)**.**

$$\mathsf{py_call_dict} : \mathsf{PyObj} \to \mathsf{PyObj} \qquad \text{construct from pairs} \tag{25.39}$$

$$\mathsf{py_meth___getitem__} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \qquad \text{key lookup} \tag{25.40}$$

$$\mathsf{py_meth___setitem__} : \mathsf{PyObj} \times \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \qquad \text{key assignment (returns new dict)} \tag{25.41}$$

$$\mathsf{py_meth_get} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \qquad \text{get with default} \tag{25.42}$$

$$\mathsf{py_meth_keys} : \mathsf{PyObj} \to \mathsf{PyObj} \qquad \text{key set} \tag{25.43}$$

$$\mathsf{py_meth_values} : \mathsf{PyObj} \to \mathsf{PyObj} \qquad \text{value collection} \tag{25.44}$$

$$\mathsf{py_meth_items} : \mathsf{PyObj} \to \mathsf{PyObj} \qquad \text{key-value pairs} \tag{25.45}$$

$$\mathsf{py_meth_update} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \qquad \text{merge dicts} \tag{25.46}$$

$$\mathsf{py_meth_pop} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \qquad \text{remove key} \tag{25.47}$$

These are *uninterpreted* by default (Z3 knows nothing about them except that they are functions). Axioms can be added as path constructors:

$$\forall d, k, v.\ \mathsf{py_meth___getitem__}(\mathsf{py_meth___setitem__}(d, k, v), k) = v \tag{25.48}$$

(set-then

$$\forall d, k_1, k_2, v.\ k_1 \neq k_2 \implies \mathsf{py_meth___getitem__}(\mathsf{py_meth___setitem__}(d, k_1, v), k_2) = \mathsf{py_meth___getitem__}(d, k_2) \tag{25.49}$$

(set-then

Proposition 25.16 (Dict Axioms Preserve Decidability)**.** *These axioms are universally quantified over* PyObj*, so they technically exit the QF fragment. However:*

1. *On a specific fiber, the quantifiers are bounded (the variables have fixed tags), so E-matching terminates.*
2. *The axioms are* conditional *($k_1 \neq k_2$ guard), which limits instantiation.*
3. *They are added only when both programs actually use dict operations (axiom relevance filtering).*

In practice, these axioms do not cause Z3 timeouts on the benchmark.

25.4.2 List Operations

Definition 25.17 (List Semantic Functions)**.**

$$\mathsf{py_mut_append} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{append element (mutation)} \tag{25.50}$$
$$\mathsf{py_mut_extend} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{extend with another list} \tag{25.51}$$
$$\mathsf{py_mut_insert} : \mathsf{PyObj} \times \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{insert at index} \tag{25.52}$$
$$\mathsf{py_mut_pop} : \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{remove last element} \tag{25.53}$$
$$\mathsf{py_mut_sort} : \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{in-place sort (mutation!)} \tag{25.54}$$
$$\mathsf{py_mut_reverse} : \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{in-place reverse (mutation!)} \tag{25.55}$$

Note the distinction between *mutation* operations (prefixed mut_, which modify a Ref in place) and *pure* operations (like py_sorted, which return a new value). This distinction is critical for the effect discipline (Chapter **??**): mutation operations use different shared symbols than their pure counterparts, so Z3 correctly distinguishes lst.sort() (mutation, returns None) from sorted(lst) (pure, returns new list).

25.4.3 Set Operations

Definition 25.18 (Set Semantic Functions)**.**

$$\mathsf{py_mut_add} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{add element} \tag{25.56}$$
$$\mathsf{py_mut_discard} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{remove element} \tag{25.57}$$
$$\mathsf{py_call_union} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{set union} \tag{25.58}$$
$$\mathsf{py_call_intersection} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{set intersection} \tag{25.59}$$
$$\mathsf{py_call_difference} : \mathsf{PyObj} \times \mathsf{PyObj} \to \mathsf{PyObj} \quad \text{set difference} \tag{25.60}$$

Set axioms (optional path constructors):

$$\mathsf{py_call_union}(a, b) = \mathsf{py_call_union}(b, a) \quad \text{(union commutativity)} \tag{25.61}$$
$$\mathsf{py_call_intersection}(a, b) = \mathsf{py_call_intersection}(b, a) \quad \text{(intersection commutativity)} \tag{25.62}$$
$$\mathsf{py_call_union}(a, a) = a \quad \text{(union idempotence)} \tag{25.63}$$

25.5 The Heap Model

Mutable Python objects live on a *heap* — a global store that maps addresses to values. Two references to the same address are *aliases*: modifying one is visible through the other.

25.5.1 Heap Representation

Definition 25.19 (Abstract Heap)**.** The heap is modeled *implicitly* through the Ref constructor and mutation operations:

- $\mathsf{Ref}(\mathsf{addr})$ represents a heap-allocated object at address addr.
- Fresh allocations use fresh addresses: $\mathsf{mkref}() = \mathsf{Ref}(\mathsf{next_uid})$ where $\mathsf{next_uid}$ is a monotonically increasing counter.
- Mutation operations ($\mathsf{mut_append}$, $\mathsf{mut_sort}$, etc.) return a *new version* of the object: $\mathsf{mut_op}(\mathsf{Ref}(a), \ldots) = v'$ where v' represents the modified object.

Remark 25.20 (SSA for the Heap)**.** This is essentially *Static Single Assignment* (SSA) form for heap operations: each mutation produces a new value, and the environment tracks the latest version. Aliasing is modeled by having multiple environment entries point to the same Ref.

25.5.2 Aliasing and the Effect Discipline

Definition 25.21 (Aliasing Semantics)**.** Two variables x and y are *aliases* if $\mathsf{env}[x] = \mathsf{env}[y] = \mathsf{Ref}(a)$ for the same address a. A mutation on x modifies the Ref, so y (which points to the same Ref) sees the modification.

This is where the effect discipline matters:

- $\mathsf{mut_iadd}(\mathsf{Ref}(a), v)$ *modifies* the Ref — all aliases see the change.
- $\mathsf{py_call___add__}(\mathsf{Ref}(a), v)$ *creates a new value* — aliases are unaffected.

These are different shared symbols, so Z3 treats them as unrelated functions. This is how the framework correctly distinguishes `lst += [1]` (mutates alias) from `lst = lst + [1]` (rebinds variable).

25.6 Python Control Flow in Z3

25.6.1 Conditional Expressions

Python's `if`/`else` compiles to Z3's If:

$$[\![\texttt{t if c else f}]\!] = \mathsf{If}(\mathsf{truthy}([\![c]\!]), [\![t]\!], [\![f]\!])$$

This is a Z3 builtin (part of the core theory, always decidable).

25.6.2 Short-Circuit Boolean Operators

Python's `and`/`or` are short-circuit and return *values*, not booleans:

$$[\![\texttt{a and b}]\!] = \mathsf{If}(\mathsf{truthy}([\![a]\!]), [\![b]\!], [\![a]\!]) \tag{25.64}$$

$$[\![\texttt{a or b}]\!] = \mathsf{If}(\mathsf{truthy}([\![a]\!]), [\![a]\!], [\![b]\!]) \tag{25.65}$$

This faithfully models Python's semantics: `0 or "hello"` returns `"hello"` (not `True`), and `[] and 42` returns `[]` (not `False`).

25.6.3 Exception Semantics via Bottom

Exceptions are modeled by Bottom:

- Division by zero: $\mathsf{binop}(\mathsf{FLOORDIV}, x, \mathsf{IntObj}(0)) = \mathsf{Bottom}$.
- Type errors: $\mathsf{binop}(\mathsf{ADD}, \mathsf{IntObj}(n), \mathsf{StrObj}(s)) = \mathsf{Bottom}$.
- Explicit raise: compiled to Bottom.

Bottom propagates strictly: $\mathsf{binop}(\mathsf{op}, \mathsf{Bottom}, x) = \mathsf{Bottom}$ and $\mathsf{binop}(\mathsf{op}, x, \mathsf{Bottom}) = \mathsf{Bottom}$.

`try/except` blocks are modeled by wrapping the body in a shared "handler context" function:

$$[\![\texttt{try: body except E: handler}]\!] = \mathsf{py_try_h}([\![\mathsf{body}]\!])$$

where h is a hash of the handler signature. The handler context is a shared function, so two programs with the same try/except structure get the same Z3 symbol.

25.6.4 isinstance as Fiber Projection

Definition 25.22 (isinstance Semantics)**.**

$$[\![\texttt{isinstance(x, T)}]\!] = \mathsf{BoolObj}(\mathsf{tag}([\![x]\!]) = \tau_T)$$

where τ_T is the tag corresponding to type T:

`int`	$\to \mathsf{TInt}$	`str`	$\to \mathsf{TStr}$	`bool`	$\to \mathsf{TBool}$
`list`	$\to \mathsf{TPair}$	`tuple`	$\to \mathsf{TPair}$	`dict`	$\to \mathsf{TRef}$
`set`	$\to \mathsf{TRef}$	`float`	$\to \mathsf{TInt}$	`type`	$\to \mathsf{TRef}$

For tuple arguments (`isinstance(x, (int, str))`):

$$[\![\texttt{isinstance(x, (int, str))}]\!] = \mathsf{BoolObj}(\mathsf{tag}([\![x]\!]) = \mathsf{TInt} \vee \mathsf{tag}([\![x]\!]) = \mathsf{TStr})$$

This is the *fiber projection*: `isinstance` asks which fiber of the type fibration the value lives in. It is purely decidable (tag comparison in EnumSort).

25.6.5 Comprehensions

List, set, and generator comprehensions compile to shared function symbols keyed by a canonical hash of the element expression:

Definition 25.23 (Comprehension Semantics)**.**

$$[\![[\mathsf{elt}(x)\ \texttt{for}\ x\ \texttt{in}\ \mathsf{xs}]]\!] = \mathsf{py_comp}_{h(\mathsf{elt})}([\![\mathsf{xs}]\!])$$

where $h(\mathsf{elt}) = \mathsf{hash}(\mathsf{ast.dump}(\mathsf{elt})) \bmod 2^{31}$.

Two comprehensions with the same element expression AST get the same shared function symbol, regardless of whether they appear as a comprehension or an explicit loop with append (D4 path).

25.6.6 Attribute Access

Definition 25.24 (Attribute Semantics)**.** $[\![x.\mathsf{attr}]\!] = \mathsf{py_attr_attr}([\![x]\!])$

This is a shared uninterpreted function. No axioms are added by default (we don't model Python's attribute lookup protocol in full). But when both programs access the same attribute on the same object, they get the same Z3 term (by sharing).

25.7 Shared Function Symbols (Univalence)

25.7.1 The Registry

The *shared function registry* ensures both programs use the same Z3 function for the same Python operation.

Definition 25.25 (Shared Function Registry)**.** $\mathcal{R} : (\mathsf{name} \times \mathsf{arity}) \to \mathsf{Z3FuncDecl}$ maps operation names to canonical Z3 function declarations. The function $\mathsf{shared_fn}(\mathsf{name}, \mathsf{arity})$ returns the canonical Z3 function, creating it on first access.

25.7.2 Builtin Classification

Category	Functions	Shared symbol
Sorting	`sorted`, `reversed`	py_sorted, py_reversed
Collection	`list`, `tuple`, `set`, `frozenset`, `dict`	py_list, etc.
Aggregation	`sum`, `any`, `all`, `min`, `max`	py_sum, etc.
Higher-order	`map`, `filter`, `zip`, `enumerate`	py_map, etc.
Type conversion	`int`, `str`, `bool`, `float`	via unop
Type checking	`isinstance`, `type`	via tag
String methods	`.upper`, `.lower`, `.strip`, `.split`, `.join`, `.replace`, `.find`	py_meth_upper, etc.
Dict methods	`.get`, `.keys`, `.values`, `.items`, `.update`, `.pop`	py_meth_get, etc.
List methods	`.append`, `.extend`, `.sort`, `.reverse`, `.pop`, `.insert`	py_mut_append, etc.
Mutation ops	`+=`, `*=`, etc. on containers	py_mut_iadd, etc.

25.7.3 Why Sharing Preserves Decidability

Proposition 25.26 (Shared Symbols Do Not Affect Decidability)**.** *Replacing unique function symbols (*sorted_1, sorted_5*) with shared symbols (*$\mathsf{py_sorted}$*) does not change the decidability status of any query.*

In the QF fragment, all function symbols are uninterpreted — Z3 treats them as "black boxes" regardless of name. Using the same name for two occurrences constrains *Z3 (both must have the same interpretation), which can only make queries* easier *to decide (fewer models to search), never harder.*

Listing 25.6: Shared function registry: complete implementation

```
class Theory:
    def __init__(self):
        # ... PyObj ADT, TypeTag, RecFunctions ...
        self._shared_fns = {}  # (name, arity) -> Z3 Function

    def shared_fn(self, name, arity):
        """Return canonical Z3 function for a Python operation.

        Both programs use the SAME Z3 function for the same op.
        This is the univalence principle in action.
        """
        key = (name, arity)
        if key not in self._shared_fns:
            sorts = [self.S] * arity + [self.S]
            self._shared_fns[key] = Function(
                f'py_{name}', *sorts)
        return self._shared_fns[key]

    def semantic_axioms(self, solver):
        """Add equational axioms (path constructors)."""
```

```
        x = Const('_ax', self.S)
        # sorted: idempotent
        if ('sorted', 1) in self._shared_fns:
            sf = self._shared_fns[('sorted', 1)]
            solver.add(ForAll([x], sf(sf(x)) == sf(x)))
        # reversed: involution
        if ('reversed', 1) in self._shared_fns:
            rf = self._shared_fns[('reversed', 1)]
            solver.add(ForAll([x], rf(rf(x)) == x))
        # len: preserved by sorted, reversed
        if ('len', 1) in self._shared_fns:
            lf = self._shared_fns[('len', 1)]
            for dep in ('sorted', 'reversed', 'list'):
                if (dep, 1) in self._shared_fns:
                    df = self._shared_fns[(dep, 1)]
                    solver.add(ForAll([x],
                        lf(df(x)) == lf(x)))

# Usage in the compiler:
SHARED_BUILTINS = {
    'sorted','reversed','list','tuple','set',
    'frozenset','dict','sum','any','all',
    'enumerate','zip','map','filter',
}

def compile_call(node, env):
    T = env.T
    name = node.func.id
    args = [compile_expr(a, env) for a in node.args]
    if name in SHARED_BUILTINS and args:
        return T.shared_fn(name, len(args))(*args)
    # ... other cases ...
```

Listing 25.7: unop: complete Z3 implementation

```
NEG=50; ABS_=52; NOT=53; BOOL_=56; INT_=57; LEN_=55

unop = RecFunction('unop', IntSort(), S, S)
op = Int('_uo'); a = Const('_ua', S)
ai = S.ival(a)

RecAddDefinition(unop, [op, a],
    If(S.is_IntObj(a),
        If(op==NEG,  S.IntObj(-ai),
        If(op==ABS_, S.IntObj(If(ai>=0, ai, -ai)),
        If(op==NOT,  S.BoolObj(ai==0),
        If(op==BOOL_,S.BoolObj(ai!=0),
        If(op==INT_, a,
        S.Bottom))))),
    If(S.is_BoolObj(a),
        If(op==NOT,  S.BoolObj(Not(S.bval(a))),
        If(op==BOOL_,a,
        If(op==INT_, S.IntObj(If(S.bval(a),1,0)),
        S.Bottom))),
    If(S.is_StrObj(a),
        If(op==LEN_, S.IntObj(Length(S.sval(a))),
        If(op==BOOL_,S.BoolObj(Length(S.sval(a))>0),
        S.Bottom)),
```

```
    If(S.is_NoneObj(a),
        If(op==BOOL_,S.BoolObj(False),
        If(op==NOT,  S.BoolObj(True),
        S.Bottom)),
    If(S.is_Pair(a),
        If(op==BOOL_,S.BoolObj(True),
        S.Bottom),
    S.Bottom))))))
```

25.7.4 The Extensibility Principle

Theorem 25.27 (Safe Extension of the Shared Registry)**.** *Adding a new shared function* $\mathsf{py_new_op} : \mathsf{PyObj}^k \to \mathsf{PyObj}$ *to the registry* $\mathcal{R}$ *is* always safe*:*

1. *It does not break decidability of existing queries (the new function is uninterpreted; it adds no quantifiers).*
2. *It can only* increase *the number of provable equivalences (if both programs call the same operation, they now get the same Z3 symbol).*
3. *Adding axioms for the new function is* optional *— they increase expressiveness but may exit the QF fragment.*

This means the theory can grow incrementally: every new Python operation that appears in benchmark programs can be added as a shared function without auditing the entire theory for decidability regressions.

Chapter 26

The Semantic Algebra: Complete Axiomatization

This chapter develops the full algebraic theory underlying CTTT. Each axiom serves a dual role: it is an *equational law* of Python's operations AND a *path constructor* in the cubical HIT PyComp (Definition 4.1). The axioms are organized by algebraic structure, with proofs of soundness, explicit Z3 encodings, and analysis of which benchmark pairs each axiom enables.

26.1 The Integer Ring

The integers under $+$ and $\times$ form a commutative ring. This is the most fundamental algebraic structure for program equivalence: whenever two programs compute the same arithmetic expression via different orderings or groupings, the ring axioms provide the connecting path.

26.1.1 Ring Axioms as Path Constructors

Definition 26.1 (The Integer Ring Paths)**.** The following paths are added to PyComp:
Additive abelian group:

$$\mathsf{add_comm} : \prod_{a,b:\mathsf{PyObj}} \mathsf{Path}(\mathsf{binop}(\mathsf{ADD}, a, b),\ \mathsf{binop}(\mathsf{ADD}, b, a)) \tag{R1}$$

$$\mathsf{add_assoc} : \prod_{a,b,c} \mathsf{Path}(\mathsf{binop}(\mathsf{ADD}, \mathsf{binop}(\mathsf{ADD}, a, b), c),\ \mathsf{binop}(\mathsf{ADD}, a, \mathsf{binop}(\mathsf{ADD}, b, c))) \tag{R2}$$

$$\mathsf{add_zero_r} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{ADD}, a, \mathsf{IntObj}(0)),\ a) \tag{R3}$$

$$\mathsf{add_zero_l} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{ADD}, \mathsf{IntObj}(0), a),\ a) \tag{R4}$$

$$\mathsf{add_neg} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{ADD}, a, \mathsf{unop}(\mathsf{NEG}, a)),\ \mathsf{IntObj}(0)) \tag{R5}$$

Multiplicative commutative monoid:

$$\mathsf{mul_comm} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\mathsf{MUL}, a, b),\ \mathsf{binop}(\mathsf{MUL}, b, a)) \tag{R6}$$

$$\mathsf{mul_assoc} : \prod_{a,b,c} \mathsf{Path}(\mathsf{binop}(\mathsf{MUL}, \mathsf{binop}(\mathsf{MUL}, a, b), c),\ \mathsf{binop}(\mathsf{MUL}, a, \mathsf{binop}(\mathsf{MUL}, b, c))) \tag{R7}$$

$$\mathsf{mul_one_r} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{MUL}, a, \mathsf{IntObj}(1)),\ a) \tag{R8}$$

$$\mathsf{mul_one_l} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{MUL}, \mathsf{IntObj}(1), a),\ a) \tag{R9}$$

$$\mathsf{mul_zero} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{MUL}, a, \mathsf{IntObj}(0)),\ \mathsf{IntObj}(0)) \tag{R10}$$

Distributivity:

$$\mathsf{distrib_l} : \prod_{a,b,c} \mathsf{Path}(\mathsf{binop}(\mathsf{MUL}, a, \mathsf{binop}(\mathsf{ADD}, b, c)),\ \mathsf{binop}(\mathsf{ADD}, \mathsf{binop}(\mathsf{MUL}, a, b), \mathsf{binop}(\mathsf{MUL}, a, c))) \tag{R11}$$

Involution:

$$\mathsf{neg_neg} : \prod_{a} \mathsf{Path}(\mathsf{unop}(\mathsf{NEG}, \mathsf{unop}(\mathsf{NEG}, a)),\ a) \tag{R12}$$

$$\mathsf{abs_idem} : \prod_{a} \mathsf{Path}(\mathsf{unop}(\mathsf{ABS}, \mathsf{unop}(\mathsf{ABS}, a)),\ \mathsf{unop}(\mathsf{ABS}, a)) \tag{R13}$$

$$\mathsf{abs_neg} : \prod_{a} \mathsf{Path}(\mathsf{unop}(\mathsf{ABS}, \mathsf{unop}(\mathsf{NEG}, a)),\ \mathsf{unop}(\mathsf{ABS}, a)) \tag{R14}$$

Theorem 26.2 (Soundness of Ring Paths)**.** *Each path in Definition 26.1 is sound: it holds for all* PyObj *values on the* TInt *fiber.*

Proof. On the Int fiber, $a = \mathsf{IntObj}(m)$, $b = \mathsf{IntObj}(n)$. The binop RecFunction reduces to Z3 native integer arithmetic: $\mathsf{binop}(\mathsf{ADD}, \mathsf{IntObj}(m), \mathsf{IntObj}(n)) = \mathsf{IntObj}(m + n)$. The axioms then follow from Z3's built-in integer theory (commutativity, associativity, etc. are tautologies of LIA/NIA). □

26.1.2 Z3 Encoding

The ring axioms are NOT added as ForAll constraints (which could cause E-matching loops). Instead, they are applied as *rewrite rules* during normalization (Phase 2 of Definition 18.3):

Listing 26.1: Ring axiom application during normalization

```
def normalize_arithmetic(term, S):
    """Apply ring axioms as rewrite rules."""
    if is_binop(term, ADD):
        a, b = term.arg(1), term.arg(2)
        # R1: canonical ordering (commutativity)
        if term_order(b) < term_order(a):
            term = S.binop(ADD, b, a)
        # R3/R4: identity elimination
        if is_zero(a): return b
        if is_zero(b): return a
    if is_binop(term, MUL):
        a, b = term.arg(1), term.arg(2)
        # R6: canonical ordering
```

```
        if term_order(b) < term_order(a):
            term = S.binop(MUL, b, a)
        # R8/R9: identity elimination
        if is_one(a): return b
        if is_one(b): return a
        # R10: zero absorption
        if is_zero(a) or is_zero(b):
            return S.IntObj(IntVal(0))
    # R12: double negation
    if is_unop(term, NEG) and is_unop(term.arg(1), NEG):
        return term.arg(1).arg(1)
    return term
```

This is decidable (it's syntactic pattern matching on Z3 terms, terminating because each rule reduces term complexity).

26.1.3 What the Ring Axioms Enable

The ring axioms are the path constructors for the D1 (Rec↔Iter) dichotomy. They enable:

- Factorial: $n \times f(n-1) \equiv f(n-1) \times n$ (R6).
- Sum: $n + f(n-1) \equiv f(n-1) + n$ (R1).
- Product: $\prod_{i=1}^{n} a_i$ in any order (R6 + R7 iterated).
- Polynomial equivalence: $n(n+1)/2 \equiv 1 + 2 + \ldots + n$ (R1 + R2 + R11 iterated).

Proposition 26.3 (Ring Paths for Fold Equivalence)**.** *If the fold operation* op *is in* $\{+, \times\}$ *and the step functions of two programs differ only by operand order, the ring commutativity paths (R1 or R6) connect them.*

More generally: any two ways of computing a polynomial over the input variables that give the same polynomial (by the polynomial identity) are connected by a finite sequence of ring paths.

26.2 The Lattice of Min and Max

The operations min and max form a *distributive lattice* on the integers (with the usual ordering).

Definition 26.4 (Min-Max Lattice Paths).

$$\mathsf{min_comm} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\mathsf{MIN}, a, b),\ \mathsf{binop}(\mathsf{MIN}, b, a)) \tag{L1}$$

$$\mathsf{max_comm} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\mathsf{MAX}, a, b),\ \mathsf{binop}(\mathsf{MAX}, b, a)) \tag{L2}$$

$$\mathsf{min_assoc} : \prod_{a,b,c} \mathsf{Path}(\mathsf{binop}(\mathsf{MIN}, \mathsf{binop}(\mathsf{MIN}, a, b), c),\ \mathsf{binop}(\mathsf{MIN}, a, \mathsf{binop}(\mathsf{MIN}, b, c))) \tag{L3}$$

$$\mathsf{max_assoc} : \prod_{a,b,c} \ldots \tag{L4}$$

$$\mathsf{min_idem} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{MIN}, a, a),\ a) \tag{L5}$$

$$\mathsf{max_idem} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{MAX}, a, a),\ a) \tag{L6}$$

$$\mathsf{min_max_absorb} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\mathsf{MIN}, a, \mathsf{binop}(\mathsf{MAX}, a, b)),\ a) \tag{L7}$$

$$\mathsf{max_min_absorb} : \prod_{a,b} \mathsf{Path}(\mathsf{binop}(\mathsf{MAX}, a, \mathsf{binop}(\mathsf{MIN}, a, b)),\ a) \tag{L8}$$

$$\mathsf{min_max_distrib} : \prod_{a,b,c} \mathsf{Path}(\mathsf{binop}(\mathsf{MIN}, a, \mathsf{binop}(\mathsf{MAX}, b, c)),\ \mathsf{binop}(\mathsf{MAX}, \mathsf{binop}(\mathsf{MIN}, a, b), \mathsf{binop}(\mathsf{MIN}, a, c))) \tag{L9}$$

These paths are critical for programs that compute minimums/maximums in different orders (e.g., DP algorithms that compute $\min(\min(a, b), c)$ vs. $\min(a, \min(b, c))$).

Theorem 26.5 (Lattice Paths for DP Equivalence). *If two DP algorithms compute the same recurrence using* min *or* max *but with different association/ordering of the terms, the lattice paths (L1–L9) connect them.*

In particular: the edit distance recurrence $d(i, j) = \min(d(i-1, j)+1,\ d(i, j-1)+1,\ d(i-1, j-1)+c)$ *is invariant under permutation of the three arguments to* min *(by L1 + L3 iterated).*

26.3 The Sequence Algebra

Sequences (lists, tuples) form a *free monoid* under concatenation, with additional structure from sorting, reversal, and length.

26.3.1 The Free Monoid

Definition 26.6 (Sequence Monoid Paths).

$$\mathsf{concat_assoc} : \prod_{x,y,z} \mathsf{Path}((x \mathbin{++} y) \mathbin{++} z,\ x \mathbin{++} (y \mathbin{++} z)) \tag{S1}$$

$$\mathsf{concat_empty_r} : \prod_{x} \mathsf{Path}(x \mathbin{++} [],\ x) \tag{S2}$$

$$\mathsf{concat_empty_l} : \prod_{x} \mathsf{Path}([] \mathbin{++} x,\ x) \tag{S3}$$

where $[] = \mathsf{NoneObj}$ (the empty Pair-chain) and $\mathbin{++}$ is modeled by the shared function $\mathsf{py_call___add__}$ on Pair-chains.

26.3.2 Sorting as a Retraction

Definition 26.7 (Sorting Paths).

$$\mathsf{sorted_idem} : \prod_{x} \mathsf{Path}(\mathsf{py_sorted}(\mathsf{py_sorted}(x)),\ \mathsf{py_sorted}(x)) \tag{S4}$$

$$\mathsf{sorted_perm_inv} : \prod_{x,y} \mathsf{Path}(\mathsf{py_sorted}(x +\!\!+ y),\ \mathsf{py_sorted}(y +\!\!+ x)) \tag{S5}$$

$$\mathsf{sorted_len} : \prod_{x} \mathsf{Path}(\mathsf{py_len}(\mathsf{py_sorted}(x)),\ \mathsf{py_len}(x)) \tag{S6}$$

Remark 26.8 (Sorting as a Retraction in CTTT). In cubical type theory, sorted is a *retraction* of the sequence type onto the subtype of sorted sequences:

$$r : \mathsf{Seq} \to \mathsf{SortedSeq} \hookrightarrow \mathsf{Seq}$$

with $r \circ r = r$ (idempotence, path S4). The retraction forgets the ordering but preserves the multiset of elements.

This means: in the *quotient type* $\mathsf{Seq}/\sim_{\mathsf{perm}}$ (sequences modulo permutation), sorted is the *canonical representative* of each equivalence class. Two sequences with the same sorted form are permutations of each other.

Path S5 says: the sorted form is permutation-invariant. This is the key property that makes sorting-based algorithms equivalent to non-sorting ones when the output only depends on the multiset.

26.3.3 Reversal as an Involution

Definition 26.9 (Reversal Paths).

$$\mathsf{reversed_invol} : \prod_{x} \mathsf{Path}(\mathsf{py_reversed}(\mathsf{py_reversed}(x)),\ x) \tag{S7}$$

$$\mathsf{reversed_len} : \prod_{x} \mathsf{Path}(\mathsf{py_len}(\mathsf{py_reversed}(x)),\ \mathsf{py_len}(x)) \tag{S8}$$

$$\mathsf{sorted_reversed} : \prod_{x} \mathsf{Path}(\mathsf{py_sorted}(\mathsf{py_reversed}(x)),\ \mathsf{py_sorted}(x)) \tag{S9}$$

S7 makes reversed a $\mathbb{Z}/2\mathbb{Z}$-action on sequences: applying it twice returns to the identity. In cubical type theory, this is a *loop* in the function space: $\mathsf{reversed} \cdot \mathsf{reversed} = \mathsf{id}$ (a 2-torsion element of π_1).

S9 says sorting absorbs reversal: the sorted form of a reversed list equals the sorted form of the original. This is because sorting is permutation-invariant (S5), and reversal is a permutation.

26.3.4 Length as a Monoid Homomorphism

Definition 26.10 (Length Paths).

$$\mathsf{len_additive} : \prod_{x,y} \mathsf{Path}(\mathsf{py_len}(x +\!\!+ y),\ \mathsf{binop}(\mathsf{ADD}, \mathsf{py_len}(x), \mathsf{py_len}(y))) \tag{S10}$$

$$\mathsf{len_empty} : \mathsf{Path}(\mathsf{py_len}(\mathsf{NoneObj}),\ \mathsf{IntObj}(0)) \tag{S11}$$

$$\mathsf{len_singleton} : \prod_{a} \mathsf{Path}(\mathsf{py_len}(\mathsf{Pair}(a, \mathsf{NoneObj})),\ \mathsf{IntObj}(1)) \tag{S12}$$

len is a *monoid homomorphism* from $(\mathsf{Seq}, +\!\!+, [])$ to $(\mathbb{N}, +, 0)$. This is a genuine algebraic structure preservation — it means Z3 can reason about lengths using the additive monoid theory (decidable QF_LIA) even when the sequences themselves are symbolic.

26.3.5 Fold as Universal Catamorphism

The most powerful algebraic structure: fold is the *unique catamorphism* from the free monoid of sequences to any algebra.

Definition 26.11 (Fold Algebra Paths)**.**

$$\mathsf{fold_empty} : \prod_{\mathsf{op},e} \mathsf{Path}(\mathsf{fold}(\mathsf{op}, 0, 0, e),\ e) \tag{F1}$$

$$\mathsf{fold_comm} : \prod_{\mathsf{op},e,x,y} \mathsf{Path}(\mathsf{fold}(\mathsf{op}, 0, 2, e)|_{[x,y]},\ \mathsf{fold}(\mathsf{op}, 0, 2, e)|_{[y,x]}) \tag{F2}$$

when op is commutative

$$\mathsf{fold_assoc} : \prod_{\mathsf{op},e,x,y,z} \mathsf{Path}(\mathsf{binop}(\mathsf{op}, \mathsf{fold}(\mathsf{op}, \ldots)|_{[x,y]}, z),\ \mathsf{fold}(\mathsf{op}, \ldots)|_{[x,y,z]}) \tag{F3}$$

when op is associative

$$\mathsf{fold_fusion} : \prod_{h,\mathsf{op}_1,e_1,\mathsf{op}_2,e_2} \mathsf{Path}(h(\mathsf{fold}(\mathsf{op}_1, e_1, \ldots)),\ \mathsf{fold}(\mathsf{op}_2, h(e_1), \ldots)) \tag{F4}$$

when h is a homomorphism from (op_1, e_1) to (op_2, e_2)

Theorem 26.12 (Fold Universality)**.** *For any monoid $(M, \oplus, e)$, the function $\mathsf{fold}(\oplus, \mathsf{start}, \mathsf{stop}, e)$ is the* unique *monoid homomorphism from $(\mathbb{N}, +, 0)$ to $(M, \oplus, e)$ that maps $1 \mapsto \mathsf{IntObj}(\mathsf{start})$.*

This uniqueness is the Nat-recursor path (D1): any two computations that fold the same operation with the same identity produce the same result, regardless of fold direction or order (when the operation is commutative and associative).

Proof. By the universal property of $\mathbb{N}$ as the initial object in the category of monoid homomorphisms from $(\mathbb{N}, +, 0)$. The fold is the image of the unique morphism. Any other computation satisfying the same recurrence factors through this unique morphism, hence equals the fold. □

26.3.6 Sum and Product as Specific Folds

Corollary 26.13 (Sum Permutation Invariance)**.** $\mathsf{py_sum}$ *is permutation-invariant:* $\mathsf{py_sum}(\mathsf{py_sorted}(x)) \equiv \mathsf{py_sum}(x)$ *for all x.*

Proof. sum is $\mathsf{fold}(+, 0, \ldots)$. Since $(+)$ is commutative and associative (R1, R2), the fold is permutation-invariant (F2, F3). Sorting is a permutation. Therefore $\mathsf{sum}(\mathsf{sorted}(x)) = \mathsf{sum}(x)$. □

Similarly for $\mathsf{product}$, min, max, gcd, lcm, xor — any fold over a commutative and associative operation.

26.4 The Boolean Algebra

Python's boolean operations form a *Boolean algebra* (actually a Heyting algebra, since Python 3 doesn't have classical excluded middle for general predicates, but booleans are classical).

Definition 26.14 (Boolean Paths).

$$\mathsf{not_not} : \prod_{p} \mathsf{Path}(\mathsf{unop}(\mathsf{NOT}, \mathsf{unop}(\mathsf{NOT}, p)),\ p) \tag{B1}$$

$$\mathsf{and_comm} : \prod_{p,q} \mathsf{Path}(\mathsf{BoolObj}(\mathsf{bval}(p) \wedge \mathsf{bval}(q)),\ \mathsf{BoolObj}(\mathsf{bval}(q) \wedge \mathsf{bval}(p))) \tag{B2}$$

$$\mathsf{or_comm} : \prod_{p,q} \ldots \tag{B3}$$

$$\mathsf{demorgan_and} : \prod_{p,q} \mathsf{Path}(\mathsf{unop}(\mathsf{NOT}, \mathsf{BoolObj}(\mathsf{bval}(p) \wedge \mathsf{bval}(q))),\ \mathsf{BoolObj}(\neg\mathsf{bval}(p) \vee \neg\mathsf{bval}(q))) \tag{B4}$$

$$\mathsf{demorgan_or} : \prod_{p,q} \ldots \tag{B5}$$

$$\mathsf{demorgan_all} : \prod_{\mathsf{xs}} \mathsf{Path}(\mathsf{py_all}(\mathsf{xs}),\ \mathsf{unop}(\mathsf{NOT}, \mathsf{py_any}(\mathsf{py_map}(\mathsf{NOT}, \mathsf{xs})))) \tag{B6}$$

$$\mathsf{demorgan_any} : \prod_{\mathsf{xs}} \mathsf{Path}(\mathsf{py_any}(\mathsf{xs}),\ \mathsf{unop}(\mathsf{NOT}, \mathsf{py_all}(\mathsf{py_map}(\mathsf{NOT}, \mathsf{xs})))) \tag{B7}$$

B6 and B7 are the *quantifier duality* paths: $\forall x. P(x) \iff \neg\exists x. \neg P(x)$. These directly enable the equivalence of `all`-based and `any`-based programs (benchmark pair eq35).

26.5 The Idempotent Semiring

Several Python operations form *idempotent semirings* — algebraic structures where the "addition" is idempotent ($a + a = a$).

Definition 26.15 (Idempotent Semiring Paths). For min/max as "addition" and $+/\times$ as "multiplication":

$$\mathsf{min_idem} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{MIN}, a, a),\ a) \tag{I1}$$

$$\mathsf{max_idem} : \prod_{a} \mathsf{Path}(\mathsf{binop}(\mathsf{MAX}, a, a),\ a) \tag{I2}$$

$$\mathsf{set_idem} : \prod_{x} \mathsf{Path}(\mathsf{py_set}(\mathsf{py_set}(x)),\ \mathsf{py_set}(x)) \tag{I3}$$

$$\mathsf{frozenset_idem} : \prod_{x} \mathsf{Path}(\mathsf{py_frozenset}(\mathsf{py_frozenset}(x)),\ \mathsf{py_frozenset}(x)) \tag{I4}$$

$$\mathsf{sorted_idem} : \prod_{x} \mathsf{Path}(\mathsf{py_sorted}(\mathsf{py_sorted}(x)),\ \mathsf{py_sorted}(x)) \tag{I5}$$

$$\mathsf{list_idem} : \prod_{x} \mathsf{Path}(\mathsf{py_list}(\mathsf{py_list}(x)),\ \mathsf{py_list}(x)) \tag{I6}$$

Idempotence is critical for programs that apply an operation "just in case" (e.g., $\mathsf{sorted}(\mathsf{sorted}(x))$ when one branch already sorts and another doesn't). The idempotence path collapses the double application to a single one.

Proposition 26.16 (Idempotent Fold). *If* op *is idempotent* ($\mathsf{op}(a, a) = a$ *for all* a*), then:*

$$\mathsf{fold}(\mathsf{op}, e, \mathsf{xs_with_duplicates}) \equiv \mathsf{fold}(\mathsf{op}, e, \mathsf{deduplicated_xs})$$

Duplicates in the fold input don't affect the result.

26.6 The Effect Algebra

The *effect algebra* distinguishes mutation from pure computation. Unlike the previous algebras (which generate paths), the effect algebra generates *non-paths*: it asserts that certain terms are *NOT* equal.

Definition 26.17 (Effect Separation Axioms)**.**

$$\mathsf{mut_ne_pure} : \mathsf{py_mut_iadd}(x, v) \neq \mathsf{py_call___add__}(x, v) \tag{E1}$$

$$\mathsf{mut_sort_ne_sorted} : \mathsf{py_mut_sort}(x) \neq \mathsf{py_sorted}(x) \tag{E2}$$

$$\mathsf{mut_reverse_ne_reversed} : \mathsf{py_mut_reverse}(x) \neq \mathsf{py_reversed}(x) \tag{E3}$$

$$\mathsf{ref_ne_val} : \mathsf{Ref}(a) \neq \mathsf{IntObj}(n) \tag{E4}$$

$$\mathsf{ref_identity} : a \neq b \implies \mathsf{Ref}(a) \neq \mathsf{Ref}(b) \tag{E5}$$

E1–E3 are the *mutation-purity separation*: in-place operations and their pure counterparts are *different shared symbols*, so Z3 treats them as unrelated. This is how CTTT correctly handles the 2 FP benchmark cases (neq06, neq20).

E4–E5 are *constructor disjointness*: a Ref is never equal to a value (different constructors in the ADT), and two Refs with different addresses are different objects (identity semantics).

Remark 26.18 (Effects in Cubical Type Theory)**.** In standard cubical type theory, the effect axioms are *negative paths* — they assert the *absence* of a path. mut_ne_pure says: there is NO path from mut_iadd to call_add.

This is formalized using the *empty type* 0:

$$\mathsf{mut_ne_pure} : \mathsf{Path}(\mathsf{mut_iadd}(x, v),\ \mathsf{call_add}(x, v)) \to 0$$

— assuming such a path exists leads to a contradiction.

In Z3, this is implemented trivially: since mut_iadd and call_add are different function symbols, Z3 never identifies them (different uninterpreted functions are always distinguishable).

26.7 The Composed Algebra

The full semantic algebra is the *product* of all sub-algebras:

Definition 26.19 (The Full Semantic Algebra)**.** $\mathcal{A}_{\mathsf{Py}} = \mathcal{R}_{\mathbb{Z}} \times \mathcal{L}_{\min,\max} \times \mathcal{M}_{\mathsf{Seq}} \times \mathcal{B}_{\mathbb{B}} \times \mathcal{I}_{\mathsf{idem}} \times \mathcal{E}_{\mathsf{eff}}$ where:

Sub-algebra	Paths	Structure	Applications
$\mathcal{R}_{\mathbb{Z}}$	R1–R14	Commutative ring	Arithmetic, fold order
$\mathcal{L}_{\min,\max}$	L1–L9	Distributive lattice	DP, optimization
$\mathcal{M}_{\mathsf{Seq}}$	S1–S12, F1–F4	Free monoid + sort/rev	Collection processing
$\mathcal{B}_{\mathbb{B}}$	B1–B7	Boolean algebra	Logical equivalence
$\mathcal{I}_{\mathsf{idem}}$	I1–I6	Idempotent semiring	Deduplication, caching
$\mathcal{E}_{\mathsf{eff}}$	E1–E5	Effect separation	Mutation detection

Total: **52 path constructors** (46 positive paths + 5 negative separation axioms + 1 identity axiom).

Theorem 26.20 (Algebra Soundness)**.** *Every positive path in* $\mathcal{A}_{\mathsf{Py}}$ *is sound: it holds for all* PyObj *values on the relevant type fiber. Every negative axiom (effect separation) is sound: the separated operations are genuinely different.*

Proof. Each sub-algebra's paths are proved sound in its respective section above. The product algebra inherits soundness componentwise (paths in one sub-algebra don't interact with paths in another, since they operate on disjoint sets of function symbols). □

Theorem 26.21 (Algebra Decidability Preservation). *Adding the paths of $\mathcal{A}_{\mathsf{Py}}$ to the path search engine (Definition 18.3) preserves decidability:*

1. *The rewrite system remains terminating (each rule reduces a complexity measure).*
2. *The rewrite system remains confluent (by Theorem 18.4).*
3. *All previously decidable queries remain decidable.*
4. *The paths are applied as rewrite rules (no quantifiers added to the Z3 solver).*

26.8 Interaction Between Sub-Algebras

The sub-algebras are not fully independent — they interact through the PyObj ADT. The interactions provide *additional* paths beyond what each sub-algebra provides alone:

26.8.1 Ring–Fold Interaction

The fold catamorphism (F1–F4) interacts with the ring axioms (R1–R14) to produce the *fold canonicalization* path: any fold of a ring operation produces a canonical form.

Theorem 26.22 (Fold Canonicalization). *For a ring operation* $\mathsf{op} \in \{+, \times\}$*:*

$$\mathsf{fold}(\mathsf{op}, \mathsf{start}, \mathsf{stop}, e) = \mathsf{fold}(\mathsf{op}, \mathsf{start}', \mathsf{stop}', e')$$

if and only if they compute the same ring element (same value as a polynomial in start *and* stop*).*

This is decidable: polynomial identity testing over $\mathbb{Z}$ is in P (by the Schwartz–Zippel lemma applied to integer polynomials, or by direct Z3 integer arithmetic).

26.8.2 Lattice–Fold Interaction

Folding a lattice operation (min or max) produces the lattice join/meet of the elements. Combined with the lattice distributivity (L9), this enables:

Proposition 26.23. $\min(\mathsf{fold}(+, \ldots), \mathsf{fold}(+, \ldots))$ *can be distributed over the folds (by L9 + F3), enabling DP-style equivalences where the "min" is applied at different levels of the recursion.*

26.8.3 Sequence–Boolean Interaction

The De Morgan paths (B6, B7) interact with the sequence fold structure. $\mathsf{all}(\mathsf{xs})$ is semantically $\mathsf{fold}(\wedge, \top, \mathsf{xs})$ (fold of conjunction), and $\mathsf{any}(\mathsf{xs})$ is $\mathsf{fold}(\vee, \bot, \mathsf{xs})$ (fold of disjunction). The De Morgan path transposes these:

$$\mathsf{fold}(\wedge, \top, \mathsf{xs}) \equiv \neg\, \mathsf{fold}(\vee, \bot, \mathsf{map}(\neg, \mathsf{xs}))$$

This is a *natural transformation* between two functors on the sequence category — a genuinely categorical path that the ring and lattice algebras alone cannot produce.

Chapter 27

The Nat-Eliminator Extraction

27.1 The General Extraction Algorithm

Given a Python AST for a recursive function, the Nat-eliminator extraction identifies:

1. The **base case**: the guard condition and return value when the parameter is "small enough."
2. The **step operation**: the binary operation applied to the parameter and the recursive result.
3. The **threshold**: the parameter value at which the base case triggers.

The extraction is *not algorithm-specific*: it works for any recursive function of the form:

```
def f(n):
    if n OP threshold: return base_val
    return BINOP(n, f(n - 1))
```

where `OP` is any comparison ($\leq$, $<$, $=$), `BINOP` is any binary operation ($+$, $\times$, $-$, $//$, $\%$, $|$, $\&$, $\oplus$), and either n or $f(n-1)$ can be on either side of `BINOP` (handled by commutativity when applicable).

27.2 Correctness

Theorem 27.1 (Extraction Correctness)**.** *If the Nat-eliminator extraction succeeds with parameters* $(\mathsf{base}, \mathsf{step}, \mathsf{threshold}, \mathsf{op})$*, then the canonical fold* $\mathsf{fold}(\mathsf{op}, \mathsf{threshold}, \mathsf{ival}(p)+1, \mathsf{IntObj}(\mathsf{base}))$ *computes the same function as the original recursive program.*

Proof. The fold satisfies the same recurrence as the recursive program:

$$\mathsf{fold}(\mathsf{op}, k, k, \mathsf{base}) = \mathsf{base} \quad (\text{base case: } i \geq \mathsf{stop}) \tag{27.1}$$

$$\mathsf{fold}(\mathsf{op}, i, \mathsf{stop}, \mathsf{acc}) = \mathsf{binop}(\mathsf{op}, \mathsf{IntObj}(i), \mathsf{fold}(\mathsf{op}, i+1, \mathsf{stop}, \mathsf{acc})) \quad (\text{step}) \tag{27.2}$$

This matches the recursive definition $f(n) = \mathsf{binop}(\mathsf{op}, n, f(n-1))$ with $f(k) = \mathsf{base}$, by the Nat-recursor uniqueness (Theorem D1). □

Chapter 28

Iteration Beyond Range: Collection Catamorphisms

The Nat-eliminator handles `for i in range(n)`. Real programs iterate over lists, dicts, strings, generators, and composed iterators.

28.1 The List Catamorphism

A list $[a, b, c]$ is the Pair-chain $\mathsf{Pair}(a, \mathsf{Pair}(b, \mathsf{Pair}(c, \mathsf{NoneObj})))$. Iteration over it is a *list fold* (catamorphism on the free monoid):

Definition 28.1 (List Fold)**.** For `for x in lst: acc = step(acc, x)`:

$$\mathsf{list_fold}(\mathsf{lst}, \mathsf{acc}) = \begin{cases} \mathsf{acc} & \mathsf{is_NoneObj}(\mathsf{lst}) \\ \mathsf{list_fold}(\mathsf{snd}(\mathsf{lst}),\ \mathsf{step}(\mathsf{acc}, \mathsf{fst}(\mathsf{lst}))) & \mathsf{is_Pair}(\mathsf{lst}) \\ \mathsf{acc} & \text{otherwise} \end{cases}$$

Listing 28.1: List fold Z3 RecFunction

```
def compile_for_over_list(stmt, env, T):
    S = T.S; T._uid += 1; uid = T._uid
    target = stmt.target.id
    iter_term = compile_expr(stmt.iter, env)
    modified = find_modified(stmt.body)

    for var in modified:
        init = env.get(var)
        rfn = RecFunction(f'lfold_{var}_{uid}', S, S, S)
        lst_s = FreshConst(S, f'_lf_{uid}')
        acc_s = FreshConst(S, f'_la_{uid}')
        body_env = env.copy()
        body_env.put(target, S.fst(lst_s))
        body_env.put(var, acc_s)
        exec_stmts(stmt.body, body_env)
        new_val = body_env.get(var)
        RecAddDefinition(rfn, [lst_s, acc_s],
            If(S.is_NoneObj(lst_s), acc_s,
               If(S.is_Pair(lst_s),
                  rfn(S.snd(lst_s), new_val), acc_s)))
        env.put(var, rfn(iter_term, init))
```

Theorem 28.2 (List Fold = Nat Fold on Length)**.** *For a list of length n:* $\mathsf{list_fold}([a_1, \ldots, a_n], e) = \mathsf{fold}(\mathsf{step}', 0, n, e)$ *where* $\mathsf{step}'(i, \mathsf{acc}) = \mathsf{step}(\mathsf{acc}, a_{i+1})$*. This connects list iteration to range iteration via the D1 path.*

28.2 Dict Iteration

`for k,v in d.items()` compiles as a list fold over the shared function $\mathsf{py_meth_items}(d)$, which returns a Pair-chain of key-value Pairs.

Proposition 28.3 (Dict Iteration Order Independence)**.** *If the fold operation is commutative and associative, the result is independent of dict iteration order. The commutative monoid paths (F2, F3) provide the connecting path.*

28.3 String Iteration

`for ch in s` is a Nat fold over the character index: $\mathsf{str_fold}(s, i, \mathsf{acc}) = \begin{cases} \mathsf{acc} \\ \mathsf{str_fold}(s, i{+}1, \mathsf{step}(\mathsf{acc}, \mathsf{StrObj}(\mathsf{SubSt} \end{cases}$

This integrates directly with the existing Nat fold machinery.

28.4 Composed Iterators

`zip`, `enumerate`, `map`, `filter` transform iterators. Each compiles to a shared function wrapping the inner iterator:

Listing 28.2: Composed iterators

```
# for i,x in enumerate(xs): ...   => list_fold(py_enumerate(xs), acc)
# for a,b in zip(xs,ys): ...      => list_fold(py_zip(xs,ys), acc)
# sum(x*2 for x in xs)            => list_fold(py_map(h, xs), IntObj
    (0))
```

28.5 Generator Consumption

A generator yielding values is modeled as producing a Pair-chain. Full consumption by `list()`, `for`, or `sum` gives a list fold. This is the D6 path: both generator and eager versions produce the same Pair-chain.

Chapter 29

The Recursion Zoo: Beyond Nat Descent

29.1 Tree Recursion

Definition 29.1 (Tree Catamorphism). $f(\mathsf{tree}) = \begin{cases} \mathsf{base}(\mathsf{tree}) & \mathsf{is_leaf} \\ \mathsf{combine}(f(\mathsf{left}), f(\mathsf{right}), \mathsf{tree}) & \text{otherwise} \end{cases}$

This is the unique algebra morphism from the initial algebra of binary trees.

Listing 29.1: Tree recursion Z3 encoding

```
def compile_tree_rec(func, body, env, T):
    S = T.S; T._uid += 1; uid = T._uid
    param = func.args.args[0].arg
    sym = FreshConst(S, f'_tree_{uid}')
    rfn = RecFunction(f'tree_{func.name}_{uid}', S, S)
    rec_env = Env(T, func_name=func.name, is_rec=True)
    rec_env.put(param, sym)
    secs = extract_sections(body, rec_env)
    combined = secs[-1].term
    for s in reversed(secs[:-1]):
        combined = If(s.guard, s.term, combined)
    RecAddDefinition(rfn, [sym], combined)
    return rfn
```

Theorem 29.2 (Tree Catamorphism Uniqueness). *Two functions with the same* $(\mathsf{base}, \mathsf{combine})$ *are equal by the initial algebra universal property. Stack-based traversal with the same combine is connected by D9 (Stack↔Recursion).*

29.2 Multi-Parameter Recursion (2D)

Edit distance, knapsack, LCS all recurse on two parameters: $f(i,j) = \mathsf{step}(f(i-1,j), f(i,j-1), f(i-1,j-1))$.

Listing 29.2: 2D RecFunction

```
f = RecFunction('f_2d', IntSort(), IntSort(), S)
i, j = Ints('i j')
RecAddDefinition(f, [i, j],
    If(i <= 0, base_row(j),
    If(j <= 0, base_col(i),
       step(i, j, f(i-1,j), f(i,j-1), f(i-1,j-1)))))
```

Both memoized and bottom-up DP satisfy the *same* 2D recurrence. By $\mathbb{N} \times \mathbb{N}$-eliminator uniqueness: D16 path.

29.3 Mutual Recursion

$f(n) = \ldots g(n-1) \ldots$, $g(n) = \ldots f(n-1) \ldots$. Model as product recursor $h(n) = (f(n), g(n))$:

Listing 29.3: Mutual recursion as product

```
h = RecFunction('mutual', IntSort(), S)
RecAddDefinition(h, [n],
    If(n <= 0, S.Pair(base_f, base_g),
       S.Pair(step_f(n, S.fst(h(n-1)), S.snd(h(n-1))),
              step_g(n, S.fst(h(n-1)), S.snd(h(n-1))))))
```

29.4 Well-Founded Recursion with Fuel

General recursion on any well-founded relation, made terminating with a fuel parameter:

Listing 29.4: Well-founded recursion

```
f = RecFunction('wf', S, IntSort(), S)
RecAddDefinition(f, [x, fuel],
    If(fuel <= 0, S.Bottom,
    If(is_base(x), base(x),
       step(x, f(smaller(x), fuel - 1)))))
```

Decidable for ground arguments (fuel bounds the depth). Two well-founded recursions with same $(\mathsf{base}, \mathsf{step}, \prec)$ connected by W-type eliminator uniqueness.

Chapter 30

Higher-Order Functions: Compilation and Normalization

30.1 Compilation Rules

Listing 30.1: HOF compilation

```
# map(lambda x: x*2, xs)
# => py_map_{hash("x*2")}(xs)          [shared symbol]

# filter(lambda x: x > 0, xs)
# => py_filter_{hash("x > 0")}(xs)     [shared symbol]

# sorted(xs, key=len)
# => py_sorted_by_{hash("len")}(xs)    [shared symbol]

# reduce(op, xs, init)
# => py_reduce_{hash(op)}(xs, init)    [shared symbol]
```

The key insight: two programs using the *same* lambda body get the *same* shared symbol, regardless of whether it's written as `map(f, xs)`, a comprehension `[f(x) for x in xs]`, or an explicit loop.

30.2 HOF Normalization Rules

1. $\mathsf{reduce}(\oplus, e, \mathsf{map}(h, xs)) \to \mathsf{reduce}(\lambda a, x.\, a \oplus h(x), e, xs)$ (map–fold fusion)
2. $\mathsf{map}(f, \mathsf{map}(g, xs)) \to \mathsf{map}(f \circ g, xs)$ (map composition)
3. $\mathsf{filter}(p, \mathsf{map}(h, xs)) \to \mathsf{map}(h, \mathsf{filter}(p \circ h, xs))$ (filter–map swap, when applicable)
4. $\mathsf{sorted}(\mathsf{list}(x)) \to \mathsf{sorted}(x)$ (list absorption)

Example 30.1 (map+filter vs comprehension)**.** $f(xs) = \mathsf{list}(\mathsf{filter}_{h_1}(\mathsf{map}_{h_2}(xs)))$ vs. $g(xs) = \mathsf{comp}_{h_3}(xs)$.

Path: D4 identifies the comprehension with filter∘map when h_3 matches the composition of h_1 and h_2.

Chapter 31

Complete Normalization Traces

31.1 Trace 1: Factorial (Rec → Fold)

Input: `def f(n): if n<=1: return 1; return n*f(n-1)`

1. **Compilation**: $[\![f]\!] = \mathsf{rec_f}(p_0)$ where $\mathsf{rec_f}(n) = \mathsf{If}(\mathsf{ival}(n) \leq 1, \mathsf{IntObj}(1), \mathsf{binop}(\mathsf{MUL}, n, \mathsf{rec_f}(\mathsf{binop}(\mathsf{SUB}$
2. **Phase 3 (Nat-eliminator)**: Detects base=1, threshold=1, op=MUL. Produces $\mathsf{fold}(\mathsf{MUL}, 1, \mathsf{ival}(p_0)+1, \mathsf{IntObj}(1))$.
3. **Iterative version**: same fold via loop-to-fold.
4. **Comparison**: identical Z3 terms. Path: refl.

31.2 Trace 2: Edit Distance (2D Recurrence)

Input A (memoized): nested `dp(i,j)` with `@lru_cache`.

1. **Phase 1**: Inline nested `dp` (D2). Ignore decorator.
2. **Compilation**: 2D RecFunction $\mathsf{dp}(i, j) = \mathsf{If}(i = 0, j, \mathsf{If}(j = 0, i, \mathsf{binop}(\mathsf{MIN}, \ldots)))$.
3. **Phase 2**: Canonicalize min argument order (L1).
4. **Phase 5**: Shared symbols for subscript, len.
5. **Input B** (DP table): same 2D RecFunction after compiling nested loops as 2D fold.
6. **Comparison**: same recurrence, same bases. Path: D16 (Memo↔DP).

31.3 Trace 3: all vs any (De Morgan)

Input A: `all(pred(x) for x in seq)`.
Input B: `not any(not pred(x) for x in seq)`.

1. **Phase 5**: A → $\mathsf{py_all}(\mathsf{comp}_{h_1}(p_0))$. B → $\mathsf{unop}(\mathsf{NOT}, \mathsf{py_any}(\mathsf{comp}_{h_2}(p_0)))$.
2. comp_{h_2} has body `not pred(x)` = $\mathsf{map}(\mathsf{NOT}, \mathsf{comp}_{h_1})$.
3. **Path B6** (De Morgan): $\mathsf{py_all}(xs) \equiv \neg\, \mathsf{py_any}(\mathsf{map}(\neg, xs))$.
4. Certificate: 2 steps.

31.4 Trace 4: Sorted Unique

Input A: `return sorted(set(xs))`.
Input B: manual dedup + insertion sort.

1. **Phase 5**: A $\to$ $\mathsf{py_sorted}(\mathsf{py_set}(p_0))$.
2. B compiles to a list fold with insert operations.
3. **Path**: requires axiom $\mathsf{py_sorted}(\mathsf{py_set}(x))$ = canonical "sorted unique elements". Both programs produce this canonical form. Path: $\mathsf{I3} \cdot \mathsf{I5}$ (set idempotence + sorted idempotence).

Part VII

Specification Checking and Bug Finding via CTTT

Chapter 32

The Three Problems, One Framework

The cubical–cohomological framework solves three problems with the *same* mathematical machinery:

Problem	**CTTT formulation**	**Z3 query**
Equivalence	$\mathsf{Path}_{A\to B}(f,g)$ exists?	$\exists x.\, [\![f]\!](x) \neq [\![g]\!](x)$?
Spec checking	$\mathsf{Path}_{A\to B}(f,\mathsf{spec})$ exists?	$\exists x.\, \neg\, S(x, [\![f]\!](x))$?
Bug finding	$H^1(\mathcal{U}, \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_{\mathsf{spec}})) \neq 0$?	which fiber has the obstruction?

Equivalence asks: is there a cubical path from f to g?

Spec checking asks: is there a cubical path from f to *any* program satisfying the specification S? Since S is deterministic (at most one valid output per input), this reduces to: does f itself satisfy S?

Bug finding asks: on which type fiber does f *fail* to satisfy S? The cohomological decomposition localizes the bug to a specific fiber — a specific type of input where the program misbehaves.

All three use:

1. The **PyObj Z3 theory** to compile programs to Z3 terms.
2. The **type fibration** to decompose the input space.
3. The **Čech complex** to relate local and global properties.
4. The **path constructors** to bridge structural gaps.

The only difference is what we compare the program's denotation *against*: another program (equivalence), a specification (spec checking), or an ideal behavior (bug finding).

Chapter 33

Specification Checking

33.1 Specifications as Dependent Types

In CTTT, a specification is a *dependent type over the value presheaf*:

Definition 33.1 (Specification)**.** A specification S for a function type $A \to B$ is a type family:

$$S : \prod_{x:A} B \to \mathsf{Type}$$

where $S(x, y)$ is inhabited iff y is a valid output for input x.

A program $f : A \to B$ *satisfies* S iff:

$$\vdash \lambda x.\, \mathsf{witness}(x) : \prod_{x:A} S(x, f(x))$$

— for every input x, there is a witness that $f(x)$ is valid.

In Z3, this becomes: is there any x where $\neg S(x, [\![f]\!](x))$?

- **UNSAT**: f satisfies S for all inputs. The UNSAT certificate is the constructive witness.
- **SAT**: f violates S on the counterexample x from the Z3 model. This is a *bug*.

33.2 Specifications from Natural Language

Deppy's `@guarantee` decorator attaches a natural-language specification to a function:

```
@guarantee("returns a sorted list with no duplicates")
def unique_sorted(lst):
    return sorted(set(lst))
```

The NL specification is parsed into a conjunction of Z3 predicates:

$$S_1(x, y) : \mathsf{is_sorted}(y) \qquad \text{“sorted”} \tag{33.1}$$
$$S_2(x, y) : \mathsf{no_duplicates}(y) \qquad \text{“no duplicates”} \tag{33.2}$$
$$S_3(x, y) : \mathsf{subset}(y, x) \qquad \text{“from the input”} \tag{33.3}$$
$$S(x, y) = S_1 \wedge S_2 \wedge S_3 \tag{33.4}$$

Each predicate is encoded using the shared function symbols:

- $\mathsf{is_sorted}(y) \equiv \mathsf{py_sorted}(y) = y$ (using the $\mathsf{sorted_idem}$ axiom).
- $\mathsf{no_duplicates}(y) \equiv \mathsf{py_set}(y) = y$ (using the $\mathsf{set_idem}$ axiom).
- $\mathsf{subset}(y, x) \equiv \mathsf{py_set}(y) \subseteq \mathsf{py_set}(x)$.

33.3 Fiber-Wise Spec Checking

The cohomological decomposition applies to spec checking:

Definition 33.2 (Specification Presheaf)**.** The *specification presheaf* Spec_S assigns to each type fiber U the set of valid behaviors:

$$\mathsf{Spec}_S(U) = \{(x, y) \in U \times B \mid S(x, y)\}$$

The program f satisfies S on fiber U iff:

$$\forall x \in U.\ (x, [\![f]\!](x)) \in \mathsf{Spec}_S(U)$$

This is checked fiber-by-fiber, just like equivalence:

1. For each fiber U_τ, ask Z3: $\exists x.\, \mathsf{tag}(x) = \tau \wedge \neg S(x, [\![f]\!](x))$.
2. Collect local verdicts.
3. Compute H^1 to check if local satisfaction glues globally.

The Čech decomposition is especially valuable for spec checking because specifications often have *different constraints on different types*:

- On the Int fiber: "output is non-negative."
- On the Str fiber: "output is non-empty."
- On the List fiber: "output has the same length."

Each constraint is checked independently on its fiber, then glued.

33.4 Example: Sorting Specification

Listing 33.1: Sorting with spec check

```
@guarantee("returns a sorted permutation of the input")
def my_sort(lst):
    result = []
    for x in lst:
        # BUG: inserts at wrong position
        result.insert(0, x)
    return result
```

Compilation: $[\![\mathsf{my_sort}]\!](p_0)$ produces a sequence built by repeated $\mathsf{py_meth_insert}(0, x)$ — which prepends each element, producing the *reverse* of the input.

Spec: $S(x, y) = \mathsf{py_sorted}(y) = y \ \wedge\ \mathsf{len}(y) = \mathsf{len}(x)$.

Z3 check: Is there an x where $\mathsf{py_sorted}([\![f]\!](x)) \neq [\![f]\!](x)$? Z3 finds a counterexample: $x = [2, 1]$, where $[\![f]\!]([2, 1]) = [1, 2]$ is actually sorted (by accident!), but $x = [1, 2]$ gives $[\![f]\!]([1, 2]) = [2, 1]$ which is NOT sorted. **SAT** — bug found.

Fiber localization: The bug is on the Pair fiber (list inputs). $H^1(\{\mathsf{Pair}\}, \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_S)) \neq 0$.

33.5 Example: Fibonacci Specification

Listing 33.2: Fibonacci with off-by-one bug

```
@guarantee("returns the n-th Fibonacci number")
def fib(n):
    if n <= 1: return n
    a, b = 0, 1
    for _ in range(n):   # BUG: should be range(n-1)
        a, b = b, a + b
    return a
```

Spec: $S(n, y)$ is the Fibonacci recurrence: $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$.

Compilation: The loop runs n times instead of $n - 1$, producing $F(n + 1)$ instead of $F(n)$ for $n \geq 2$.

Z3 check: $\exists n. \mathsf{tag}(n) = \mathsf{TInt} \wedge n \geq 0 \wedge [\![f]\!](n) \neq F(n)$. Z3 finds $n = 2$: $[\![f]\!](2) = 2$ but $F(2) = 1$. **SAT** — bug found.

Fiber localization: Bug is on the Int fiber for $n \geq 2$. The base cases $n = 0, 1$ pass (the loop doesn't execute or executes once correctly).

33.6 Spec Checking via Equivalence

A powerful technique: reduce spec checking to equivalence checking.

Theorem 33.3 (Spec Checking as Equivalence)**.** *Let S be a deterministic specification with a known* reference implementation *r (a program that satisfies S). Then f satisfies S iff $f \equiv r$.*

Proof. Since S is deterministic, there is exactly one valid output per input. Both f and r must produce this output. So f satisfies S iff $f(x) = r(x)$ for all x iff $f \equiv r$. □

This reduces spec checking to the equivalence problem, which we already solve with the full cubical–cohomological machinery (path constructors, dichotomy theories, Čech complex).

Example 33.4. To check that a sorting implementation is correct, compare it against `sorted` (the reference implementation). The shared function symbol $\mathsf{py_sorted}$ and the sorted axioms handle the comparison.

Chapter 34

Bug Finding via Cohomological Obstruction

34.1 Bugs as H^1 Obstructions

A *bug* is a violation of a specification. In CTTT, bugs are *cohomological obstructions*: elements of H^1 that prevent local correctness from gluing to global correctness.

Definition 34.1 (Bug as Obstruction)**.** A bug in program f with respect to specification S is a non-trivial element of:

$$H^1(\mathcal{U}, \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_S))$$

where Sem_S is the presheaf of the specification (the "ideal" behavior).

The obstruction element tells us:

1. **Which fiber**: the type of input that triggers the bug.
2. **Which overlap**: whether the bug is a local failure (wrong answer on one fiber) or a gluing failure (correct on each fiber separately but inconsistent across fibers).
3. **The counterexample**: the Z3 model gives a concrete input that exhibits the bug.

34.2 Bug Localization

The Čech decomposition provides automatic *bug localization*:

Theorem 34.2 (Bug Localization via Fiber Decomposition)**.** *If $H^1(\mathcal{U}, \mathsf{Iso}(\mathsf{Sem}_f, \mathsf{Sem}_S)) \neq 0$, the obstruction is localized to specific fibers $U_{i_1}, \ldots, U_{i_k}$ where the local checks fail. The Z3 counterexamples on these fibers are the* minimal bug-triggering inputs.

Example 34.3 (Type-Dependent Bug)**.**

```
@guarantee("returns the absolute value")
def my_abs(x):
    if x < 0:
        return -x
    return x
```

This is correct on the Int fiber. But on the Str fiber, the comparison `x < 0` is meaningless (Python 3 raises `TypeError` for `str < int`).

The Čech decomposition detects:

- $\mathsf{Iso}(\mathsf{Int})$: UNSAT (correct on integers).

- $\mathsf{Iso}(\mathsf{Str})$: SAT (bug on strings — comparison with integer fails).
- $H^1 \neq 0$ on the Str fiber.

The bug is localized to the string fiber. The fix: add a type guard `if not isinstance(x, (int, float)): raise TypeError(...)`.

34.3 Bug Classification by Cohomological Degree

Bugs can be classified by which cohomological degree they appear in:

Degree	Meaning	Example
$H^0 = 0$	No correct behavior at all	Function always crashes
$H^0 \neq 0, H^1 \neq 0$	Correct on some fibers, wrong on others	Off-by-one for large n
$H^1 = 0, H^2 \neq 0$	Locally correct, inconsistent gluing	Type coercion bug

$H^0 = 0$ **bugs**: The program fails the specification on *every* fiber. This is a total failure — the algorithm is fundamentally wrong.

$H^0 \neq 0, H^1 \neq 0$ **bugs**: The program is correct on some fibers but not others. This is the most common case — edge cases, boundary conditions, type-specific failures.

$H^1 = 0, H^2 \neq 0$ **bugs**: These are subtle — the program is correct on each fiber individually, but the transition functions between fibers are inconsistent. This occurs when a program handles type coercions incorrectly (e.g., treating `True` as `1` in one branch but as a boolean in another).

34.4 The Bug-Finding Pipeline

1. **Input**: program f + specification S (NL or formal).
2. **Compile**: $[\![f]\!]$ and $[\![S]\!]$ to Z3 terms.
3. **Decompose**: build type site, identify fibers.
4. **Local check**: for each fiber U_τ, ask Z3: $\exists x \in U_\tau. \neg S(x, [\![f]\!](x))$.
5. **Report**:
   - If all fibers pass: "No bugs found (verified on all type fibers)."
   - If fiber U_τ fails: "Bug on τ inputs: $f(\mathsf{counterexample}) = \mathsf{wrong_value}$, expected to satisfy S."
6. **Localize**: rank fibers by severity (how many section pairs fail, how deep the obstruction).

34.5 Example: Off-by-One in Range

Listing 34.1: Sum with off-by-one

```python
@guarantee("returns the sum 1 + 2 + ... + n")
def sum_to_n(n):
    total = 0
    for i in range(n):  # BUG: should be range(1, n+1)
        total += i
    return total
```

Spec: $S(n, y) = (y = n \cdot (n + 1)/2)$.

Compilation: The loop computes $\mathsf{fold}(\mathsf{ADD}, 0, n, \mathsf{IntObj}(0))$, which is $0+1+\ldots+(n-1) = n(n-1)/2$.

Z3 check: $\exists n.\, n \geq 0 \wedge n(n-1)/2 \neq n(n+1)/2$. Z3 finds $n = 2$: program returns 1, spec says 3.

Obstruction: $H^1(\mathsf{Int}) \neq 0$. The bug is an off-by-one in the range bounds — the fold starts at 0 instead of 1 and stops at n instead of $n + 1$.

34.6 Example: Mutation Bug (False Aliasing)

Listing 34.2: Mutable default argument

```
@guarantee("returns a list containing just x")
def make_list(x, lst=[]):
    lst.append(x)
    return lst
```

Spec: $S(x, y) = (y = [x])$.

Bug: The mutable default argument `lst=[]` is shared across calls. On the first call, $f(1) = [1]$ (correct). On the second call, $f(2) = [1, 2]$ (wrong — the list persists).

CTTT analysis: The parameter `lst` has type Ref (mutable). On the Ref fiber, the mutation $\mathsf{py_mut_append}$ modifies a shared heap location. The spec says the output should be $[\mathsf{x}]$, but the Z3 term shows accumulation: $\mathsf{mut_append}(\mathsf{mut_append}(\ldots, x_1), x_2)$.

Obstruction: $H^1(\mathsf{Ref}) \neq 0$. The bug is on the Ref fiber — exactly where mutation effects matter.

Chapter 35

Unified Theory: Equivalence, Specs, and Bugs

35.1 The Verification Triangle

The three problems form a triangle connected by the CTTT framework:

$$\begin{tikzcd}
\text{Equivalence} \arrow[r, "\text{spec as ref. impl.}"] \arrow[d, "\text{bug = failed equiv.}"'] & \text{Spec Checking} \arrow[dl, "\text{bug = spec violation}"] \\
\text{Bug Finding} &
\end{tikzcd}$$

- **Equivalence → Spec Checking**: Given a reference implementation r satisfying spec S, check $f \equiv r$ (Theorem 33.3).
- **Spec Checking → Bug Finding**: If the spec check fails, the counterexample is a bug.
- **Bug Finding → Equivalence**: If two programs have the *same* bugs (same H^1 obstructions on the same fibers), they might still be equivalent ("equivalently buggy").

35.2 The Universal Verification Query

All three problems reduce to a single Z3 query form:

$$\exists x.\ \mathsf{tag}(x) \in F \ \wedge\ [\![f]\!](x) \neq \mathsf{target}(x)$$

where:

- F is the set of allowed type fibers (from duck type inference).
- target is:
  - $[\![g]\!]$ for equivalence checking,
  - the specification's unique output for spec checking,
  - the "ideal" behavior for bug finding.

The cohomological decomposition splits this into per-fiber queries. The path constructors (equational axioms) help Z3 prove that $[\![f]\!](x) = \mathsf{target}(x)$ even when the terms are structurally different.

35.3 Composition: Spec-Guided Equivalence

A powerful composition: use a specification to *guide* equivalence checking.

Theorem 35.1 (Spec-Guided Equivalence)**.** *If both f and g satisfy a deterministic specification S, then $f \equiv g$.*

This is Theorem 24.2 from Chapter 24, but now we have the full pipeline to check it:

1. Check $f \models S$ (spec checking via Z3).
2. Check $g \models S$ (spec checking via Z3).
3. If both pass, conclude $f \equiv g$ (by deterministic specification uniqueness).

This handles the hardest equivalence cases (dichotomy D20: different algorithms for the same problem) by reducing them to two independent spec checks.

35.4 Composition: Equivalence-Guided Bug Finding

Another composition: use a known-correct program to find bugs in a new implementation.

Theorem 35.2 (Differential Testing via CTTT)**.** *Given a reference implementation r (known correct) and a new implementation f, the equivalence check $f \equiv r$ either:*

1. *Succeeds ($H^1 = 0$): f is correct (no bugs).*
2. *Fails ($H^1 \neq 0$): the obstruction localizes the bug to a specific fiber, with a concrete counterexample.*

This is *differential testing without execution*: we compare Z3 denotations, not runtime outputs. The counterexample is a *symbolic* input, not a random sample.

35.5 The Graded Verification Verdict

The full CTTT verdict is richer than a simple pass/fail:

Definition 35.3 (Graded Verification Verdict)**.** The verdict for program f against target T (another program or a specification) is a triple:

$$(H^0, H^1, \mathsf{obstructions})$$

where:

- H^0 = number of fibers where f matches T ("how much is correct").
- H^1 = number of obstructions ("how many bugs").
- $\mathsf{obstructions}$ = list of $(\mathsf{fiber}, \mathsf{counterexample})$ pairs ("where and what are the bugs").

A perfect program has $(H^0 = |\mathcal{U}|, H^1 = 0, [])$. A totally wrong program has $(H^0 = 0, H^1 = |\mathcal{U}|, [\ldots])$. A program with one edge-case bug has $(H^0 = |\mathcal{U}| - 1, H^1 = 1, [(\tau, x)])$.

This graded verdict is more informative than "correct" or "incorrect" — it tells the developer *exactly* where the program fails and on what type of input.

Part VIII

Implementation Architecture

Chapter 36

The Verification Pipeline

36.1 Pipeline Overview

1. **Parse**: Python source → AST.
2. **Compile**: AST → Z3 PyObj terms (sections).
3. **Duck type inference**: narrow type fibers per parameter.
4. **Build site category**: type fibers → sites + morphisms + overlaps.
5. **Local equivalence**: per-fiber Z3 SAT check.
6. **Čech** H^1: full cohomology computation.
7. **Verdict**: $H^1 = 0 \Rightarrow$ Equivalent; $H^1 \neq 0 \Rightarrow$ Inequivalent.

36.2 AST Compilation

The compiler walks the Python AST depth-first, producing Z3 terms:

Python construct	**Z3 term**
Integer literal n	$\mathsf{IntObj}(n)$
String literal s	$\mathsf{StrObj}(s)$
Variable x	$\mathsf{env}[x]$ (Z3 Const)
$x + y$	$\mathsf{binop}(\mathsf{ADD}, [\![x]\!], [\![y]\!])$
$-x$	$\mathsf{unop}(\mathsf{NEG}, [\![x]\!])$
$x > y$	$\mathsf{binop}(\mathsf{GT}, [\![x]\!], [\![y]\!])$
x `and` y	$\mathsf{If}(\mathsf{truthy}([\![x]\!]), [\![y]\!], [\![x]\!])$
x `or` y	$\mathsf{If}(\mathsf{truthy}([\![x]\!]), [\![x]\!], [\![y]\!])$
`if c: t else: f`	$\mathsf{If}(\mathsf{truthy}([\![c]\!]), [\![t]\!], [\![f]\!])$
`sorted(x)`	$\mathsf{py_sorted}([\![x]\!])$ (shared)
`x.append(v)`	$\mathsf{py_mut_append}([\![x]\!], [\![v]\!])$ (shared)
`isinstance(x, int)`	$\mathsf{BoolObj}(\mathsf{tag}([\![x]\!]) = \mathsf{TInt})$
`for i in range(n)`	fold or RecFunction
Recursive $f(n-1)$	RecFunction or canonical fold
`[e for x in xs]`	$\mathsf{py_comp}_h([\![xs]\!])$ (shared)

36.3 Section Extraction

The compiler produces a list of *sections* — pairs $(\mathsf{guard}, \mathsf{term})$:

- Each `return` statement in the function body produces one section.

- The `guard` is the conjunction of all branch conditions leading to that return.
- The `term` is the Z3 expression for the return value.

These sections form the value presheaf: each section is a "local behavior" of the program under a specific set of conditions.

36.4 Site Category Construction

The type site is built from duck type inference:

1. For each parameter, analyze its usage in both programs to determine allowed types (int, str, list, etc.).
2. Form the Cartesian product of allowed types across parameters.
3. Each element of the product is a site (type fiber assignment).
4. Morphisms connect fibers that share at least one type component.

36.5 Local Equivalence Check

For each site U_i, the Z3 solver checks:

$$\exists \vec{p}.\ \bigwedge_k \mathsf{tag}(p_k) = \tau_k^{(i)} \ \wedge\ \mathsf{tag}(p_k) \neq \mathsf{TBot} \ \wedge\ [\![f]\!](\vec{p}) \neq [\![g]\!](\vec{p})$$

If UNSAT: local equivalence holds ($\mathsf{Iso}(U_i) = 1$). If SAT: counterexample found ($\mathsf{Iso}(U_i) = 0$).

The solver includes all semantic axioms (Section 26), enabling Z3 to use the path constructors during proof search.

36.6 Čech Cohomology Computation

The local judgments feed into the `CechCohomologyComputer`:

1. Build C^0 from local judgments.
2. Compute δ^0 via `CoboundaryOperator.d0`.
3. Compute δ^1 via `CoboundaryOperator.d1`.
4. Compute $H^1 = \ker(\delta^1)/\mathrm{im}(\delta^0)$ via GF(2) rank computation.
5. If $H^1 = 0$: EQUIVALENT.
6. If $H^1 \neq 0$: INEQUIVALENT with obstruction data.

Chapter 37

The AST Compiler in Detail

This chapter specifies the compilation from Python AST to Z3 PyObj terms with enough detail that a reader could implement it from scratch.

37.1 Compilation State

The compiler maintains an *environment* — a mapping from variable names to Z3 terms:

Definition 37.1 (Compilation Environment)**.**

$$\mathsf{Env} = \{T : \mathsf{Theory},\ \mathsf{bindings} : \mathsf{str} \to \mathsf{PyObj},\ \mathsf{func_name} : \mathsf{str},\ \mathsf{is_rec} : \mathbb{B}\}$$

The environment is *immutable* (in the cubical sense): branching creates copies. When an `if` statement has both a `then` and `else` branch, the compiler copies the environment for each branch and merges the results with If expressions.

37.2 Expression Compilation Rules

Each Python expression AST node maps to a Z3 PyObj term. We give the complete rule set:

37.2.1 Rule E1: Literals

$$[\![n]\!]_{\mathsf{env}} = \mathsf{IntObj}(n) \quad [\![s]\!]_{\mathsf{env}} = \mathsf{StrObj}(s) \quad [\![\mathsf{True}]\!]_{\mathsf{env}} = \mathsf{BoolObj}(\top) \quad [\![\mathsf{None}]\!]_{\mathsf{env}} = \mathsf{NoneObj}$$

37.2.2 Rule E2: Variables

$$[\![x]\!]_{\mathsf{env}} = \begin{cases} \mathsf{env.bindings}[x] & \text{if } x \in \mathsf{env.bindings} \\ \mathsf{FreshConst}(\mathsf{PyObj}) & \text{otherwise (undeclared variable)} \end{cases}$$

37.2.3 Rule E3: Binary Operations

$$[\![a \oplus b]\!]_{\mathsf{env}} = \mathsf{binop}([\![\oplus]\!], [\![a]\!]_{\mathsf{env}}, [\![b]\!]_{\mathsf{env}})$$

where $[\![\oplus]\!]$ maps Python operators to integer operation tags:

+	1	-	2	*	3	//	5
%	6	<	20	<=	21	>	22
>=	23	==	24	!=	25	|	12

37.2.4 Rule E4: Unary Operations

$$[\![-a]\!]_{\mathsf{env}} = \mathsf{unop}(\mathsf{NEG}, [\![a]\!]_{\mathsf{env}}) \qquad [\![\mathsf{not}\ a]\!]_{\mathsf{env}} = \mathsf{unop}(\mathsf{NOT}, [\![a]\!]_{\mathsf{env}})$$

37.2.5 Rule E5: Comparisons (Chained)

For $a < b \leq c$:

$$[\![a < b \leq c]\!]_{\mathsf{env}} = \mathsf{If}(\mathsf{truthy}(\mathsf{binop}(\mathsf{LT}, [\![a]\!], [\![b]\!])),\ \mathsf{binop}(\mathsf{LE}, [\![b]\!], [\![c]\!]),\ \mathsf{BoolObj}(\bot))$$

37.2.6 Rule E6: Boolean Operations

$$[\![a \text{ and } b]\!]_{\mathsf{env}} = \mathsf{If}(\mathsf{truthy}([\![a]\!]), [\![b]\!], [\![a]\!]) \tag{37.1}$$

$$[\![a \text{ or } b]\!]_{\mathsf{env}} = \mathsf{If}(\mathsf{truthy}([\![a]\!]), [\![a]\!], [\![b]\!]) \tag{37.2}$$

Note: Python's `and`/`or` return the *value*, not a boolean. The Z3 encoding faithfully preserves this.

37.2.7 Rule E7: Conditional Expression

$$[\![t \text{ if } c \text{ else } f]\!]_{\mathsf{env}} = \mathsf{If}(\mathsf{truthy}([\![c]\!]), [\![t]\!], [\![f]\!])$$

37.2.8 Rule E8: Function Calls

Function calls have the most complex compilation:

1. **Self-recursive call**: If $\mathsf{env.is_rec}$ and the callee is $\mathsf{env.func_name}$, create a RecFunction application: $\mathsf{rec_f}([\![a_1]\!], \ldots, [\![a_n]\!])$.
2. **Nested function call**: If the callee is in the environment as $(\mathsf{__func__}, \mathsf{AST})$, invoke $\mathsf{inline_call}$: substitute arguments into the body and compile the body.
3. **isinstance**: If the callee is `isinstance` with two arguments, compile to: $\mathsf{BoolObj}(\mathsf{tag}([\![x]\!]) = \tau)$ where τ is the type tag corresponding to the second argument.
4. **Shared builtins**: If the callee name is in $\mathsf{SHARED_BUILTINS}$ (`sorted`, `len`, `sum`, etc.), use the shared function symbol: $T.\mathsf{shared_fn}(\mathsf{name}, n)([\![a_1]\!], \ldots, [\![a_n]\!])$.
5. **Known builtins with Z3 semantics**: `abs`, `int`, `bool`, `len`, `max`, `min` compile to unop or binop with the appropriate operation tag.
6. **Other named calls**: Use a shared function symbol keyed by the callee name: $T.\mathsf{shared_fn}(\mathsf{call_name}, n)([\![$
7. **Method calls**: Compile receiver and arguments, use shared method symbol: $T.\mathsf{shared_fn}(\mathsf{meth_name}, n+1)([\![\mathsf{recv}]\!], [\![a_1]\!], \ldots, [\![a_n]\!])$.

37.2.9 Rule E9: Subscript

$$[\![a[b]]\!]_{\mathsf{env}} = \mathsf{binop}(\mathsf{GETITEM}, [\![a]\!], [\![b]\!])$$

37.2.10 Rule E10: Tuple/List Display

$$[\![[a, b, c]]\!]_{\mathsf{env}} = \mathsf{Pair}([\![a]\!], \mathsf{Pair}([\![b]\!], \mathsf{Pair}([\![c]\!], \mathsf{NoneObj})))$$

37.2.11 Rule E11: Comprehension

$$[\![[e \text{ for } x \text{ in } \mathsf{xs}]]\!]_{\mathsf{env}} = T.\mathsf{shared_fn}(\mathsf{comp}_{h(e)}, 1)([\![\mathsf{xs}]\!])$$

where $h(e) = \mathsf{hash}(\mathsf{ast.dump}(e)) \mod 2^{31}$ is the canonical hash of the element expression.

37.2.12 Rule E12: Attribute Access

$$[\![a.\mathsf{attr}]\!]_{\mathsf{env}} = T.\mathsf{shared_fn}(\mathsf{attr_attr}, 1)([\![a]\!])$$

37.3 Statement Compilation Rules

Statements modify the environment and produce *sections* (for `return` statements).

37.3.1 Rule S1: Assignment

$$\mathsf{exec}(x = e, \mathsf{env}) = \mathsf{env}[x \mapsto [\![e]\!]]$$

For tuple unpacking $x, y = e$:

$$\mathsf{exec}((x, y) = e, \mathsf{env}) = \mathsf{env}[x \mapsto \mathsf{binop}(\mathsf{GETITEM}, [\![e]\!], \mathsf{IntObj}(0)),\ y \mapsto \mathsf{binop}(\mathsf{GETITEM}, [\![e]\!], \mathsf{IntObj}(1))]$$

37.3.2 Rule S2: Augmented Assignment

$$\mathsf{exec}(x \oplus= e, \mathsf{env}) = \mathsf{env}[x \mapsto \mathsf{binop}([\![\oplus]\!], \mathsf{env}[x], [\![e]\!])]$$

37.3.3 Rule S3: Return

$$\mathsf{exec}(\mathsf{return}\ e, \mathsf{env}, \mathsf{guard}) = \mathsf{Section}(\mathsf{guard}, [\![e]\!])$$

This terminates the current execution path and produces a section.

37.3.4 Rule S4: If/Elif/Else

$$\mathsf{exec}(\mathsf{if}\ c : B_1\ \mathsf{else} : B_2, \mathsf{env}, \mathsf{guard})$$

1. Compile the condition: $\mathsf{cond} = \mathsf{truthy}([\![c]\!])$.
2. True branch: $\mathsf{guard}_T = \mathsf{guard} \wedge \mathsf{cond}$, compile B_1 with $\mathsf{env}_T = \mathsf{env.copy}()$.
3. False branch: $\mathsf{guard}_F = \mathsf{guard} \wedge \neg\mathsf{cond}$, compile B_2 with $\mathsf{env}_F = \mathsf{env.copy}()$.
4. If both branches return: collect sections from both.
5. If only one returns: collect its sections, continue with the other branch's modified environment.
6. If neither returns: merge environments with $\mathsf{env}[x] = \mathsf{If}(\mathsf{cond}, \mathsf{env}_T[x], \mathsf{env}_F[x])$ for each modified variable x.

37.3.5 Rule S5: For Loop

$$\mathsf{exec}(\mathsf{for}\ i\ \mathsf{in}\ \mathsf{range}(\mathsf{start}, \mathsf{stop}) : B, \mathsf{env})$$

Case 1: Canonical fold. If B is a single augmented assignment $\mathsf{acc} \oplus= i$ (the loop variable):

$$\mathsf{env}[\mathsf{acc}] \leftarrow \mathsf{fold}([\![\oplus]\!], \mathsf{start}, \mathsf{stop}, \mathsf{env}[\mathsf{acc}])$$

Case 2: General RecFunction. For complex loop bodies, create a RecFunction for each modified variable:

$$\mathsf{lp_acc}(i) = \begin{cases} \mathsf{env}[\mathsf{acc}] & \text{if } i \geq \mathsf{stop} \\ \mathsf{body}(\mathsf{lp_acc}(i+1), i) & \text{otherwise} \end{cases}$$

Set $\mathsf{env}[\mathsf{acc}] \leftarrow \mathsf{lp_acc}(\mathsf{start})$.

37.3.6 Rule S6: While Loop

Similar to S5 but with a guard condition instead of a range. Create a RecFunction with a bounded counter:

$$\mathsf{wh_acc}(k) = \begin{cases} \mathsf{env}[\mathsf{acc}] & \text{if } k > 50 \vee \neg\mathsf{guard} \\ \mathsf{body}(\mathsf{wh_acc}(k+1)) & \text{otherwise} \end{cases}$$

37.3.7 Rule S7: Function Definition

$$\mathsf{exec}(\mathsf{def}\ f(\ldots) : B, \mathsf{env}) = \mathsf{env}[f \mapsto (__\mathsf{func}__, \mathsf{AST}(f))]$$

Store the AST node for later inlining (D2 defunctionalization).

37.3.8 Rule S8: Expression Statement (Mutation)

For $\mathsf{obj.method}(\mathsf{args})$:

$$\mathsf{env}[\mathsf{obj}] \leftarrow T.\mathsf{shared_fn}(\mathsf{mut_method}, |\mathsf{args}| + 1)(\mathsf{env}[\mathsf{obj}], [\![\mathsf{args}]\!])$$

This models mutation: calling a method on an object produces a new version of that object.

37.3.9 Rule S9: Try/Except

Compile the `try` body. For each section produced, wrap with an uninterpreted "handler context" function keyed by the exception types:

$$\mathsf{Section}(\mathsf{guard}, \mathsf{try_ctx}(\mathsf{term}))$$

where $\mathsf{try_ctx}$ is a shared function based on the handler signature.

37.4 The Section Extractor

The section extractor traverses a statement list and collects all $\mathsf{Section}(\mathsf{guard}, \mathsf{term})$ pairs reachable by any execution path:

Listing 37.1: Section extraction algorithm

```
def extract_sections(stmts, env, guard=True):
    sections = []
    for stmt in stmts:
        if isinstance(stmt, Return):
            sections.append(Section(guard, compile(stmt.value, env))
                )
            return sections  # path terminates
        elif isinstance(stmt, If):
            cond = truthy(compile(stmt.test, env))
            t_guard = And(guard, cond)
            f_guard = And(guard, Not(cond))
            # Recursively extract from both branches
            t_secs = extract_sections(stmt.body, env.copy(), t_guard
                )
            f_secs = extract_sections(stmt.orelse, env.copy(),
                f_guard)
            sections.extend(t_secs)
            sections.extend(f_secs)
            if has_return(stmt.body) and has_return(stmt.orelse):
                return sections  # both branches return
```

```
            # ... handle partial returns by continuing with modified env
        elif isinstance(stmt, For):
            exec_for(stmt, env)  # modifies env in place
        else:
            exec_one(stmt, env)  # modifies env in place
    return sections
```

The output is a list of sections forming the *value presheaf* of the program. Each section is a local behavior under a specific path condition (guard).

Chapter 38

Duck Type Inference in Detail

38.1 The Duck Type Lattice

Duck type inference determines which type fibers are relevant for each parameter. The result is a *lattice element* from the duck type lattice:

Duck type	Allowed fibers	Triggering operations
int	{TInt}	`-`, `*`, `//`, `%`, `range(x)`
str	{TStr}	`.upper()`, `.split()`, `.strip()`
bool	{TBool}	(rare — usually widened to int)
ref	{TRef}	`.get()`, `.keys()`, `.items()`
list	{TPair, TRef}	`.append()`, `.sort()`, `.pop()`
collection	{TPair, TRef, TStr}	`for x in p`, `len(p)`, `sorted(p)`
any	{TInt, . . . , TRef}	no type-constraining operations

38.2 The Inference Algorithm

Listing 38.1: Duck type inference

```
def infer_duck_type(func_f, func_g, param_name):
    ops_f = extract_operations(func_f, param_name)
    ops_g = extract_operations(func_g, param_name)
    ops = ops_f | ops_g  # union of operations from both programs

    if ops & NUMERIC_ONLY_OPS:
        return 'int'     # arithmetic forces integer
    if ops & STRING_METHOD_OPS:
        return 'str'     # string methods force string
    if ops & DICT_METHOD_OPS:
        return 'ref'     # dict methods force reference
    if ops & LIST_METHOD_OPS:
        return 'list'    # list methods force list/ref
    if ops & ITERATION_OPS:
        return 'collection'  # iteration allows pair/ref/str
    return 'any'         # no constraints
```

Key insight: we analyze *both* programs to determine the duck type. If program A uses arithmetic on parameter x but program B only uses comparison, the union of operations includes arithmetic, so x is constrained to int. This ensures both programs are compared on the *same* type fiber.

38.3 Operation Extraction from AST

The operation extractor walks the AST and collects all operations applied to a given parameter:

Listing 38.2: Operation extraction

```
def extract_operations(func_node, param_name):
    ops = set()
    for node in ast.walk(func_node):
        # Binary op with param on either side
        if isinstance(node, BinOp):
            if mentions(node.left, param_name) or \
               mentions(node.right, param_name):
                ops.add(binop_name(node.op))  # 'add', 'mul', etc.

        # Method call on param
        if isinstance(node, Call) and \
           isinstance(node.func, Attribute) and \
           isinstance(node.func.value, Name) and \
           node.func.value.id == param_name:
            ops.add('method_' + node.func.attr)

        # Iteration over param
        if isinstance(node, For) and \
           isinstance(node.iter, Name) and \
           node.iter.id == param_name:
            ops.add('iter')

        # Builtin call with param as argument
        if isinstance(node, Call) and \
           isinstance(node.func, Name):
            for arg in node.args:
                if mentions(arg, param_name):
                    ops.add('call_' + node.func.id)
    return ops
```

38.4 Fiber Narrowing and the Site Category

Given duck types for all parameters, the site category is constructed:

Listing 38.3: Site category construction

```
def build_site_category(param_fibers):
    """param_fibers: list of lists of allowed tags per param."""
    import itertools
    combos = list(itertools.product(*param_fibers))
    # Limit combinatorial explosion
    if len(combos) > 50:
        combos = combos[:50]

    sites = {}
    for combo in combos:
        site_id = SiteId(name='fiber_' + 'x'.join(combo))
        sites[combo] = site_id

    # Overlaps: two combos that share at least one fiber component
    overlaps = []
    for i, ci in enumerate(combos):
```

```
        for j, cj in enumerate(combos):
            if i < j and any(ci[k] == cj[k] for k in range(len(ci)))
                :
                overlaps.append((sites[ci], sites[cj]))

    return sites, overlaps
```

Chapter 39

The Čech Complex Construction Algorithm

This chapter gives the complete algorithm for building the Čech complex from local equivalence judgments, computing the coboundary operators, and determining H^1.

39.1 Step 1: Local Equivalence Judgments

For each site U_i (type fiber combination), run the Z3 query:

Listing 39.1: Local equivalence check

```
def check_local(fiber_combo, secs_f, secs_g, params, T, timeout):
    solver = Solver()
    solver.set('timeout', timeout)

    # Add semantic axioms
    T.semantic_axioms(solver)

    # Constrain params to this fiber
    for idx, fiber in enumerate(fiber_combo):
        solver.add(T.tag(params[idx]) == tag_constant(fiber))
        solver.add(T.tag(params[idx]) != T.TBot)

    # Check each section pair
    for sf in secs_f:
        for sg in secs_g:
            overlap = And(sf.guard, sg.guard)
            solver.push()
            solver.add(overlap)
            solver.add(sf.term != sg.term)
            result = solver.check()
            solver.pop()
            if result == sat:
                return INEQUIVALENT, solver.model()
            elif result == unsat:
                continue  # this pair agrees
    return EQUIVALENT, None
```

39.2 Step 2: Building C^0

The 0-cochain group is a vector over GF(2), one entry per site:

$$C^0 = \mathrm{GF}(2)^{|I|}$$

where entry i is 1 if site U_i is locally equivalent, 0 otherwise.

Listing 39.2: Building C^0

```
def build_c0(local_judgments):
    """Returns dict: site_id -> CochainElement."""
    c0 = {}
    for site_id, judgment in local_judgments.items():
        is_iso = (judgment.verdict == EQUIVALENT)
        c0[site_id] = CochainElement(
            sites=(site_id,),
            is_iso=is_iso,
            predicate=TruePred() if is_iso else FalsePred()
        )
    return CochainGroup(degree=0, elements=c0)
```

39.3 Step 3: Computing $\delta^0 : C^0 \to C^1$

The coboundary operator δ^0 computes transition functions on pairwise overlaps:

Listing 39.3: Coboundary δ^0

```
def delta_0(c0, overlaps):
    """Compute C^1 from C^0 and overlap list."""
    c1 = {}
    for (site_i, site_j) in overlaps:
        phi_i = c0.at(site_i)
        phi_j = c0.at(site_j)
        if phi_i is None or phi_j is None:
            continue
        # Transition: phi_j|_{ij} o (phi_i|_{ij})^{-1}
        # Both are truth values, so transition is trivial
        # iff both are isomorphisms
        is_trivial = phi_i.is_iso and phi_j.is_iso
        c1[(site_i, site_j)] = CochainElement(
            sites=(site_i, site_j),
            is_iso=is_trivial,
            predicate=TruePred() if is_trivial else FalsePred()
        )
    return CochainGroup(degree=1, elements=c1)
```

39.4 Step 4: Computing $\delta^1 : C^1 \to C^2$

The second coboundary checks the cocycle condition on triple overlaps:

Listing 39.4: Coboundary δ^1

```
def delta_1(c1, triple_overlaps):
    """Compute C^2 from C^1 and triple overlap list."""
    c2 = {}
    for (si, sj, sk) in triple_overlaps:
```

```
        psi_ij = c1.at(si, sj)
        psi_jk = c1.at(sj, sk)
        psi_ik = c1.at(si, sk)
        # Cocycle condition: psi_jk o psi_ij^{-1} o psi_ik
        # Over GF(2): sum mod 2
        vals = [psi_ij, psi_jk, psi_ik]
        non_trivial_count = sum(1 for v in vals
                                if v and not v.is_iso)
        is_cocycle = (non_trivial_count % 2 == 0)
        c2[(si, sj, sk)] = CochainElement(
            sites=(si, sj, sk),
            is_iso=is_cocycle
        )
    return CochainGroup(degree=2, elements=c2)
```

39.5 Step 5: Computing H^1

Listing 39.5: H^1 computation

```
def compute_h1(c0, c1, c2, overlaps, triple_overlaps):
    """Compute H^1 = ker(delta^1) / im(delta^0)."""
    # Build coboundary matrices over GF(2)
    # delta^0 matrix: rows = overlaps, cols = sites
    # delta^1 matrix: rows = triple overlaps, cols = overlaps

    n_sites = len(c0.elements)
    n_overlaps = len(overlaps)
    n_triples = len(triple_overlaps)

    # Compute rank of kernel and image
    rank_ker_d1 = n_overlaps - gf2_rank(delta1_matrix)
    rank_im_d0 = gf2_rank(delta0_matrix)

    h1_rank = rank_ker_d1 - rank_im_d0
    return h1_rank  # 0 means equivalence glues
```

39.6 The GF(2) Rank Algorithm

Listing 39.6: Gaussian elimination over GF(2)

```
def gf2_rank(matrix):
    """Row-reduce a binary matrix and return its rank."""
    m = [row[:] for row in matrix]  # copy
    rows, cols = len(m), len(m[0]) if m else 0
    rank = 0
    for col in range(cols):
        # Find pivot
        pivot = None
        for row in range(rank, rows):
            if m[row][col] == 1:
                pivot = row
                break
        if pivot is None:
            continue
        # Swap pivot row to position 'rank'
```

```
        m[rank], m[pivot] = m[pivot], m[rank]
        # Eliminate column in other rows
        for row in range(rows):
            if row != rank and m[row][col] == 1:
                m[row] = [(m[row][j] + m[rank][j]) % 2
                          for j in range(cols)]
        rank += 1
    return rank
```

39.7 Correctness Theorem

Theorem 39.1 (Correctness of the H^1 Algorithm)**.** *The algorithm computes the rank of* $H^1(\mathcal{U}, \mathsf{Iso}) = \ker(\delta^1)/\mathrm{im}(\delta^0)$ *over* GF(2) *correctly. Specifically:*

1. $\mathrm{rank}(H^1) = 0$ *iff all 1-cocycles are coboundaries iff local equivalences glue.*
2. $\mathrm{rank}(H^1) > 0$ *gives the number of independent obstructions.*
3. *The algorithm runs in* $O(n^3)$ *where* n *is the number of overlaps.*

Proof. Standard linear algebra over a finite field. The rank is the dimension of the quotient space, which equals the dimension of the kernel minus the dimension of the image. Gaussian elimination computes both dimensions in $O(n^3)$. □

39.8 The UserPropagator API

Z3's `UserPropagator` allows implementing custom theory solvers that participate in Z3's DPLL(T) solving loop:

Listing 39.7: UserPropagator skeleton

```
class MyProp(UserPropagateBase):
    def __init__(self, s=None, ctx=None):
        super().__init__(s, ctx)
        self.add_fixed(self._on_fixed)
        self.add_final(self._on_final)
        self.add_eq(self._on_eq)
        self.add_created(self._on_created)

    def push(self): ...    # save state
    def pop(self, n): ... # restore state
    def fresh(self, ctx): return MyProp(ctx=ctx)

    def _on_fixed(self, t, v): ...
    def _on_final(self): ...
    def _on_eq(self, a, b): ...
    def _on_created(self, t):
        self.add(t)  # register for tracking
```

39.9 PropagateFunction

A `PropagateFunction` is a Z3 function declaration that triggers the propagator's `_on_created` callback whenever a term using that function appears:

```
sem = PropagateFunction('sem', IntSort(), IntSort())
# When sem(x) appears, the propagator is notified
```

39.10 Planned Propagators

1. **Commutativity Propagator**: When Z3 assigns $\mathsf{op}(a, b) = v$, propagate $\mathsf{op}(b, a) = v$ for commutative operations.

2. **Fold Propagator**: Track fold terms and enforce permutation invariance for commutative+associative operations.

3. **Effect Propagator**: Distinguish mutation operations from pure operations to prevent false equivalences.

Part IX

Practical Engineering

Chapter 40

End-to-End Pipeline Trace

This chapter traces a *complete* equivalence check from raw Python source to final verdict, showing every intermediate step.

40.1 The Input Pair

Listing 40.1: Program A: recursive sum

```
def f(n):
    if n <= 0: return 0
    return n + f(n - 1)
```

Listing 40.2: Program B: iterative sum

```
def g(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total
```

40.2 Step 1: Parse to AST

Both sources are parsed by Python's `ast.parse`. Program A produces a `FunctionDef` with an `If` and a `Return` containing a `BinOp(Add)` with a recursive `Call`. Program B produces a `FunctionDef` with an `Assign`, a `For` over `range`, and a `Return`.

40.3 Step 2: Create Theory

A fresh `Theory` is instantiated:

- PyObj ADT created (7 constructors).
- binop, unop, truthy, fold RecFunctions defined.
- TypeTag EnumSort and tag RecFunction defined.
- Shared function registry initialized (empty).

40.4 Step 3: Compile Program A

Parameters: [`n`]. Create $p_0 = \mathsf{Const}(\texttt{p0}, \mathsf{PyObj})$. Env: $\{\texttt{n} \mapsto p_0\}$.

Detect self-recursion (`f` appears in body). Invoke Nat-eliminator extractor:

- Base guard: `n <= 0` $\Rightarrow$ threshold $= 0$.
- Base value: `0` $\Rightarrow \mathsf{IntObj}(0)$.
- Step: `n + f(n-1)` $\Rightarrow \mathsf{op} = \mathsf{ADD}$, param left, recursive call right.

Result: $[\![f]\!] = \mathsf{fold}(\mathsf{ADD}, 0, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(0))$.
Sections: $[(\top, \mathsf{fold}(\mathsf{ADD}, 0, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(0)))]$.

40.5 Step 4: Compile Program B

Parameters: [`n`]. Create p_0 (same as A).

`total = 0`: env $\leftarrow \{\texttt{n} \mapsto p_0, \texttt{total} \mapsto \mathsf{IntObj}(0)\}$.

`for i in range(1, n+1)`: extract range: $\mathsf{start} = 1$, $\mathsf{stop} = \mathsf{ival}(p_0) + 1$.

Loop body: `total += i` is `AugAssign(Add)`, target `total`, value `i` (the loop variable). This matches the canonical fold pattern: single accumulator, augmented by loop var.

Result: env[`total`] $\leftarrow \mathsf{fold}(\mathsf{ADD}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(0))$.

`return total`: section $(\top, \mathsf{fold}(\mathsf{ADD}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(0)))$.

Note: A starts fold at 0, B starts at 1. But $\mathsf{fold}(\mathsf{ADD}, 0, n{+}1, \mathsf{IntObj}(0))$ includes $\mathsf{binop}(\mathsf{ADD}, \mathsf{IntObj}(0), \ldots) = \ldots$ (adding 0 is identity by R3). So both compute $0 + 1 + 2 + \ldots + n$.

40.6 Step 5: Unify Parameters

Both have 1 parameter. p_0 is shared (same Z3 Const). No substitution needed.

40.7 Step 6: Duck Type Inference

Analyze both programs for operations on `n`:

- A: comparison (`n <= 0`), subtraction (`n-1`), addition (`n + f(n-1)`).
- B: `range(1, n+1)` (addition).

Union of operations: $\{\mathsf{le}, \mathsf{sub}, \mathsf{add}, \mathsf{call_range}\}$. Numeric operations present $\Rightarrow$ duck type = `int`.

40.8 Step 7: Build Site Category

Single parameter, single type (int). Site category:

- One site: $U_{\mathsf{int}} = (\mathsf{TInt})$.
- No overlaps (single site).

40.9 Step 8: Local Equivalence Check

Solver created. Semantic axioms added (none relevant — no shared builtins used). Tag constraint: $\mathsf{tag}(p_0) = \mathsf{TInt}$, $\mathsf{tag}(p_0) \neq \mathsf{TBot}$.

Section comparison: A has 1 section, B has 1 section. Overlap: $\top \wedge \top = \top$ (always overlapping).

Z3 query: $\mathsf{fold}(\mathsf{ADD}, 0, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(0)) \neq \mathsf{fold}(\mathsf{ADD}, 1, \mathsf{ival}(p_0) + 1, \mathsf{IntObj}(0))$?

Z3 unfolds the fold RecFunction: the first fold includes $i = 0$ (adding $\mathsf{IntObj}(0)$, which is identity by Z3's integer arithmetic). Both produce the same sum.

Result: **UNSAT**. Local equivalence holds.

40.10 Step 9: Čech Cohomology

Single site, no overlaps. $C^0 = \{1\}$ (one local isomorphism). $C^1 = \emptyset$ (no overlaps). $H^1 = 0$ trivially.

40.11 Step 10: Verdict

$H^1 = 0$, $H^0 = 1$.

EQUIVALENT: $f \equiv g$. $\mathrm{H}^0 = 1$ section verified across 1 type fiber. The cubical path is: $[\![f]\!] \xrightarrow{\mathsf{D1}} \mathsf{fold}(\mathsf{ADD}, \ldots) = [\![g]\!]$.

Chapter 41

The Fallback Strategy: When No Dichotomy Applies

41.1 The Problem

For 8 of the 50 equivalent benchmark pairs, none of the 24 dichotomies directly applies. These are genuinely different algorithms for the same problem:

- Graham scan vs. monotone chain (convex hull)
- Tarjan vs. Kosaraju (SCC)
- Matrix exponentiation vs. fast doubling (Fibonacci)
- Multiplicative formula vs. Pascal's triangle ($\binom{n}{k}$)

The path search normalizes both programs, but the normal forms are *different* (different algorithms produce different Z3 terms). What now?

41.2 Fallback 1: Specification Discovery

The system attempts to *automatically discover* a specification that both programs satisfy:

Listing 41.1: Automatic specification discovery

```
def discover_spec(term_f, term_g, params, theory, solver):
    """Try to find a spec S such that both f and g satisfy S.

    Strategy: extract observable properties of both terms
    and check if they match.
    """
    properties = []

    # Property 1: output type (tag of return value)
    tag_f = theory.tag(term_f)
    tag_g = theory.tag(term_g)
    solver.push()
    solver.add(tag_f != tag_g)
    if solver.check() == unsat:
        properties.append(('same_output_type', tag_f))
    solver.pop()

    # Property 2: monotonicity
    p_inc = Const('_p_inc', theory.S)
```

```
    solver.push()
    solver.add(theory.binop(GT, p_inc, params[0]))
    solver.add(Not(theory.binop(GE,
        substitute(term_f, (params[0], p_inc)),
        term_f)))
    if solver.check() == unsat:
        properties.append(('monotone_increasing',))
    solver.pop()

    # Property 3: idempotence (f(f(x)) == f(x))
    ff = substitute(term_f, (params[0], term_f))
    solver.push()
    solver.add(ff != term_f)
    if solver.check() == unsat:
        properties.append(('idempotent',))
    solver.pop()

    # Property 4: base case agreement
    for base_val in [0, 1, -1]:
        base_term = theory.mkint(base_val)
        f_at_base = simplify(substitute(term_f,
            (params[0], base_term)))
        g_at_base = simplify(substitute(term_g,
            (params[0], base_term)))
        solver.push()
        solver.add(f_at_base != g_at_base)
        if solver.check() == unsat:
            properties.append(('agree_at', base_val))
        solver.pop()

    return properties
```

If both programs agree on base cases AND have the same output type AND satisfy the same recurrence (detected by the 2D RecFunction analysis from Chapter 29), they satisfy the same deterministic specification by Theorem 24.2.

41.3 Fallback 2: Shared Symbol Unification

Even when the algorithms differ, they may call the same builtins. The shared function symbols create implicit equations:

If A uses py_sorted(...) and B uses py_sorted(...), Z3 knows they use the *same* function. The remaining question is whether the arguments match.

The system tries progressively deeper argument matching:

1. Syntactic: are the arguments identical Z3 terms?
2. Axiom-assisted: does applying path constructors to the arguments make them match?
3. Fiber-restricted: do the arguments match on each type fiber individually?

41.4 Fallback 3: Graded Verdict

When no proof strategy succeeds, the system returns a *graded verdict* instead of a binary answer:

Listing 41.2: Graded fallback verdict

```
def graded_verdict(term_f, term_g, params, theory, sites):
```

```
    """Return a graded verdict when full proof fails."""
    # How many fibers agree?
    agree = 0
    disagree = 0
    unknown = 0

    for combo, site_id in sites.items():
        result = check_local(combo, term_f, term_g, ...)
        if result == EQUIVALENT: agree += 1
        elif result == INEQUIVALENT: disagree += 1
        else: unknown += 1

    total = agree + disagree + unknown
    if disagree > 0:
        return Verdict(INEQUIVALENT,
            confidence=disagree/total,
            evidence=counterexamples)
    elif agree == total:
        return Verdict(EQUIVALENT,
            confidence=1.0,
            evidence='all fibers verified')
    else:
        return Verdict(UNKNOWN,
            confidence=agree/total,
            note=f'{agree}/{total} fibers verified, '
                 f'{unknown} timed out')
```

This gives the user actionable information even when the system can't produce a definitive proof.

Chapter 42

OOP: Classes, Inheritance, and Method Resolution

42.1 Class Compilation

Python classes compile to Ref objects with shared method symbols:

Listing 42.1: Class compilation

```
# class MyClass:
#     def __init__(self, x):
#         self.x = x
#     def compute(self):
#         return self.x * 2

# Compilation:
# MyClass(v)  => Ref(fresh_addr)
# obj.x       => py_attr_x(obj)
# obj.compute() => py_meth_compute(obj)
#               => inlined: py_attr_x(obj) * 2

def compile_class(cls_node, env, T):
    env.put(cls_node.name, ('__class__', cls_node))
    # Methods are stored for inlining
    for item in cls_node.body:
        if isinstance(item, ast.FunctionDef):
            env.put(f'{cls_node.name}.{item.name}',
                    ('__func__', item))
```

42.2 Inheritance as Fiber Refinement

A class hierarchy $A \leftarrow B \leftarrow C$ (C inherits B inherits A) defines a *refinement* of the type fibration: the fiber TRef is refined into sub-fibers for each class.

Definition 42.1 (Class Fiber)**.** For class C, the class fiber $U_C = \{v : \mathsf{PyObj} \mid \mathsf{tag}(v) = \mathsf{TRef} \wedge \mathsf{class_of}(v) = C\}$.

The method resolution order (MRO) determines which method is called: $\mathsf{meth_m}(v) = \mathsf{mro}(C)[0].\mathsf{m}(v)$.

Both programs using the same class hierarchy get the same MRO, hence the same method dispatch, hence the same shared symbols for each method call.

42.3 Descriptors and Properties

`@property` compiles to a shared function: obj.prop → py_prop_prop(obj). Since both programs access the same property, they get the same shared symbol.

Chapter 43

Error Handling and Graceful Degradation

43.1 Unsupported Syntax

When the AST compiler encounters a construct it cannot handle (e.g., `async/await`, `match/case`, `global`, `exec`), it produces a *fresh uninterpreted constant*: FreshConst(PyObj, `unsupported`).

This is *sound*: the fresh constant is unique, so two programs with different unsupported constructs produce different terms (no false equivalences). It is *incomplete*: two programs with the *same* unsupported construct also produce different fresh constants (false inequivalence).

43.2 Z3 Timeout Handling

When Z3 times out on a fiber:

1. Mark the fiber as UNKNOWN.
2. Continue checking other fibers.
3. In the H^1 computation, treat UNKNOWN fibers as "not verified" (not as failures).
4. Report a partial verdict: "Equivalent on k/n fibers, m fibers timed out."

Listing 43.1: Timeout handling

```
def check_with_timeout(fiber, secs_f, secs_g, solver, timeout):
    solver.set('timeout', timeout)
    solver.push()
    # ... add constraints ...
    result = solver.check()
    solver.pop()
    if result == unknown:
        # Try with longer timeout on just this fiber
        solver.set('timeout', timeout * 3)
        result = solver.check()
    return result
```

43.3 Combinatorial Explosion in Site Category

When duck type inference allows k types per parameter and there are n parameters, the number of fiber combinations is k^n. Mitigation:

1. Cap at 50 fiber combinations (skip rare combinations).
2. Prioritize fibers by duck type confidence (high-confidence single-type fibers first).
3. Use the spectral sequence (Chapter 13) to check important fibers first and skip if degeneration is detected.

43.4 Partial Verdicts and Confidence Levels

Definition 43.1 (Confidence Level)**.** $\mathsf{confidence}(f, g) = \frac{\text{fibers verified equivalent}}{\text{total non-timed-out fibers}}$

A confidence of 1.0 means full equivalence proof. A confidence of 0.8 means 80% of fibers verified, 20% timed out. A confidence of 0.0 with counterexample means definite inequivalence.

Chapter 44

Scalability

44.1 Complexity of the Full Pipeline

Step	Complexity
AST parsing	$O(\|S\|)$ (linear in source size)
AST compilation	$O(\|A\|)$ (linear in AST size)
Duck type inference	$O(\|A\|)$ (single AST walk)
Site construction	$O(k^n)$ (exponential in params)
Local checks	$O(k^n \cdot C_{\mathsf{Z3}})$
H^1 computation	$O(k^{3n})$ (GF(2) rank)
Total	$O(k^n \cdot C_{\mathsf{Z3}} + k^{3n})$

For the common case ($k = 1$, single-typed parameters): $O(C_{\mathsf{Z3}})$ — a single Z3 query.

For the benchmark ($k \leq 2$, $n \leq 4$): $O(16 \cdot C_{\mathsf{Z3}} + 4096)$ — at most 16 Z3 queries plus cheap linear algebra. Runs in < 3 seconds.

44.2 Z3 Query Complexity

The Z3 query complexity C_{Z3} depends on:

- **Term depth**: deeper Z3 terms (from deeper nesting or longer loops) produce harder queries. Fold RecFunctions add depth proportional to the fold range.
- **Number of sections**: more branches produce more section pairs to compare. m sections in A $\times$ n sections in B $= m \cdot n$ comparisons.
- **Axiom count**: more ForAll axioms slow down E-matching. We add axioms only for shared functions actually used (axiom relevance filtering).

Typical Z3 query time: 1–50ms for ground queries (no RecFunctions), 50–500ms with RecFunctions, 500ms–5s with axioms.

44.3 Heuristics for Large Programs

For programs with 100+ lines:

1. **Function-level decomposition**: verify each function independently, then compose results (Theorem 11.18).
2. **Incremental checking**: check modified functions only, reuse cached results for unchanged ones.

3. **Early termination**: if any fiber produces a counterexample, return INEQUIVALENT immediately (no need to check remaining fibers).

4. **Timeout budget**: allocate a total timeout budget (e.g., 10s) and divide across fibers. Skip low-priority fibers if budget is exhausted.

Appendix A

Mathematical Prerequisites

A.1 Category Theory Essentials

A *category* $\mathcal{C}$ consists of objects, morphisms, composition, and identity, satisfying associativity and unit laws. A *functor* $F : \mathcal{C} \to \mathcal{D}$ preserves composition and identity. A *natural transformation* $\eta : F \Rightarrow G$ is a family of morphisms $\eta_A : F(A) \to G(A)$ commuting with all morphisms.

Key concepts used:

- **Initial algebra**: The initial object in the category of F-algebras for an endofunctor F. Lists are the initial algebra for $F(X) = 1 + A \times X$. The unique morphism from the initial algebra is the *catamorphism* (fold).
- **Grothendieck construction**: Turns a pseudofunctor $F : \mathcal{C}^{\mathrm{op}} \to \mathbf{Cat}$ into a fibered category $\int F$ over $\mathcal{C}$. Our type fibration is a Grothendieck construction.
- **Yoneda lemma**: $\mathrm{Nat}(\mathrm{Hom}(A, -), F) \cong F(A)$. Functions on representable presheaves are determined by their value at the representing object.

A.2 Sheaf Theory Essentials

A *site* is a category with a Grothendieck topology (covering families). A *presheaf* is a contravariant functor to **Set**. A *sheaf* satisfies the gluing axiom: local sections that agree on overlaps glue uniquely.

The *Čech complex* computes cohomology from a covering: $C^p = \prod \mathcal{F}(U_{i_0 \cdots i_p})$ with coboundary operators δ^p. $H^1 = \ker \delta^1 / \mathrm{im}\, \delta^0$ measures obstruction to gluing.

A.3 Homotopy Type Theory Essentials

Types are spaces. Terms are points. Equalities are paths. $\mathsf{Path}_A(a, b) = \{p : \mathbb{I} \to A \mid p(0) = a, p(1) = b\}$. Function extensionality: $(f = g) \simeq \prod_x f(x) = g(x)$. Univalence: $(A \simeq B) \simeq (A =_{\mathsf{Type}} B)$. HITs: types with point + path constructors.

A.4 SMT Solving Essentials

Z3 decides satisfiability of first-order formulas in theories (arithmetic, strings, arrays, datatypes). Key features: algebraic datatypes, recursive functions, quantifiers (E-matching + MBQI), enumeration sorts, custom theories (UserPropagator).

Appendix B

Benchmark Pair Catalog

The complete catalog of 50 equivalent and 50 non-equivalent benchmark pairs, with dichotomy classification, is maintained in the repository at `benchmarks/hard_equiv_suite.py` and `tests/test_equivalen`

Each pair consists of two Python functions (20–60 lines each) that exercise deep Python semantics. The equivalent pairs span all 24 dichotomies; the non-equivalent pairs test subtle behavioral differences including mutation, aliasing, closure binding, NaN handling, and edge cases.

Bibliography

[1] C. Cohen, T. Coquand, S. Huber, A. Mörtberg. *Cubical Type Theory: A Constructive Interpretation of the Univalence Axiom.* Journal of Automated Reasoning, 2018.

[2] T. Coquand, S. Huber, A. Mörtberg. *On Higher Inductive Types in Cubical Type Theory.* LICS 2018.

[3] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics.* Institute for Advanced Study, 2013.

[4] C. Angiuli, R. Harper, T. Wilson. *Computational Higher-Dimensional Type Theory.* POPL 2018.

[5] V. Voevodsky. *Univalent Foundations of Mathematics.* WoLLIC 2010.

[6] L. de Moura, N. Bjørner. *Z3: An Efficient SMT Solver.* TACAS 2008.

[7] J. G. Politz et al. *Python: The Full Monty.* OOPSLA 2013.

[8] A. Grothendieck. *Sur quelques points d'algèbre homologique.* Tôhoku Math. J., 1957.

[9] J. C. Reynolds. *Definitional Interpreters for Higher-Order Programming Languages.* ACM Annual Conference, 1972.

[10] A. Pnueli, M. Siegel, E. Singerman. *Translation Validation.* TACAS 1998.

[11] B. Godlin, O. Strichman. *Regression Verification.* DAC 2009.

[12] G. Barthe, J. M. Crespo, C. Kunz. *Relational Verification Using Product Programs.* FM 2011.

[13] P. Cousot, R. Cousot. *Abstract Interpretation.* POPL 1977.

[14] P. M. Rondon, M. Kawaguchi, R. Jhala. *Liquid Types.* PLDI 2008.

[15] P. L. Lumsdaine. *Higher Inductive Types: A Tour.* 2011.

[16] M. Shulman. *Brouwer's Fixed-Point Theorem in Real-Cohesive Homotopy Type Theory.* MSCS 2018.

[17] J. Curry. *Sheaves, Cosheaves and Applications.* PhD thesis, 2014.

[18] S. Abramsky. *Domain Theory in Logical Form.* PhD thesis, 1987.